

PENGEMBANGAN STATIFY: APLIKASI ANALISIS

STATISTIK

(Modul Analisis *Time Series*)

GUSWANA ADVENTUS

222112082

**PROGRAM STUDI : PROGRAM STUDI KOMPUTASI STATISTIK
PROGRAM DIPLOMA IV
PEMINATAN : SISTEM INFORMASI STATISTIK**



POLITEKNIK STATISTIKA STIS

JAKARTA

2025

PENGEMBANGAN STATIFY: APLIKASI ANALISIS

STATISTIK

(Modul Analisis *Time Series*)

SKRIPSI

Diajukan sebagai Salah Satu Syarat untuk Memperoleh Sebutan

Sarjana Terapan Ststistika pada Politeknik Statistika STIS

Oleh:

GUSWANA ADVENTUS

222112082



POLITEKNIK STATISTIKA STIS

JAKARTA

2025

PERNYATAAN
Skripsi dengan Judul
PENGEMBANGAN STATIFY: APLIKASI ANALISIS
STATISTIK
(Modul Analisis *Time Series*)

Oleh:
GUSWANA ADVENTUS
222112082

adalah benar-benar hasil penelitian sendiri dan bukan hasil plagiat atau hasil karya orang lain. Jika di kemudian hari diketahui ternyata skripsi ini hasil plagiat atau hasil karya orang lain, penulis bersedia skripsi ini dinyatakan tidak sah dan sebutan Sarjana Terapan Statistika dicabut atau dibatalkan.

Jakarta, 5 Agustus 2025



Guswana Adventus

PENGEMBANGAN STATIFY: APLIKASI ANALISIS

STATISTIK

(Modul Analisis *Time Series*)

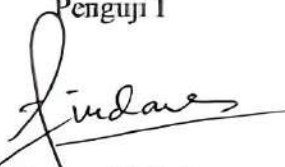
Oleh:

GUSWANA ADVENTUS

222112082

Tim Penguji

Penguji I


Firdaus, M.B.A.
NIP 197205261991121001

Penguji II


Anugerah Karta Monika, S.Si. M.E.
NIP 197502041996122001

Mengetahui/Menyetujui

Ketua Program Studi Komputasi Statistik
Program Diploma IV



Ibnu Santoso, S.S.T., M.T.
NIP 198601202008011002

Pembimbing


Ibnu Santoso, S.S.T., M.T.
NIP 198601202008011002

© Hak Cipta milik Politeknik Statistika STIS, Tahun 2025

Hak Cipta dilindungi undang-undang

1. *Dilarang mengutip sebagian atau seluruh karya tulis, hasil analisis, perancangan, basis data, program, dan artefak hasil skripsi ini tanpa mencantumkan atau menyebutkan sumbernya.*
 - a. *Pengutipan hanya untuk kepentingan pendidikan, penelitian, penulisan karya ilmiah, penyusunan laporan, penulisan kritik atau tinjauan suatu masalah.*
 - b. *Pengutipan tidak merugikan kepentingan yang wajar Politeknik Statistika STIS.*
2. *Dilarang mengumpulkan dan memperbanyak sebagian atau seluruh karya tulis, hasil analisis, perancangan, basis data, program, dan artefak hasil skripsi ini dalam bentuk apapun tanpa seizin Politeknik Statistika STIS.*

PRAKATA

Puji syukur penulis ucapkan kepada Tuhan Yang Maha Esa, karena atas izin-Nya penulis dapat menyelesaikan skripsi yang berjudul “Pengembangan Statify: Aplikasi Analisis Statistik Modul Analisis *Time Series*”. Penulis juga mengucapkan terima kasih kepada:

1. Ibu Dr. Erni Tri Astuti, M. Math., selaku Direktur Politeknik Statistika STIS;
2. Bapak Ibnu Santoso, SST., M.T., selaku Ketua Program Studi D-IV Komputasi Statistik Politeknik Statistika STIS dan selaku dosen pembimbing yang telah bersedia meluangkan waktu dalam membimbing penyusunan skripsi ini;
3. Bapak Firdaus, M.B.A. dan Ibu Anugerah Karta Monika, S.Si, M.E., selaku dosen penguji atas koreksi dan saran yang disampaikan;
4. Dr. Jurek Kirakowski yang telah memberikan *academic licence* untuk evaluasi *usability testing* Statify Modul Analisis *Time Series* menggunakan *System Usability Measurement Inventory (SUMI)*;
5. Mama, Papa, serta keluarga besar saya yang telah memberikan banyak dukungan secara moril;
6. Tim Statify dan semua pihak yang telah memberikan dukungan dalam penulisan skripsi ini.

Penulis menyadari skripsi ini masih mempunyai kekurangan, baik dari isi maupun susunannya. Oleh karena itu, saran dan kritik yang membangun sangat penulis harapkan demi perbaikan skripsi ini. Semoga skripsi ini dapat bermanfaat bagi semua pihak.

Jakarta, 5 Agustus 2025

Guswana Adventus

ABSTRAK

GUSWANA ADVENTUS, “Pengembangan Statify: Aplikasi Analisis Statistik Modul Analisis *Time Series*”.

x+196 halaman

Analisis Runtun Waktu (*Time Series*) merupakan salah satu mata kuliah yang diajarkan di lingkungan Politeknik Statistika STIS. Aplikasi Eviews, SPSS, dan R menjadi aplikasi yang sering digunakan dalam pembelajaran Analisis Runtun Waktu. Namun, pemanfaatan aplikasi tersebut belum optimal karena metode analisis yang tersedia belum terintegrasi dengan alur materi Analisis Runtun Waktu. Selain itu, keterbatasan fitur eksplorasi awal pada SPSS dan ketiadaan otomatisasi visualisasi grafik pada Eviews, tentunya cukup menyulitkan mahasiswa dalam melakukan tahapan analisis secara berkesinambungan dan terstruktur. Dengan demikian, diperlukan aplikasi statistik yang mampu menyelesaikan permasalahan tersebut. Sebagai solusi maka dikembangkan Statify Modul Analisis *Time Series*, aplikasi berbasis web yang responsif dengan pendekatan optimasi *client-side computing*. Aplikasi ini dikembangkan menggunakan *framework* next.js dan teknologi WebAssembly. Adapun, Statify dikembangkan menggunakan metode pengembangan *Feature-Driven Development* (FDD). Selanjutnya aplikasi ini dievaluasi menggunakan metode *white-box testing* dengan teknik *basis path* dan diperoleh hasil bahwa algoritma penghitungan statistik Statify sudah berjalan dengan benar. Selain itu, evaluasi dilanjutkan dengan *usability testing* yang dilakukan oleh pengguna dengan bantuan instrumen penilaian kuesioner *Software Usability Measurement Inventory* (SUMI). Adapun hasil diperoleh dari evaluasi tersebut adalah Statify Modul Analisis *Time Series* memperoleh nilai di atas 50 poin untuk setiap aspek penilaian yang berarti Statify memiliki usability yang cukup baik.

Kata kunci: *Time Series*, FDD, *White-Box Testing*, *Usability Testing*, SUMI

DAFTAR ISI

	Halaman
PRAKATA.....	i
ABSTRAK	ii
DAFTAR ISI.....	iii
DAFTAR TABEL	v
DAFTAR GAMBAR	vi
DAFTAR LAMPIRAN	x
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Identifikasi dan Batasan Masalah.....	4
1.3 Tujuan Penelitian	5
1.4 Manfaat Penelitian	6
1.5 Sistematika Penulisan	8
BAB II KAJIAN PUSTAKA	11
2.1 Landasan Teori.....	11
2.2 Penelitian Terkait	35
2.3 Kerangka Pikir	38
BAB III METODE PENELITIAN.....	41
3.1 Tinjauan Umum	41
3.1.1 Metode Pengembangan	41
3.1.2 Metode Evaluasi.....	42
3.2 Tahapan Penelitian	43
3.2.1 Tahapan Feature-Driven Development (FDD)	43
3.2.2 Tahapan White-Box Testing	47

3.2.3 Tahapan Usability Testing.....	48
BAB IV ANALISIS DAN PERANCANGAN.....	51
4.1 Analisis	51
4.1.1 Kebutuhan Umum Pengguna.....	51
4.1.2 Kebutuhan Fungsional.....	54
4.1.3 Kebutuhan Non-Fungsional.....	59
4.1.4 Daftar Fitur	60
4.2 Perancangan.....	64
4.2.1 Perencanaan Jadwal Pengembangan Fitur.....	64
4.2.2 Sequence Diagram.....	67
4.2.3 Desain Antarmuka Pengguna	73
BAB V IMPLEMENTASI DAN EVALUASI.....	79
5.1 Implementasi	79
5.1.1 Lingkungan Implementasi	79
5.1.2 Struktur Proyek.....	80
5.1.3 Implementasi Fitur.....	82
5.2 Evaluasi	106
5.2.1 <i>White-Box Testing</i>	106
5.2.2 <i>Usability Testing</i>	151
BAB VI KESIMPULAN DAN SARAN.....	157
6.1 Kesimpulan.....	157
6.2 Saran	158
DAFTAR PUSTAKA.....	161
LAMPIRAN	165
RIWAYAT HIDUP	197

DAFTAR TABEL

No. Tabel	Judul Tabel	Halaman
1.	Interpretasi <i>T-Score</i> Kuesioner SUMI	50
2.	Kebutuhan Fungsional Statify Modul Analisis <i>Time Series</i>	57
3.	Kebutuhan Non-fungsional Statify Modul Analisis <i>Time Series</i>	59
4.	Daftar Fitur Statify Modul Analisis <i>Time Series</i>	60
5.	Daftar Simpul Menu <i>Smoothing</i>	107
6.	Hasil <i>Test Cases</i> Menu <i>Smoothing</i>	115
7.	Daftar Simpul Menu <i>Decomposition</i>	121
8.	Hasil <i>Test Cases</i> Menu <i>Decomposition</i>	123
9.	Daftar Simpul Menu <i>Autocorrelation</i>	126
10.	Hasil <i>Test Cases</i> Menu <i>Autocorrelation</i>	129
11.	Daftar Simpul Menu <i>Unit Root Test</i>	133
12.	Hasil <i>Test Cases</i> Menu <i>Unit Root Test</i>	140
13.	Daftar Simpul Menu <i>Box-Jenkins Model</i>	145
14.	Hasil <i>Test Cases</i> Menu <i>Box-Jenkins Model</i>	147

DAFTAR GAMBAR

No. Gambar	Judul Gambar	Halaman
1.	Tahapan FDD	30
2.	Kerangka Pikir Penelitian.....	38
3.	Contoh <i>Class Diagram</i>	44
4.	Contoh <i>Use Case Diagram</i>	45
5.	Contoh <i>Sequence Diagram</i>	46
6.	<i>Class Diagram</i> Statify	52
7.	<i>Use Case Diagram</i> Statify Keseluruhan.....	52
8.	<i>Use Case Diagram</i> Statify Analisis Data	52
9.	<i>Use Case Diagram</i> Statify Modul Analisis <i>Time Series</i>	54
10.	<i>Gantt Chart</i> Statify Modul Analisis <i>Time Series</i> Desember 2024	65
11.	<i>Gantt Chart</i> Statify Modul Analisis <i>Time Series</i> Januari 2025.....	65
12.	<i>Gantt Chart</i> Statify Modul Analisis <i>Time Series</i> Februari 2025.....	66
13.	<i>Gantt Chart</i> Statify Modul Analisis <i>Time Series</i> Maret 2025.....	66
14.	<i>Gantt Chart</i> Statify Modul Analisis <i>Time Series</i> April 2025	66
15.	<i>Gantt Chart</i> Statify Modul Analisis <i>Time Series</i> Mei 2025	66
16.	<i>Sequence Diagram</i> Kegiatan Mengakses Menu <i>Time Series</i>	67
17.	<i>Sequence Diagram</i> Kegiatan Mengatur Variabel.....	68
18.	<i>Sequence Diagram</i> Kegiatan Mengatur Opsi Menu <i>Time Series</i>	69
19.	<i>Sequence Diagram</i> Kegiatan Mengatur Opsi Pengaturan Waktu	70
20.	<i>Sequence Diagram</i> Kegiatan Menyimpan Histori Pengaturan.....	71
21.	<i>Sequence Diagram</i> Kegiatan Menghitung <i>Output</i> Menu <i>Time Series</i>	72
22.	<i>Mockup</i> Rancangan Dasar Desain Modal Statify	73
23.	<i>Mockup Variable Tab</i> Menu <i>Time Series</i> Dengan <i>Checkbox</i> Menyimpan <i>Forecasting</i> Sebagai Variabel	74
24.	<i>Mockup Variable Tab</i> Menu <i>Time Series</i> Tanpa <i>Checkbox</i> Menyimpan <i>Forecasting</i> Sebagai Variabel	75
25.	<i>Mockup Time Tab</i> Menu <i>Time Series</i>	75
26.	<i>Mockup Option Tab</i> Menu <i>Smoothing</i>	76
27.	<i>Mockup Option Tab</i> Menu <i>Decomposition</i>	76
28.	<i>Mockup Option Tab</i> Menu <i>Autocorrelation</i>	77

29. <i>Mockup Option Tab Menu Unit Root Test</i>	77
30. <i>Mockup Option Tab Menu Box-Jenkins Model</i>	78
31. Struktur Proyek Statify Modul Analisis <i>Time Series</i>	81
32. Struktur Folder <i>Source Code</i> Antarmuka Modal Menu <i>Time Series</i>	81
33. Struktur Folder <i>Source Code</i> Penghitungan Menu <i>Time Series</i>	81
34. Implementasi Mengakses Menu <i>Smoothing</i>	83
35. Implementasi Mengonfigurasi Variabel Menu <i>Smoothing</i>	84
36. Implementasi Mengonfigurasi Opsi Menu <i>Smoothing</i>	84
37. Implementasi Mengonfigurasi Waktu Menu <i>Smoothing</i>	85
38. Implementasi Potongan Kode Program Menu <i>Smoothing</i>	86
39. Implementasi Melihat Variabel Baru Menu <i>Smoothing</i>	86
40. Implementasi Tabel Analisis Menu <i>Smoothing</i>	87
41. Implementasi Grafik <i>Smoothing</i>	87
42. Implementasi Mengakses Menu <i>Decomposition</i>	88
43. Implementasi Mengonfigurasi Variabel Menu <i>Decomposition</i>	89
44. Implementasi Mengonfigurasi Opsi Menu <i>Decomposition</i>	90
45. Implementasi Mengonfigurasi Waktu Menu <i>Decomposition</i>	90
46. Implementasi Potongan Kode Program Menu <i>Decomposition</i>	91
47. Implementasi Melihat Variabel Baru Hasil Dekomposisi	92
48. Implementasi Tabel Analisis Menu <i>Decomposition</i>	92
49. Implementasi Grafik Komponen Seasonal Hasil <i>Decomposition</i>	93
50. Implementasi Mengakses Menu <i>Autocorrelation</i>	94
51. Implementasi Mengonfigurasi Variabel Menu <i>Autocorrelation</i>	94
52. Implementasi Mengonfigurasi Opsi Menu <i>Autocorrelation</i>	95
53. Implementasi Mengonfigurasi Waktu Menu <i>Autocorrelation</i>	96
54. Implementasi Potongan Kode Program Menu <i>Autocorrelation</i>	96
55. Implementasi Tabel Analisis Menu <i>Autocorrelation</i>	97
56. Implementasi Grafik Koeogram Menu <i>Autocorrelation</i>	97
57. Implementasi Mengakses Menu <i>Unit Root Test</i>	98
58. Implementasi Mengonfigurasi Variabel Menu <i>Unit Root Test</i>	99
59. Implementasi Mengonfigurasi Opsi Menu <i>Unit Root Test</i>	99
60. Implementasi Potongan Kode Program Menu <i>Unit Root Test</i>	100
61. Implementasi Tabel Analisis Menu <i>Unit Root Test</i>	101
62. Implementasi Mengakses Menu <i>Box-Jenkins Model</i>	102

63. Implementasi Mengonfigurasi Variabel Menu <i>Box-Jenkins Model</i>	102
64. Implementasi Mengonfigurasi Opsi Menu <i>Box-Jenkins Model</i>	103
65. Implementasi Mengonfigurasi Waktu Menu <i>Box-Jenkins Model</i>	104
66. Implementasi Potongan Kode Program Menu <i>Box-Jenkins Model</i>	104
67. Implementasi Melihat Variabel Baru Menu <i>Box-Jenkins Model</i>	105
68. Implementasi Tabel Analisis <i>Box-Jenkins Model</i>	106
69. Implementasi Grafik <i>Forecasting Box-Jenkins Model</i>	106
70. <i>Flow Graph</i> Menu <i>Smoothing</i> Metode <i>Simple Moving Average</i>	109
71. <i>Flow Graph</i> Menu <i>Smoothing</i> Metode <i>Double Moving Average</i>	110
72. <i>Flow Graph</i> Menu <i>Smoothing</i> Metode <i>Simple Exponential Smoothing</i>	111
73. <i>Flow Graph</i> Menu <i>Smoothing</i> Metode <i>Double Exponential Smoothing</i>	112
74. <i>Flow Graph</i> Menu <i>Smoothing</i> Metode <i>Holt's Method Exponential Smoothing</i>	113
75. <i>Flow Graph</i> Menu <i>Smoothing</i> Metode <i>Winter's Method Exponential Smoothing</i>	114
76. Grafik <i>Moving Average</i> dengan Periode 3	118
77. Grafik <i>Simple Exponential Smoothing</i> dengan Alpa 0.1	118
78. Grafik <i>Double Exponential Smoothing</i> dengan Alpa 0.1	119
79. Grafik <i>Holt's Method Exponential Smoothing</i> dengan Alpa 0.1 dan Beta 0.1	119
80. Grafik <i>Winter's Method Exponential Smoothing</i> dengan Alpa 0.1, Beta 0.1, dan Gama 0.1.	120
81. <i>Flow Graph</i> Menu <i>Decomposition</i>	122
82. Grafik <i>Trend Component</i>	125
83. Grafik <i>Seasonal Component</i>	125
84. Grafik <i>Irregular Component</i>	125
85. <i>Flow Graph</i> Menu <i>Autocorrelation</i> Tanpa Musim	127
86. <i>Flow Graph</i> Menu <i>Autocorrelation</i> Dengan Musim.....	128
87. Tabel Autokorelasi pada Data Aktual dengan 20 <i>Lag</i> Data	131
88. Tabel Autokorelasi pada Data Diferensiasi Pertama dengan 20 <i>Lag</i> Data...	131
89. Tabel Autokorelasi pada Data Diferensiasi Kedua dengan 20 <i>Lag</i> Data	132
90. <i>Flow Graph</i> Menu <i>Unit Root Test</i> Metode <i>Dickey Fuller</i> Tanpa Diferensiasi	134
91. <i>Flow Graph</i> Menu <i>Unit Root Test</i> Metode <i>Dickey-Fuller</i> Diferensiasi Pertama	135

92. <i>Flow Graph Menu Unit Root Test Metode Dickey-Fuller Diferensiasi Kedua</i>	136
93. <i>Flow Graph Menu Unit Root Test Metode Augmented Dickey-Fuller Tanpa Diferensiasi</i>	137
94. <i>Flow Graph Menu Unit Root Test Metode Augmented Dickey-Fuller Diferensiasi Pertama</i>	138
95. <i>Flow Graph Menu Unit Root Test Metode Augmented Dickey-Fuller Diferensiasi Kedua</i>	139
96. <i>Tabel Hasil Dickey-Fuller Test Data Aktual dengan Intersep</i>	143
97. <i>Tabel Hasil Augmented Dickey-Fuller Test Data Aktual dengan Intersep dan 2 Lag Data</i>	144
98. <i>Flow Graph Menu Box-Jenkins Model</i>	146
99. <i>Tabel Hasil Model AR dengan Parameter Order 2</i>	149
100. <i>Tabel Hasil Model MA dengan Parameter Order 2</i>	149
101. <i>Tabel Hasil Model ARMA dengan Parameter Order 2,1</i>	150
102. <i>Tabel Hasil Model ARIMA dengan Parameter Order 2,1,1</i>	150
103. <i>Summary Statistics SUMI</i>	153
104. <i>Record User SUMI</i>	153
105. <i>Median Box-Plot</i>	153
106. <i>Means with 95% CIs</i>	154
107. <i>Means with Standard Deviations</i>	154

DAFTAR LAMPIRAN

No. Lampiran	Judul Lampiran	Halaman
1.	RPS Mata Kuliah Analisis Runtun Waktu Polstat STIS T.A. 2024/2025.....	165
2.	Alamat Akses Statify	165
3.	Source Code Statify Modul Analisis <i>Time Series</i>	165
4.	Dokumentasi Panduan Penggunaan Statify Modul Analisis <i>Time Series</i>	165
5.	Dokumentasi Algoritma Rumus Statify Modul Analisis <i>Time Series</i>	166
6.	Data <i>Testing</i> Perbandingan <i>Output</i> antara Statify dan Eviews.....	166
7.	Kode Program <i>White-Box Testing</i> Menu <i>Smoothing</i>	169
8.	Hasil <i>White-Box Testing</i> Menu <i>Smoothing</i>	172
9.	Kode Program <i>White-Box Testing</i> Menu <i>Decomposition</i>	173
10.	Hasil <i>White-Box Testing</i> Menu <i>Decomposition</i>	175
11.	Kode Program <i>White-Box Testing</i> Menu <i>Autocorrelation</i>	176
12.	Hasil <i>White-Box Testing</i> Menu <i>Autocorrelation</i>	179
13.	Kode Program <i>White-Box Testing</i> Menu <i>Unit Root Test</i>	180
14.	Hasil <i>White-Box Testing</i> Menu <i>Unit Root Test</i>	186
15.	Kode Program <i>White-Box Testing</i> Menu <i>Box-Jenkins Model</i>	187
16.	Hasil <i>White-Box Testing</i> Menu <i>Box-Jenkins Model</i>	190
17.	Surat Permohonan Lisensi SUMI.....	191
18.	Kuesioner SUMI.....	192
19.	Tingkat Kesiapterapan Teknologi (TKT) Statify	195

BAB I

PENDAHULUAN

1.1 Latar Belakang

Politeknik Statistika STIS merupakan salah satu perguruan tinggi kedinasan yang berada di bawah naungan Badan Pusat Statistik (BPS). Institusi ini menyelenggarakan pendidikan vokasi yang fokus pada ilmu statistika terapan dan komputasi statistik (Peraturan Badan Pusat Statistik Nomor 87 Tahun 2018, 2018). Oleh karena itu pembelajaran metode statistik dijadikan sebagai landasan utama dalam proses pendidikan di perguruan tinggi tersebut. Proses pembelajaran tidak hanya menekankan pada penguasaan teori secara mendalam, tetapi juga dilengkapi dengan pengembangan keterampilan praktis yang memadai, sehingga lulusan mampu mengaplikasikan ilmu statistik secara efektif dalam berbagai konteks profesional.

Analisis Runtun Waktu (*Time Series*) menjadi salah satu mata kuliah yang diajarkan dalam proses pembelajaran di Politeknik Statistika STIS. Materi ini diajarkan untuk memberikan pemahaman tentang metode statistik yang memanfaatkan data historis dalam meramalkan fenomena di masa depan (Nurhasanah et al., 2022). Melalui pembelajaran ini, mahasiswa dilatih untuk mengidentifikasi lintasan dinamis suatu deret data dan melakukan ekstrapolasi komponen-komponen deret ke masa depan guna menghasilkan peramalan yang lebih akurat (Enders, 2014). Selain sebagai alat analisis, hasil pembelajaran analisis runtun waktu juga difokuskan agar mahasiswa mampu menggunakan hasil analisis tersebut sebagai dasar pertimbangan dalam penyusunan strategi, kebijakan, dan

perencanaan bisnis yang efektif dan efisien (Makridakis et al., 1997). Dengan demikian, penguasaan konsep dan teknik analisis runtun waktu menjadi salah satu kompetensi utama yang dikembangkan dalam kurikulum di Politeknik Statistika STIS guna mempersiapkan lulusan yang siap menghadapi tantangan di era saat ini.

Dalam proses pembelajaran Analisis Runtun Waktu, pemanfaatan aplikasi statistik memegang peranan penting dalam mendukung pemahaman konsep dan implementasi teknik analisis. Di Politeknik Statistika STIS, aplikasi Eviews, SPSS, dan R menjadi aplikasi yang sering digunakan. Namun, pemanfaatan aplikasi-aplikasi tersebut belum sepenuhnya optimal karena belum terintegrasi metode Analisis Runtun Waktu yang sesuai dengan alur materi dalam Rencana Pembelajaran Semester (RPS), selain juga terkendala oleh masalah akses serta kebutuhan kemampuan pemrograman.

Salah satu masalah proses pembelajaran Analisis Runtun Waktu adalah terbatasnya interaksi pembelajaran akibat kurang terintegrasinya tahapan metode Analisis Runtun Waktu dalam aplikasi statistik yang digunakan. Sebagai contoh, pada aplikasi SPSS tidak terdapat fitur eksplorasi awal data *time series* yaitu metode *smoothing* yang cukup membingungkan mahasiswa. Selain itu, pada aplikasi ini juga tidak terdapat tambahan visualisasi grafik hasil metode *decomposition* yang dalam penerapannya hal ini penting bagi mahasiswa untuk memahami struktur data *time series*. Begitu juga pada aplikasi Eviews, proses eksplorasi awal untuk memvisualisasikan data *time series* masih terhambat karena belum adanya otomatisasi pembuatan grafik pada aplikasi tersebut. Kondisi ini tentunya membuat mahasiswa kesulitan dalam menggunakan aplikasi untuk melakukan tahapan Analisis Runtun Waktu secara berkesinambungan dan terstruktur.

Oleh karena itu, diperlukan pengembangan aplikasi statistik yang mampu mendukung proses pembelajaran Analisis Runtun Waktu secara terpadu, mulai dari tahap eksplorasi awal data hingga analisis lanjutan. Aplikasi ini diharapkan tidak hanya menyediakan fitur eksplorasi awal data dan otomatisasi visualisasi grafik, tetapi juga mengintegrasikan setiap tahapan analisis secara sistematis. Dengan demikian, proses pembelajaran dapat berlangsung lebih efektif, terstruktur, dan selaras dengan Rencana Pembelajaran Semester (RPS) mata kuliah Analisis Runtun Waktu di Politeknik Statistika STIS.

Maka dari itu, sebagai solusi terhadap berbagai permasalahan tersebut adalah pengembangan Aplikasi Statify. Statify adalah sebuah aplikasi statistik berbasis web yang diharapkan dapat menjadi solusi yang efektif. Dengan menggunakan teknologi berbasis web, aplikasi ini mengalihkan proses komputasi ke server dengan optimasi khusus, yaitu *client-side computing*, sehingga pengguna dapat menikmati proses penghitungan data yang lebih cepat dari sisi klien. Selain itu, Statify juga dirancang dengan mengintegrasikan berbagai metode analisis statistik dalam satu platform yang mudah digunakan, sehingga aplikasi ini diharapkan mampu menjadi media pembelajaran yang memfasilitasi pemahaman konsep, eksperimen metode, dan keterampilan analisis statistik secara menyeluruh bagi mahasiswa Politeknik Statistika STIS.

Salah satu fokus utama pengembangan Statify adalah penyediaan Modul Analisis *Time Series* yang disesuaikan dengan materi-materi yang tercantum dalam RPS Politeknik Statistika STIS T.A. 2024/2025 mata kuliah Analisis Runtun Waktu, dapat dilihat pada Lampiran 1. Modul ini mencakup pengembangan fitur-fitur penting seperti *smoothing*, *decomposition*, *autocorrelation*, *unit root test*, dan

box-jenkins model (ARIMA). Metode-metode tersebut memegang peranan krusial dalam membantu pengguna meneliti dan menganalisis data runtun waktu secara efektif dan efisien, sehingga mendukung proses pembelajaran di lingkungan Politeknik Statistika STIS.

1.2 Identifikasi dan Batasan Masalah

Berdasarkan latar belakang yang telah diuraikan, pembelajaran mata kuliah Analisis Runtun Waktu di Politeknik Statistika STIS masih menghadapi tantangan dalam pemanfaatan aplikasi yang tersedia, di antaranya keterbatasan fitur eksplorasi awal pada SPSS dan ketiadaan otomatisasi visualisasi grafik pada Eviews. Kondisi tersebut mengakibatkan kurangnya integrasi aplikasi-aplikasi tersebut dalam mendukung tahapan pembelajaran Analisis Runtun Waktu secara utuh. Oleh karena itu, Statify dikembangkan sebagai solusi untuk menjawab kebutuhan tersebut dengan mengimplementasikan pendekatan *client-side computing* dan pengembangan berbasis *Feature-Driven Development* (FDD). Untuk mewujudkan Statify sebagai media pembelajaran yang efektif, penelitian ini merumuskan beberapa permasalahan utama yang akan ditangani, sebagai berikut:

1. Bagaimana mengembangkan aplikasi statistik dengan fitur analisis *time series* yang memiliki antarmuka yang efisien, membantu pengguna, serta mudah digunakan dan dipelajari?
2. Bagaimana mengembangkan fitur-fitur Statify Modul Analisis *Time Series* memiliki proses penghitungan dan hasil statistik yang valid?

Selain itu, pembatasan penelitian diperlukan untuk memberikan ruang lingkup yang jelas dan memastikan penelitian ini berjalan dengan terarah dalam

membangun Modul Analisis *Time Series* pada Aplikasi Statify. Dengan demikian, dijabarkan beberapa batasan masalah dalam penelitian ini sebagai berikut:

1. Menu *time series* yang dibangun masih terbatas pada analisis *time series* univariat, diantaranya *smoothing*, *decomposition*, *autocorrelation*, *unit root test*, dan *box-jenkins model* karena keterbatasan waktu pengembangan aplikasi dan teknologi yang digunakan yang dijelaskan pada batasan poin tiga.
2. Tampilan atau *frontend* yang dibangun berfokus pada tampilan modal dan *output* statistik menu *time series* saja yang terdiri atas menu *smoothing*, *decomposition*, *autocorrelation*, *unit root test*, dan *box-jenkins model*.
3. Penulisan kode program *client-side computing* aplikasi Statify modul analisis *time series* dibuat menggunakan bahasa pemrograman Rust dengan tujuan memberikan proses komputasi yang ringan pada aplikasi web sisi klien. Akan tetapi bahasa ini merupakan bahasa pemrograman masih baru dan dirilis pada tahun 2015 sehingga ekosistem serta *package* yang tersedia pada bahasa ini masih terbatas. Dengan demikian, setiap kode *client-side computing* yang dibangun pada fitur ini dimulai dari awal sehingga untuk beberapa sub menu dengan proses komputasi yang kompleks dan rumit seperti *unit root test* dan *box-jenkins model* memiliki pilihan metode penghitungan yang terbatas.

1.3 Tujuan Penelitian

Berdasarkan identifikasi masalah maka penelitian ini bertujuan untuk mengembangkan Statify, sebuah aplikasi statistik yang menyediakan fitur analisis

time series yang mengakomodasi metode yang sesuai dengan RPS Mata Kuliah Analisis Runtun Waktu di Lingkungan Politeknik Statistika STIS. Dengan demikian untuk mencapai tujuan utama tersebut dijabarkan beberapa tujuan khusus pada penelitian ini, yaitu:

1. Mengembangkan Statify Modul Analisis *Time Series* dengan tampilan antarmuka yang efisien, membantu pengguna, serta mudah digunakan dan dipelajari.
2. Mengembangkan setiap fitur Statify Modul Analisis *Time Series* dengan proses penghitungan dan hasil statistik yang valid.

1.4 Manfaat Penelitian

Penelitian ini dapat memberikan manfaat sebagai berikut.

1. Bagi Mahasiswa Politeknik Statistika STIS

Mahasiswa mendapatkan alat bantu berupa aplikasi untuk melakukan analisis data *time series* khususnya pada Mata Kuliah Analisis Runtun Waktu yang dapat diakses secara bebas tanpa kendala lisensi. Aplikasi ini mendukung kegiatan pembelajaran dan praktikum Mata Kuliah Analisis Runtun Waktu dengan antarmuka yang efisien dan mudah dipelajari sehingga mempermudah pemahaman konsep analisis *time series* tanpa harus menguasai kode pemrograman yang kompleks. Selain itu, aplikasi berbasis web ini dapat diakses dari berbagai perangkat tanpa memerlukan instalasi khusus atau spesifikasi perangkat keras tertentu.

2. Bagi Dosen dan Tenaga Pendidik

Dosen dapat menggunakan aplikasi ini sebagai media pengajaran yang efektif untuk mendemonstrasikan konsep analisis runtun waktu secara visual dan interaktif. Kemampuan visualisasi data pada Statify memungkinkan dosen menjelaskan interpretasi hasil analisis seperti pola tren, musiman, dan peramalan dengan lebih mudah. Selain itu, aplikasi ini dapat mempersingkat waktu persiapan materi pengajaran dan praktikum karena tidak memerlukan instalasi perangkat lunak khusus serta menyediakan metode-metode analisis yang sesuai dengan Rencana Pembelajaran Semester (RPS) Mata Kuliah Analisis Runtun Waktu di Lingkungan Politeknik STIS.

3. Bagi Politeknik Statistika STIS sebagai Institusi

Pengembangan aplikasi statistik berbasis web ini dapat meningkatkan kemandirian Politeknik Statistika STIS dalam penyediaan perangkat lunak statistik tanpa bergantung pada lisensi komersial seperti Eviews dan SPSS. Hal ini berpotensi mengurangi biaya operasional institusi terkait pengadaan dan pembaruan lisensi perangkat lunak statistik. Selain itu, aplikasi ini dapat menjadi aset produk aplikasi pendidikan yang inklusif dan berkelanjutan serta meningkatkan citra institusi sebagai perguruan tinggi yang inovatif dalam pengembangan teknologi statistik.

4. Bagi Peneliti sebagai Pengembang Aplikasi Statistik

Melalui penelitian ini, peneliti memperoleh pengalaman praktis dalam pengembangan aplikasi berbasis web dengan pendekatan *Feature-Driven Development* (FDD) dan teknologi *client-side computing*. Pengetahuan dan keterampilan yang diperoleh selama proses

pengembangan aplikasi dapat menjadi bekal berharga untuk proyek-proyek pengembangan perangkat lunak di masa depan.

1.5 Sistematika Penulisan

Penulisan skripsi ini terdiri dari lima bab yang merujuk pada pedoman penulisan skripsi di Politeknik Statistika STIS. Bagian awal buku skripsi memuat Halaman Sampul, Halaman Judul, Halaman Pernyataan, Halaman Pengesahan, Prakata, Abstrak, Daftar Isi, Daftar Tabel, Daftar Gambar, dan Daftar Lampiran. Bagian isi terdiri atas lima bab, yaitu Bab Pendahuluan, Bab Kajian Pustaka, Bab Metode Penelitian, Bab Analisis dan Perancangan, Bab Implementasi dan Evaluasi, serta Bab Kesimpulan dan saran.

Bab I adalah Pendahuluan yang akan menjelaskan latar belakang penelitian, identifikasi dan batasan masalah, tujuan penelitian, manfaat penelitian dan sistematika penulisan.

Bab II adalah Kajian Pustaka yang terdiri dari landasan teori yang menjadi dasar penelitian, penelitian terkait, serta kerangka pikir penelitian.

Bab III adalah Metode penelitian yang terdiri dari tinjauan umum dan tahapan metode penelitian terkait. Adapun metode pengembangan yang digunakan adalah metode *Feature-Driven Development* (FDD) dan metode evaluasi yang digunakan terdiri dari metode *white-box testing* dan *usability testing*.

Bab IV adalah Analisis dan Perancangan. Adapun bagian analisis terdiri dari analisis kebutuhan umum, analisis kebutuhan fungsional, analisis kebutuhan non-fungsional, dan daftar fitur yang dikembangkan dalam penelitian. Selanjutnya

bagian perancangan terdiri atas perencanaan jadwal pengembangan fitur, *sequence diagram*, dan rancangan desain antarmuka.

Bab V adalah Implementasi dan Evaluasi. Adapun bagian implementasi terdiri dari lingkungan implementasi, struktur proyek, dan implementasi fitur. Selanjutnya bagian evaluasi terdiri atas evaluasi metode *white-box testing* dan *usability testing*.

Bab terakhir yaitu Bab VI adalah Kesimpulan dan Saran yang terdiri dari kesimpulan dari implementasi dan evaluasi aplikasi, serta rekomendasi berupa saran dari penulis bagi para penerima manfaat penelitian ini. Selain itu, sumber-sumber lain berupa buku, jurnal, hasil penelitian, dan bahan referensi lainnya yang menjadi rujukan dalam penelitian ini disajikan dalam daftar pustaka.

“...sengaja dikosongkan...”

BAB II

KAJIAN PUSTAKA

2.1 Landasan Teori

Aplikasi

Menurut Irwansyah & Moniaga (2014), aplikasi adalah sebuah *software* yang dirancang untuk meningkatkan produktivitas penggunaanya dalam melakukan suatu tugas tertentu. Dengan ini, kehadiran aplikasi memberikan peran penting dalam setiap aktivitas di era modern ini. Aplikasi dapat ditemukan dalam berbagai bentuk dan fungsi, mulai dari aplikasi perkantoran, komunikasi, pendidikan, hingga hiburan. Secara umum, aplikasi memfasilitasi proses kerja manusia agar lebih efisien, cepat, dan akurat, baik di lingkungan individu maupun organisasi. Oleh karena itu, pemahaman terhadap aplikasi menjadi dasar penting dalam mempelajari dan menerapkan teknologi informasi secara lebih luas.

Statistik

Secara umum statistik dapat diartikan sebagai suatu informasi berupa angka yang diperoleh melalui proses pengolahan dari suatu data. Dalam statistika, suatu ukuran statistik diperoleh melalui beberapa tahapan dimulai dari pengumpulan data, pengolahan, analisa, hingga penyajian. Setiap tahapan ini mengalami perkembangan pesat melalui penemuan teori probabilitas dan teori keputusan, yang penting dalam kehidupan sehari-hari dan menjadikannya valid di semua bidang, termasuk manajemen, teknik, Ekonomi, Ilmu Politik, dan Ilmu Sosial (Asari et al.,

2023). Statistik tidak hanya berperan sebagai alat untuk menyajikan data, tetapi juga sebagai dasar dalam pengambilan keputusan yang rasional dan obyektif. Dengan pendekatan kuantitatif yang sistematis, statistik membantu individu maupun organisasi dalam mengidentifikasi pola, memahami fenomena, serta merumuskan solusi berdasarkan data yang ada (Asari et al., 2023). Oleh karena itu, pemahaman terhadap statistik menjadi sangat penting di era informasi saat ini.

Aplikasi Statistik

Aplikasi statistik adalah suatu *software* yang dirancang khusus membantu penggunaannya dalam menyelesaikan tugas di bidang statistika seperti mengolah dan menganalisis data. Melalui kehadiran aplikasi statistik di bidang statistika, para akademisi tidak perlu mengolah dan menghitung data secara manual. Bahkan, menurut Gani & Amlia (2018) dikatakan bahwa para peneliti tidak perlu lagi menghafal rumus-rumus statistik karena data yang ada hanya perlu dimasukkan lalu dijalankan oleh aplikasi itu sendiri sebab yang terpenting adalah pemahaman peneliti dalam menginterpretasikan hasil penghitungan yang diperoleh. Maka dari itu, aplikasi statistik meningkatkan kecepatan proses penelitian pada tahap mengolah dan menganalisis yang bersesuaian dengan masa ini yang membutuhkan kecepatan dan hasil yang baik dalam melakukan sebuah pekerjaan.

Analisis *Time Series*

Analisis *time series* adalah suatu metode statistik yang memanfaatkan data *time series* dengan tujuan memahami, menggambarkan, dan meramalkan masa

depan hingga mengendalikan sistem secara optimal (Wei, 2006). Data *time series* sendiri adalah kumpulan pengamatan yang diambil secara berurutan dalam rentang waktu tertentu (Box et al., 1994). Analisis *time series* memiliki asumsi prosedur statistik yang berbeda dari biasanya, menurut Wei (2006) hal ini dipengaruhi oleh sifat dasar data *time series* yaitu setiap pengamatan saling bergantung dan berkorelasi. Dengan demikian sulit untuk memodelkan data *time series* ini karena kompleksitasnya dalam menjelaskan pengamatan yang saling berkorelasi ini. Maka dari itu Enders (2014) berpendapat perlunya mengembangkan model yang cukup sederhana yang mampu meramalkan, menginterpretasikan, dan menguji hipotesis data *time series*, sehingga hasil analisis dapat diterapkan secara efektif dalam berbagai bidang, baik dalam ekonomi, teknik, maupun ilmu sosial.

Smoothing

Smoothing adalah teknik dalam analisis *time series* yang digunakan untuk menghilangkan fluktuasi acak atau variasi jangka pendek dalam data, sehingga pola dasar seperti tren atau musiman dapat terlihat lebih jelas. Tujuan utama dari *Smoothing* adalah untuk mengidentifikasi struktur pola yang mendasari data tanpa terganggu oleh gangguan atau *noise* yang bersifat acak. Dalam, Hanke & Reitsch (1995) terdapat beberapa metode *smoothing* yang sering digunakan, antara lain:

1. *Moving Average* adalah metode *smoothing* yang menghitung rata-rata dalam rentang observasi tertentu yang konstan. Adapun secara matematis metode tersebut dapat dihitung dengan persamaan (1).

$$M_t = \hat{Y}_{t+1} = \frac{1}{d} \sum_{i=0}^{d-1} Y_{t-i} \quad (1)$$

Keterangan:

d = jumlah jarak yang digunakan untuk menghitung rata-rata

t = periode data dengan batas $d, d + 1, d + 2, \dots, n$

n = jumlah periode data

M_t = nilai *moving average* periode ke- t

$\hat{Y}_{t+1}$ = data ramalan satu periode selanjutnya setelah t

Y_t = data awal periode ke- t

2. *Double Moving Average* adalah metode *smoothing* yang mengembangkan teknik penghitungan *Moving Average* dengan melakukan sebanyak dua kali lalu menghitung nilai tren dari penghitungan *Moving Average* tahap pertama dan kedua. Adapun secara matematis metode tersebut dapat dihitung dengan persamaan (6).

$$M_t = \frac{1}{d} \sum_{i=0}^{d-1} Y_{t-i} \quad (2)$$

$$M'_t = \frac{1}{d} \sum_{i=0}^{d-1} M_{t-i} \quad (3)$$

$$a_t = 2M_t - M'_t \quad (4)$$

$$b_t = \frac{2}{d-1} (M_t - M'_t) \quad (5)$$

$$\hat{Y}_{t+p} = a_t + b_t p \quad (6)$$

Keterangan:

d = jumlah jarak yang digunakan untuk menghitung rata-rata

t = periode data dengan batas $d, d + 1, d + 2, \dots, n$

n = jumlah periode data

M_t = nilai *moving average* pertama periode ke- t

M'_t = nilai *moving average* kedua periode ke- t

a_t = nilai selisih antara dua *moving average* periode ke- t

b_t = nilai perubahan atau gradien antara *moving average* periode ke- t

$\hat{Y}_{t+p}$ = data ramalan p periode selanjutnya setelah t

Y_t = data awal periode ke- t

3. *Simple Exponential Smoothing* adalah metode *smoothing* yang menghitung peramalan menggunakan pembobotan nilai sekarang dan nilai hasil peramalan masa sebelumnya. Pembobotan ini menggunakan satu parameter yang umumnya sering disebut parameter alpha. Adapun secara matematis metode tersebut dapat dihitung dengan persamaan (7).

$$\hat{Y}_{t+1} = \alpha Y_t + (1 - \alpha) \hat{Y}_t \quad (7)$$

Keterangan:

$\hat{Y}_{t+1}$ = data ramalan baru satu periode selanjutnya

α = konstanta *smoothing* ($0 < \alpha < 1$)

Y_t = data awal periode ke- t

$\hat{Y}_t$ = data ramalan lama satu periode sebelum data ramalan baru

4. *Double Exponential Smoothing* adalah metode *smoothing* yang didasarkan pada Metode *Brown* yang menghitung data deret waktu dengan menggunakan tren linear yang memiliki kesamaan konsep dengan *Double Moving Average*. Adapun secara matematis metode tersebut dapat dihitung dengan persamaan (12).

$$A_t = \alpha Y_t + (1 - \alpha)A_{t-1} \quad (8)$$

$$A'_t = \alpha A_t + (1 - \alpha)A'_{t-1} \quad (9)$$

$$a_t = 2A_t - A'_t \quad (10)$$

$$b_t = \frac{\alpha}{1 - \alpha} (A_t - A'_t) \quad (11)$$

$$\hat{Y}_{t+p} = a_t + b_t p \quad (12)$$

Keterangan:

A_t = nilai *exponential smoothing* pertama periode ke-t

A'_t = nilai *exponential smoothing* kedua periode ke-t

α = konstanta *smoothing* ($0 < \alpha < 1$)

a_t = nilai selisih *exponential smoothing* pertama dan kedua periode ke-t

b_t = nilai gradien *exponential smoothing* pertama dan kedua periode ke-t

$\hat{Y}_{t+p}$ = data ramalan p periode selanjutnya setelah t

Y_t = data awal periode ke-t

5. *Holt's Exponential Smoothing* adalah metode *smoothing* yang penghitungannya menggunakan dua parameter. Parameter pertama (α) digunakan untuk menghitung *Exponential Smoothing* dasar untuk setiap periodenya sedangkan parameter kedua (β) digunakan untuk menghitung

nilai tren dari *Exponential Smoothing* sebelumnya untuk setiap periode. Adapun secara matematis metode tersebut dapat dihitung dengan persamaan (15).

$$A_t = \alpha Y_t + (1 - \alpha)(A_{t-1} - T_{t-1}) \quad (13)$$

$$T_t = \beta(A_t - A_{t-1}) + (1 - \beta)T_{t-1} \quad (14)$$

$$\hat{Y}_{t+p} = A_t + pT_t \quad (15)$$

Keterangan:

A_t = nilai *exponential smoothing* periode ke-t

T_t = nilai estimasi *trend* periode ke-t

α = konstanta *smoothing* ($0 < \alpha < 1$)

β = konstanta *trend* ($0 < \beta < 1$)

$\hat{Y}_{t+p}$ = data ramalan p periode selanjutnya setelah t

Y_t = data awal periode ke-t

6. *Winter's Exponential Smoothing* adalah metode *smoothing* yang mengembangkan dari teknik penghitungan *Holt's Exponential Smoothing* dengan menambahkan faktor parameter musim (γ) dalam penghitungannya. Adapun secara matematis metode tersebut dapat dihitung dengan persamaan (19).

$$A_t = \alpha \frac{Y_t}{S_{t-L}} + (1 - \alpha)(A_{t-1} - T_{t-1}) \quad (16)$$

$$T_t = \beta(A_t - A_{t-1}) + (1 - \beta)T_{t-1} \quad (17)$$

$$S_t = \gamma \frac{Y_t}{A_t} + (1 - \gamma)S_{t-L} \quad (18)$$

$$\hat{Y}_{t+p} = (A_t + pT_t)S_{t-L+P} \quad (19)$$

Keterangan:

A_t = nilai *exponential smoothing* periode ke-t

T_t = nilai estimasi *trend* periode ke-t

S_t = nilai estimasi musim periode ke-t

α = konstanta *smoothing* ($0 < \alpha < 1$)

β = konstanta *trend* ($0 < \beta < 1$)

γ = konstanta musim ($0 < \gamma < 1$)

L = panjang periode musim

$\hat{Y}_{t+p}$ = data ramalan p periode selanjutnya setelah t

Y_t = data awal periode ke-t

Decomposition

Secara umum, *Decomposition* (Dekomposisi) adalah suatu teknik dalam analisis time series yang bertujuan untuk menguraikan data ke beberapa komponen utama yaitu *trend* (tren), *cyclic* (siklus), *seasonal* (musiman), dan *irregular* (acak). Dalam Makridakis et al. (1997), dijelaskan lebih rinci bahwa Dekomposisi dalam ekonomi dan bisnis komponen tren dan siklus diperlakukan sebagai satu komponen yang disebut *trend-cycle* karena mempresentasikan suatu pola jangka panjang dalam suatu deret waktu. Dijelaskan juga, komponen musiman adalah suatu faktor yang berkaitan dengan fluktuasi periodik dengan jangka waktu yang tetap seperti bulan dalam setahun, triwulanan, per semester, dan lain-lain (Makridakis et al.,

1997). Komponen ini biasanya dipengaruhi oleh musim, perayaan, atau pola perilaku tertentu yang berulang secara teratur. Adapun secara matematis Teknik Dekomposisi dapat ditulis dengan persamaan (20) dan (21).

Dekomposisi Multiplikatif

$$Y_t = T_t \times C_t \times S_t \times I_t \quad (20)$$

Dekomposisi Aditif

$$Y_t = T_t + C_t + S_t + I_t \quad (21)$$

Dengan memisahkan masing-masing komponen ini, analisis dekomposisi membantu dalam memahami struktur data runtun waktu secara lebih mendalam serta meningkatkan akurasi dalam proses peramalan.

Stochastic

Stochastic (Stokastik) adalah suatu proses rangkaian kejadian acak yang berkembang seiring waktu. Proses ini berfungsi sebagai model bagi deret waktu yang akan diamati dalam suatu penelitian (Crycer & Chan, 2008). Dalam konteks analisis *time series*, proses stokastik digunakan untuk merepresentasikan data yang mengandung unsur ketidakpastian atau fluktuasi acak dari waktu ke waktu. Oleh karena itu, pemodelan deret waktu yang akurat sering kali memerlukan pemahaman mendalam terhadap sifat stokastik dari data tersebut, termasuk distribusi probabilitas dan hubungan statistik antar waktu.

Stationary

Stationary (Stasioner) adalah suatu keadaan di mana sebuah deret waktu memiliki rata-rata, varians, dan kovarians antar waktu yang bersifat konstan. Sebuah deret waktu juga dikatakan stasioner jika rata-rata dan semua autokovarians tidak terpengaruh oleh perubahan titik awal waktu (Enders, 2014). Artinya, sifat statistik dari deret tersebut tidak berubah seiring waktu yang secara matematis dapat ditulis ke dalam tiga persyaratan persamaan (22), (23), dan (24).

1. stasioner pada rata-rata

$$E(Y_t) = \mu \quad (22)$$

2. stasioner pada varians

$$Var(Y_t) = E(Y_t - \mu)^2 = \sigma^2 \quad (23)$$

3. stasioner pada kovarians

$$Cov(Y_t, Y_{t+k}) = Cov(Y_{t+m}, Y_{t+m+k}) = \gamma_k \quad (24)$$

Karakteristik ini sangat penting dalam analisis deret waktu karena banyak metode statistik, seperti model ARIMA, memerlukan data yang stasioner untuk menghasilkan estimasi dan prediksi yang valid. Jika data tidak stasioner, maka perlu dilakukan transformasi seperti *differencing* agar deret menjadi stasioner sebelum dianalisis lebih lanjut.

Differencing

Differencing (Diferensiasi) adalah teknik transformasi data deret waktu untuk menghilangkan tren stokastik dan membuat data menjadi stasioner. Teknik ini dilakukan dengan menghitung selisih antara nilai saat ini dan nilai sebelumnya

dalam deret waktu yang secara matematis dapat dihitung menggunakan persamaan (25).

$$y'_t = y_t - y_{t-1} \quad (25)$$

Menurut Enders (2014) dikatakan bahwa diferensiasi pada deret waktu akan mengungkapkan pola koefisien *autoregressive* dan *moving average*. Melalui ini proses diferensiasi menjadi suatu langkah yang direkomendasikan jika ingin memodelkan data *time series* menggunakan metode *Box-Jenkins* (Enders, 2014).

Seasonal Differencing

Seasonal Differencing (Diferensiasi Musiman) adalah teknik diferensiasi yang memberikan perhatian tambahan ke faktor musiman. Teknik ini dilakukan dengan menghitung selisih nilai saat ini dengan nilai satu periode musim sebelumnya yang secara matematis dapat ditulis menggunakan persamaan (26).

$$y'_t = y_t - y_{t-s} \quad (26)$$

Hal ini bertujuan menghilangkan efek musiman yang cukup kuat pada data sehingga membuka nilai autokorelasi yang tertutup oleh efek musiman (Enders, 2014).

Unit Root Test

Unit Root Test digunakan untuk menguji apakah suatu deret waktu bersifat stasioner atau mengandung akar unit (Makridakis et al., 1997). Keberadaan akar unit mengindikasikan bahwa data bersifat non-stasioner, sehingga perlu dilakukan transformasi seperti diferensiasi. Pengujian akar unit sangat penting dalam analisis

deret waktu karena pemodelan dalam deret waktu mengasumsikan bahwa data yang digunakan bersifat stasioner. Jika asumsi ini dilanggar, maka hasil estimasi dan prediksi dapat menjadi bias atau menyesatkan. Oleh karena itu, *Unit Root Test* menjadi langkah awal yang krusial dalam pemodelan deret waktu.

Dickey-Fuller Test

Dickey-Fuller Test adalah salah satu metode *Unit Root Test*. Uji ini dirancang untuk melakukan prosedur pengujian keberadaan akar unit secara formal (Enders, 2014). *Dickey-Fuller Test* melakukan uji hipotesis menggunakan pendekatan koefisien variabel periode sebelumnya yang telah didiferensiasi yang pengujiannya dievaluasi menggunakan distribusi yang diperoleh dari simulasi Monte Carlo (Enders, 2014). Selain itu, uji ini juga dikembangkan menjadi *Augmented Dickey-Fuller (ADF) Test* dengan menambah pertimbangan satu hingga beberapa *lag* tambahan dalam proses penghitungan koefisien variabel. Tujuannya adalah untuk mengatasi kemungkinan adanya autokorelasi dalam *error term*, yang dapat memengaruhi validitas hasil uji. *ADF Test* memungkinkan analisis yang lebih akurat terhadap sifat stasioner dari deret waktu, sehingga sering digunakan dalam praktik untuk menentukan apakah proses diferensiasi diperlukan sebelum pemodelan lanjutan dilakukan, seperti dalam model ARIMA.

Autocorrelation

Autocorrelation (Autokorelasi) adalah suatu ukuran korelasi antara nilai dalam deret waktu dengan nilai sebelumnya (Crycer & Chan, 2008). Ukuran ini

diperoleh melalui *Autocorrelation Function* (ACF), yang menggambarkan sejauh mana nilai-nilai dalam deret waktu saling berkaitan pada berbagai *lag* (jeda waktu). Autokorelasi sangat penting dalam analisis deret waktu karena membantu mengidentifikasi pola berulang, tren, atau siklus dalam data. Adapun secara matematis ACF dapat dihitung melalui persamaan (27).

$$r_k = \frac{\sum_{t=1}^{n-k} (Y_t - \bar{Y}) (Y_{t+k} - \bar{Y})}{\sum_{t=1}^{n-k} (Y_t - \bar{Y})^2} \quad (27)$$

Keterangan:

n = jumlah periode data

r_k = sampel autokorelasi lag ke- k

Y_t = data periode ke- t

$\bar{Y}$ = rata-rata data

Selain ACF, terdapat juga *Partial Autocorrelation Function* (PACF) yang digunakan untuk mengukur korelasi deret waktu secara parsial, yaitu setelah menghilangkan pengaruh nilai-nilai antara *lag* yang sedang dianalisis. Adapun secara matematis PACF dapat dihitung melalui persamaan (31).

$$\hat{\phi}_{11} = r_1 \quad (28)$$

$$\hat{\phi}_{22} = \frac{(r_2 - r_1^2)}{(1 - r_1^2)} \quad (29)$$

$$\hat{\phi}_{kj} = \hat{\phi}_{k-1,j} - \hat{\phi}_{kk} \hat{\phi}_{k-1,k-j}; k = 2, \dots, j; j = 1, 2, \dots, k-1 \quad (30)$$

$$\hat{\phi}_{kk} = \frac{(r_k - \sum_{j=1}^{k-1} \hat{\phi}_{k-1,j} r_{k-j})}{(1 - \sum_{j=1}^{k-1} \hat{\phi}_{k-1,j} r_j)}; k = 3, \dots \quad (31)$$

PACF sangat berguna untuk menentukan jumlah *lag* yang relevan dalam model *autoregressive* (AR), karena memberikan informasi lebih jelas tentang hubungan

langsung antar nilai-nilai deret waktu pada *lag* tertentu. Kombinasi analisis ACF dan PACF menjadi dasar penting dalam identifikasi model ARIMA.

Box-Jenkins Model

Box-Jenkins Model adalah pendekatan sistematis dalam pemodelan *time series* yang mempertimbangkan beberapa model faktor, yaitu *autoregressive* (AR) dan *moving average* (MA) serta kemungkinan diferensiasi data (Box, Jenkins, dan Reinsel 1994). Model ini juga sering dikenal sebagai *autoregressive integrated moving average* (ARIMA) model. Pendekatan ini mencakup identifikasi model, estimasi parameter, diagnostik model, dan peramalan. Pada tahap estimasi parameter, terdapat beberapa metode yang dapat digunakan, antara lain Metode *Conditional Least Square* (CLS), *Maximum Likelihood* (ML), dan *Nonlinear Least Squares* (NLS). Pemilihan metode estimasi bergantung pada kompleksitas model dan karakteristik data. Estimasi yang akurat sangat penting agar model mampu menggambarkan pola deret waktu dengan baik serta menghasilkan peramalan yang andal. Setelah estimasi dilakukan, model harus diuji secara diagnostik untuk memastikan tidak ada autokorelasi yang tersisa dan bahwa asumsi distribusi residual terpenuhi. Dengan demikian, pendekatan *Box-Jenkins Model* menjadi alat yang sangat kuat dan fleksibel dalam analisis serta peramalan deret waktu.

Algoritma Durbin Levinson

Algoritma Durbin-Levinson adalah sebuah algoritma rekursif yang digunakan untuk memprediksi nilai masa depan dari deret waktu stasioner dengan

rata-rata nol (Brockwell & Davis, 1996). Algoritma ini dirancang untuk menyederhanakan proses perhitungan prediktor linear satu langkah ke depan berdasarkan observasi sebelumnya. Dengan memanfaatkan sifat rekursif, algoritma ini memungkinkan prediktor berbasis n observasi digunakan untuk membangun prediktor berbasis $n+1$ observasi tanpa harus menyelesaikan ulang sistem persamaan linear yang besar dan kompleks. Melalui prosesnya tersebut algoritma ini cocok untuk menginisiasi nilai awal parameter *autoregressive* (AR) sebelum dilanjutkan ke tahapan estimasi parameter (Brockwell & Davis, 1996).

Algoritma *Innovations*

Algoritma *Innovations* adalah sebuah metode rekursif yang digunakan untuk membentuk prediktor satu langkah terbaik dari suatu deret waktu, baik stasioner maupun non-stasioner, selama deret tersebut memiliki momen kedua yang terbatas (Brockwell & Davis, 1996). Algoritma ini memanfaatkan informasi dari kesalahan prediksi sebelumnya (yang disebut inovasi) untuk membangun prediksi ke depan secara bertahap dan efisien. Inovasi didefinisikan sebagai selisih antara nilai aktual dan prediksinya. Melalui prosesnya tersebut algoritma ini cocok untuk menginisiasi nilai awal parameter *moving average* (MA) sebelum dilanjutkan ke tahapan estimasi parameter (Brockwell & Davis, 1996).

Conditional Least Square

Dalam Crycer & Chan (2008), *Conditional Least Square* (CLS) adalah metode estimasi parameter dengan teknik meminimumkan fungsi *Conditional Sum*

of Square (CSS). Metode ini digunakan untuk mengestimasi parameter model *time series*, khususnya dalam model-model *autoregressive* (AR), *moving average* (MA), maupun gabungannya seperti ARMA dan ARIMA. CLS bekerja dengan cara mengasumsikan nilai-nilai awal dari deret waktu sebagai tetap (tidak diperkirakan), sehingga perhitungan *error* dilakukan secara kondisional terhadap nilai awal tersebut. Teknik ini memiliki keunggulan dalam kesederhanaannya dan kemampuannya memberikan hasil estimasi yang cukup baik pada ukuran sampel besar. Meskipun demikian, CLS terkadang kurang efisien dibandingkan metode *Maximum Likelihood* (ML), terutama pada ukuran sampel kecil. Namun dalam praktiknya, CLS tetap menjadi pilihan populer karena lebih mudah diimplementasikan dan sering memberikan hasil estimasi yang mendekati metode yang lebih kompleks.

Limited-Memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS)

Dalam Okazaki (2014) L-BFGS adalah sebuah algoritma optimasi numerik untuk menyelesaikan masalah minimalisasi tak terbatas. Metode L-BFGS digunakan untuk menemukan *minimizer* dari fungsi objektif $F(x)$, dengan x sebagai vektor variabel, di mana fungsi tersebut dan gradiennya $G(x)$ dapat dihitung. Berbeda dengan metode Newton yang memerlukan komputasi matriks Hessian secara penuh, yang bisa sangat mahal secara komputasi terutama untuk fungsi dengan banyak variabel, L-BFGS mengestimasi invers matriks Hessian menggunakan informasi dari beberapa iterasi terakhir. Pendekatan ini secara signifikan mengurangi kebutuhan memori dan waktu komputasi pada masalah skala besar. Dengan kelebihanannya tersebut maka algoritma ini cocok untuk digunakan

dalam estimasi parameter model statistik termasuk ARIMA dan *machine learning* di masa kini yang memerlukan proses komputasi yang ringan dan efisien.

TypeScript

Dilansir dari halaman web resminya, TypeScript Community (2025), dikatakan bahwa TypeScript adalah bahasa pemrograman yang merupakan super set dari JavaScript, dengan dukungan sistem tipe statis. TypeScript memberikan keuntungan dalam pengembangan perangkat lunak modern karena meningkatkan keandalan dan pemeliharaan kode. Dengan menambahkan anotasi tipe dan kemampuan pengecekan tipe pada waktu kompilasi, TypeScript membantu pengembang menangkap kesalahan lebih awal sebelum kode dijalankan. Selain itu, fitur seperti *auto-completion*, *refactoring* yang lebih aman, dan navigasi kode yang lebih baik sangat mendukung produktivitas dalam proyek skala besar. TypeScript juga sepenuhnya kompatibel dengan JavaScript, sehingga memungkinkan adopsi secara bertahap dan integrasi yang mulus dengan pustaka serta kerangka kerja JavaScript yang sudah ada.

Rust

Dilansir dari halaman web resminya, Rust Team (2025), dikatakan bahwa Rust adalah bahasa pemrograman sistem yang berfokus pada performa, keamanan memori, dan konkurensi. Rust banyak digunakan dalam pengembangan sistem tingkat rendah, seperti pengembangan WebAssembly (WASM) dan *backend* performa tinggi. Salah satu keunggulan utama Rust adalah kemampuannya untuk

mencegah kesalahan memori umum, seperti *null pointer* dan data *race*, tanpa memerlukan *garbage collector*. Dengan sistem kepemilikan memori yang unik, Rust memastikan manajemen memori yang aman dan efisien pada waktu kompilasi. Selain itu, Rust juga menawarkan alat manajemen yang kuat seperti *Cargo* dan ekosistem *crate* yang terus berkembang, menjadikannya pilihan menarik bagi pengembang yang membutuhkan kontrol tingkat rendah dengan jaminan keamanan tinggi dan produktivitas yang baik.

Next.js

Dilansir dari halaman web resminya, Vercel (2025), dikatakan bahwa Next.js adalah *framework* React yang mendukung *client-side rendering*, *client-side computing*, *static site generation*, dan *API routing*. Cocok untuk membangun aplikasi web modern dengan arsitektur Jamstack atau SSR. Next.js memberikan kemudahan dalam pengembangan *front-end* dan *back-end* secara terpadu, dengan fitur seperti *prefetching* otomatis, pengoptimalan gambar, dan pengelolaan meta data untuk SEO. Selain itu, Next.js memungkinkan pengembang untuk membangun aplikasi yang cepat, responsif, dan *scalable*, serta mendukung berbagai metode *rendering* yang fleksibel sesuai kebutuhan performa dan pengalaman pengguna. Dukungan komunitas yang luas dan integrasi dengan platform seperti Vercel juga membuat Next.js semakin populer di kalangan pengembang modern.

WebAssembly

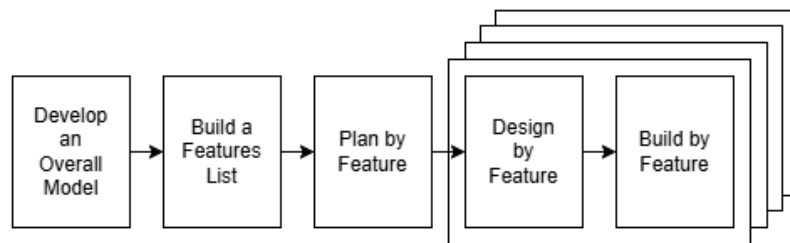
Dilansir dari halaman web resminya, WebAssembly Community (2025), dikatakan bahwa WebAssembly (WASM) adalah format instruksi biner untuk mesin virtual berbasis web yang memungkinkan eksekusi kode berperforma tinggi dalam browser. Digunakan untuk menjalankan aplikasi kompleks, seperti *game*, *machine learning*, dan komputasi numerik. WASM dirancang agar dapat berjalan dengan kecepatan mendekati *native*, menjadikannya solusi ideal untuk aplikasi yang membutuhkan kinerja tinggi namun tetap dapat dijalankan langsung di browser tanpa perlu *plugin* tambahan. Selain itu, WASM mendukung *interoperabilitas* dengan JavaScript, memungkinkan pengembang untuk memanfaatkan pustaka yang sudah ada sambil menulis bagian-bagian performa kritis dalam bahasa seperti C, C++, atau Rust. Dengan pendekatan ini, WASM membantu menjembatani kebutuhan antara fleksibilitas pengembangan web modern dan kekuatan komputasi tingkat rendah.

System Development Life Cycle (SDLC)

System Development Life Cycle (SDLC) adalah suatu model rancangan mencakup rincian dari tahapan dalam melakukan pengembangan, pembaruan, pemeliharaan, dan proses penggantian suatu sistem perangkat lunak (Gurung et al., 2020). Dalam perkembangannya, terdapat beberapa jenis metode SDLC yang umum digunakan, antara lain metode *waterfall*, *agile*, *iterative*, *spiral*, dan *feature-driven development* (FDD). Setiap metode SDLC tersebut memiliki keunikan, keunggulan, dan kelemahannya masing-masing. Maka dari itu, perlu dilakukan pemilihan metode SDLC yang sesuai dengan kebutuhan dari proyek yang akan dibangun atau dikembangkan.

Feature-Driven Development (FDD)

Feature-Driven Development (FDD) adalah salah satu jenis metode SDLC yang pengembangannya benar-benar berfokus pada kebutuhan-kebutuhan yang fungsional. Dalam Palmer & Felsing (2022), dikatakan bahwa walaupun kita mampu membangun suatu proyek sistem yang elegan namun itu tidak mampu menyediakan sistem yang sesuai dengan fungsionalitas kebutuhan klien maka proyek tersebut adalah gagal. Melalui pernyataan ini dapat ditarik kesimpulan bahwa metode ini adalah metode cocok digunakan pada suatu proyek yang memiliki informasi kebutuhan sistem yang lengkap dan merinci sehingga fungsionalitas yang dibangun dapat sesuai dengan kebutuhan klien.



Sumber: Palmer & Felsing (2022)

Gambar 1. Tahapan FDD

Seperti yang ditampilkan pada Gambar 1, metode FDD memiliki lima tahapan dalam proses pengembangan suatu *software*, antara lain *develop an overall model*, *build a features list*, *plan by feature*, *design by feature*, dan *build by feature*. Tahapan pertama sampai tahapan ketiga merupakan tahapan yang cukup krusial dalam metode ini karena penentuan model rancangan proyek, pendaftaran fitur

yang akan dibangun, penentuan prioritas fitur, hingga pembentukan jadwal kerja ditentukan pada tahap ini dan didiskusikan oleh seluruh anggota tim pengembang yang akan berdampak pada keberhasilan tahapan selanjutnya. Sedangkan pada tahapan keempat dan kelima setiap anggota tim sudah dapat bekerja sesuai rancangan yang dibangun dan tanggung jawab masing-masing.

Unified Model Language (UML)

Unified Model Language (UML) adalah sebuah kumpulan notasi grafis yang didasari dalam satu model penulisan yang bertujuan untuk mendeskripsikan suatu perancangan sistem perangkat lunak (Fowler, 2003). Dengan menyediakan representasi visual yang konsisten dan terstruktur, UML memudahkan komunikasi antar anggota tim pengembang serta antara pengembang dengan pemangku kepentingan non-teknis. Notasi-notasi dalam UML mencakup berbagai jenis diagram seperti *Class Diagram*, *Use Case Diagram*, *Sequence Diagram*, dan masih banyak lagi yang masing-masing memiliki fungsi tersendiri dalam menggambarkan struktur maupun perilaku sistem. Seiring dengan berkembangnya kebutuhan dalam pengembangan perangkat lunak, UML menjadi alat bantu yang penting dalam memastikan bahwa desain sistem dapat dipahami secara menyeluruh sebelum proses implementasi dimulai.

Class Diagram

Class Diagram adalah sebuah diagram yang menggambarkan hubungan antar objek dalam suatu perangkat lunak serta mendeskripsikan komponen-

komponen yang terkandung di objek tersebut seperti atribut dan metode objek (Fowler, 2003). Diagram ini berperan penting dalam proses perancangan sistem karena memberikan representasi visual dari struktur statis sistem. Selain itu, *Class Diagram* juga membantu dalam mengidentifikasi tanggung jawab masing-masing objek serta memastikan bahwa desain perangkat lunak selaras dengan kebutuhan fungsional dan non-fungsional sistem. Karena kemampuannya dalam menyederhanakan kompleksitas sistem, *Class Diagram* menjadi salah satu jenis diagram UML yang paling sering digunakan dalam pengembangan perangkat lunak berorientasi objek.

Use Case Diagram

Use Case Diagram adalah sebuah diagram yang mendeskripsikan kebutuhan fungsional suatu sistem yang menjelaskan berbagai skenario hubungan antara pengguna sistem dan sistem itu sendiri (Fowler, 2003). Dalam diagram ini, aktor direpresentasikan sebagai entitas eksternal yang berperan dalam menjalankan *use case* atau kasus penggunaan, yang menggambarkan fungsionalitas utama yang ditawarkan oleh sistem. Setiap *use case* mencerminkan serangkaian langkah atau interaksi yang mungkin terjadi dalam konteks tertentu, seperti melakukan pemesanan, mengelola data, atau memproses pembayaran. Dengan menyajikan gambaran menyeluruh mengenai perilaku sistem dari sudut pandang pengguna, *Use Case Diagram* memudahkan pemangku kepentingan untuk memahami kebutuhan sistem secara umum sebelum masuk ke tahap perancangan teknis yang lebih detail. Diagram ini juga menjadi alat komunikasi yang efektif antara analis sistem, pengembang, dan pengguna akhir.

Sequence Diagram

Secara umum, *Sequence Diagram* merupakan suatu diagram yang mencakup urutan dari kumpulan kegiatan-kegiatan dari suatu skenario (Fowler, 2003). *Sequence Diagram* menggambarkan bagaimana interaksi antar objek dalam suatu sistem berlangsung, dengan menampilkan pesan-pesan yang dikirimkan antar objek serta urutan waktu dari pesan-pesan tersebut. Diagram ini membantu dalam memahami bagaimana objek-objek berkolaborasi untuk mencapai tujuan tertentu dalam satu kasus penggunaan. Selain itu, *Sequence Diagram* ini juga memperlihatkan alur perjalanan yang menggambarkan keberadaan objek yang terlibat dalam interaksi serta urutan waktu pesan yang dikirimkan, membantu memperjelas alur eksekusi metode dan fungsi dalam sistem yang bersangkutan.

White-Box Testing

White-Box Testing adalah metode evaluasi pengujian *software* yang berfokus pada implementasi kode program. Pengujian ini memerlukan penentuan kasus uji yang berfungsi untuk menguji alur kerja kode program pada bagian-bagian tertentu seperti fungsi, percabangan, atau perulangan (Nidhra, 2012). Selain itu, metode ini memiliki beberapa kriteria cakupan pengujian, antara lain *path coverage*, *branch coverage*, dan *data-flow coverage*, yang bertujuan untuk memverifikasi seberapa luas bagian-bagian kode telah diuji (Nidhra, 2012). Dengan demikian, *white-box testing* menyoroti struktur internal *software* dan memberikan

jaminan yang lebih tinggi terhadap keandalan dan kualitas logika pemrograman dalam sistem.

Usability Testing

Usability Testing adalah sebuah metode evaluasi yang memeriksa kualitas hubungan antara pengguna dan tampilan antarmuka suatu *software* (Ammann & Offutt, 2013). Pengujian ini bertujuan untuk memastikan bahwa antarmuka pengguna mudah dipahami, efisien digunakan, dan mampu memenuhi kebutuhan serta harapan pengguna akhir. Dalam praktiknya, *usability testing* dilakukan dengan melibatkan pengguna secara langsung untuk menjalankan skenario penggunaan tertentu, sehingga pengembang dapat mengidentifikasi masalah kegunaan yang tidak terdeteksi melalui pengujian teknis biasa. Dengan demikian, *usability testing* menjadi bagian penting dalam menciptakan pengalaman pengguna yang optimal dan meningkatkan kualitas keseluruhan sistem *software*.

Software Usability Measurement Inventory (SUMI)

SUMI adalah salah satu instrumen atau kuesioner yang dapat digunakan dalam melakukan *usability testing*. Kuesioner ini memiliki lima pokok aspek penilaian, yaitu *Efficiency*, *Affect*, *Helpfulness*, *Control*, dan *Learnability* (Kirakowski, 1994). Setiap aspek tersebut dirancang untuk mengevaluasi pengalaman pengguna dalam berinteraksi dengan perangkat lunak secara menyeluruh, mulai dari seberapa efisien *software* dalam menyelesaikan tugas, kenyamanan emosional pengguna saat menggunakannya, sejauh mana sistem

membantu pengguna, tingkat kendali yang dirasakan pengguna terhadap sistem, hingga kemudahan dalam mempelajari dan memahami cara kerja *software*. Dengan pendekatan kuantitatif yang sistematis, SUMI memberikan gambaran objektif mengenai kualitas penggunaan suatu perangkat lunak berdasarkan persepsi nyata pengguna.

2.2 Penelitian Terkait

Berdasarkan permasalahan yang telah dibahas pada bab sebelumnya. Penelitian ini menggunakan beberapa penelitian terdahulu sebagai referensi yang mendukung proses pengembangan Statify Modul Analisis *Time Series*. Adapun, penelitian terdahulu yang digunakan memiliki keterkaitan dalam hal kesamaan aplikasi yang dikembangkan, metode pengembangan, dan teknologi yang digunakan.

Penelitian pertama ditulis oleh Paisal et al. (2021), mengembangkan aplikasi statistik analisis Uji-*T* dalam bentuk web interaktif. Selain itu, terdapat juga penelitian kedua yang ditulis oleh Sriwahyuni et al. (2018), mengembangkan aplikasi statistik analisis regresi berbasis web yang diberi nama StaWEB. Kedua penelitian ini memiliki kesamaan dalam hal jenis aplikasi yang dibangun yaitu aplikasi statistik berbasis web. Namun, kedua penelitian tersebut dan penelitian ini memiliki perbedaan pada metode pengembangan yang digunakan dan bahasa pemrograman yang digunakan. Pada penelitian pertama metode pengembangan yang digunakan adalah pendekatan *System Development Life Cycle* (SDLC) tanpa penjelasan lebih lanjut metode pengembangan apa yang digunakan dan penelitian kedua metode pengembangan yang digunakan adalah metode *waterfall*. Lalu kedua

penelitian tersebut sama-sama menggunakan *framework* R-Shiny dengan bahasa pemrograman R. Sedangkan penelitian ini melakukan pengembangan aplikasi Statify menggunakan metode *Feature-Driven Development* (FDD). Selain itu, penelitian ini mengembangkan Statify menggunakan *framework* Next.js dengan bahasa pemrograman JavaScript dan TypeScript.

Selanjutnya penelitian ketiga memiliki keterkaitan dalam penggunaan metode pengembangan yang digunakan. Penelitian ketiga ditulis oleh Al-Hakim et al. (2024), mengembangkan sistem bimbingan konseling menggunakan metode FDD. Melalui penelitian ini diperoleh informasi bahwa penerapan metode FDD dalam pengembangan sistem memungkinkan pengembangan sistem menjadi lebih terstruktur dan efisien walaupun pengembangannya dibagi berdasarkan fitur-fitur yang hendak dibangun. Dengan demikian, metode ini akan mempermudah proses integrasi fitur-fitur yang berhasil dibangun.

Penelitian keempat memiliki keterkaitan dengan metode evaluasi *white-box testing*. Penelitian keempat ditulis oleh Sie et al. (2022), menguji *website Room* dengan tujuan memeriksa keberadaan *error* pada kode program. Selain itu, penelitian tersebut memiliki kesamaan pada teknik yang digunakan yaitu teknik *basis path*. Dengan menggunakan teknik ini peneliti dapat memperoleh cakupan *test case* yang lebih menyeluruh sehingga *error* atau kegagalan proses kerja algoritma dapat ditemukan lebih awal sebelum dilanjutkan ke tahapan *black-box testing*.

Penelitian kelima memiliki keterkaitan dengan metode evaluasi *usability testing* dan kuesioner evaluasi yang digunakan. Penelitian kelima ditulis oleh

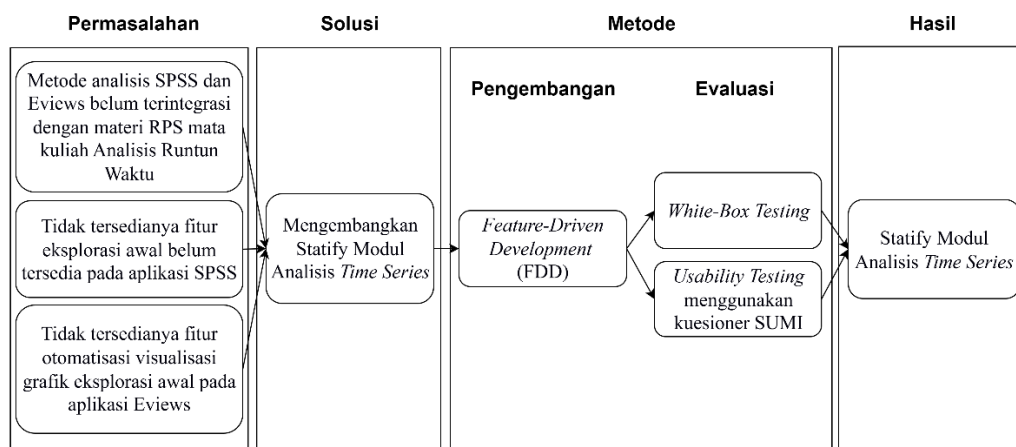
Pratomo & Mantala (2016), mengevaluasi kualitas penggunaan menggunakan kuesioner *Software Usability Measurement Inventory* (SUMI). Berdasarkan penelitian tersebut diperoleh informasi bahwa instrumen penilaian kuesioner SUMI cocok untuk digunakan sebelum tahapan peluncuran aplikasi. Hal ini dikarenakan kuesioner berfokus pada lima dimensi kualitas penggunaan yaitu *efficiency*, *affect*, *helpfulness*, *control*, dan *learnability*.

Penelitian keenam memiliki keterkaitan dengan penggunaan *framework* Next.js. Penelitian keenam ditulis oleh Jubhari Phie Joarno et al. (2022), mengembangkan *website GetHelp* menggunakan Next.js. Dalam penelitian tersebut diperoleh informasi bahwa pengembangan web menggunakan *framework* Next.js mampu meningkatkan performa *Lighthouse* sebesar 23% dan *GTMetrix* sebesar 14%. Sebagai informasi tambahan *Lighthouse* adalah ukuran untuk mengukur ketepatan ukuran konten antarmuka web dan *GTMetrix* adalah ukuran performa yang terdiri atas tujuh kriteria antara lain, *performance*, *first contentful paint*, *time to interactive*, *speed index*, *total blocking time*, *largest contentful paint*, dan *cumulative layout shift*. Dengan demikian, penggunaan *framework* Next.js merupakan pilihan yang tepat bagi penelitian ini dalam pembangunan aplikasi statistik berbasis web dengan performa yang baik.

Penelitian ketujuh memiliki keterkaitan dengan penggunaan teknologi WebAssembly (WASM). Penelitian ketujuh ditulis oleh Kim et al. (2024), menguji kinerja teknologi WASM dalam peningkatan kinerja di sistem tertanam. Penelitian ini menunjukkan penggunaan teknologi WASM memiliki peningkatan performa yang baik dalam perangkat keras desktop dikarenakan teknologi ini menggunakan

bytecode yang mampu mengurangi *overhead* pada proses kompilasi perangkat lunak. Teknologi ini diperlukan dalam pengembangan Statify karena kode program penghitungan analisis statistik yang dikembangkan dapat dieksekusi secara optimal.

2.3 Kerangka Pikir



Gambar 2. Kerangka Pikir Penelitian

Berdasarkan permasalahan dan tujuan penelitian yang telah dijelaskan sebelumnya, secara umum, kerangka pikir penelitian ini dapat dilihat pada Gambar 2. Pengembangan Statify Modul Analisis *Time Series* dimulai dari beberapa poin permasalahan, antara lain metode analisis SPSS dan Eviews belum terintegrasi dengan materi RPS mata kuliah Analisis Runtun Waktu, tidak tersedianya fitur eksplorasi awal belum tersedia pada aplikasi SPSS, dan tidak tersedianya fitur otomatisasi visualisasi grafik eksplorasi awal pada aplikasi Eviews. Selanjutnya penelitian ini menawarkan solusi permasalahan dalam bentuk pengembangan Statify Modul Analisis *Time Series*. Dalam pengembangannya peneliti

menggunakan metode pengembangan *Feature-Driven Development* (FDD). Selanjutnya pengembangan ini akan diikuti dengan tahapan evaluasi pada setiap fiturnya menggunakan metode *white-box testing* dan *usability testing* dengan kuesioner *Software Usability Measurement Inventory* (SUMI). Dengan demikian, diperoleh Statify Modul Analisis *Time Series* sebagai hasil akhir penelitian ini.

“...sengaja dikosongkan...”

BAB III

METODE PENELITIAN

3.1 Tinjauan Umum

3.1.1 Metode Pengembangan

Dalam penelitian ini, metode pengembangan aplikasi yang digunakan adalah *Feature-Driven Development* (FDD). Metode ini dipilih karena sesuai dengan kebutuhan pengembangan aplikasi Statify yang mengutamakan sistem kerja yang terstruktur dan fleksibel dalam proses pengerjaannya. Statify dibangun berdasarkan kebutuhan atau fitur spesifik yang kemudian dikumpulkan dan dikelompokkan ke dalam beberapa modul pengembangan. Mengingat terbatasnya waktu pengembangan, metode ini memungkinkan setiap anggota tim yang bertanggung jawab atas satu modul untuk bekerja secara paralel, sehingga dapat mengefisienkan waktu pengerjaan. Meskipun pengerjaan setiap modul dilakukan secara terpisah, setiap anggota tim tetap mengikuti struktur yang seragam berkat adanya penentuan batasan-batasan proyek yang jelas pada tahapan awal metode ini. Oleh karena itu, metode ini menjadi pilihan yang tepat untuk diterapkan dalam pengembangan aplikasi Statify dalam penelitian ini.

Dalam proses pengerjaannya, FDD terdiri dari lima tahapan utama. Pada tahapan pertama hingga ketiga, pengerjaan dilakukan secara bersama-sama oleh seluruh anggota tim untuk memperoleh konsep, kerangka kerja, dan aturan kerja yang jelas terkait proyek yang akan dikembangkan. Selanjutnya, pada tahapan keempat dan kelima, setiap anggota tim bertanggung jawab atas tugas dan perannya masing-masing dalam proses pengembangan. Pada tahapan ini, setiap anggota tim

dapat melakukan iterasi sesuai dengan kebutuhan pengembangan. Dalam setiap iterasi, anggota tim akan mengevaluasi fitur yang telah dibangun dan melaporkannya kepada ketua tim.

3.1.2 Metode Evaluasi

Dalam penelitian ini, peneliti menggunakan tiga metode evaluasi untuk mengukur validitas hasil penghitungan statistik *time series* dan kemudahan penggunaan aplikasi Statify menu *time series*, antara lain:

1. White-Box Testing

White-box testing adalah metode pengujian yang berfokus pada implementasi kode program. Pengujian ini bertujuan untuk mendeteksi kesalahan-kesalahan kecil dari kumpulan fungsi yang saling terintegrasi untuk menjalankan suatu tugas. Dalam *white-box testing* terdapat beberapa teknik pengujian yang umum digunakan, antara lain *control flow testing*, *basis path testing*, *loop testing*, dan *data flow testing* (Nidhra, 2012). Teknik pengujian yang digunakan dalam penelitian ini adalah *basis path testing*. Teknik ini memiliki keunggulan pada tahapan pengujiannya yang mampu mencakup *test case* yang lebih luas dengan membuat grafik alur data yang mempertimbangkan setiap kemungkinan kondisi aliran data (Sie et al., 2022). Dengan demikian peneliti dapat memastikan bahwa setiap algoritma kode program yang ditulis dapat berjalan secara terstruktur sesuai dengan rancangan.

2. Usability Testing

Usability testing adalah suatu metode pengujian yang bertujuan untuk mengevaluasi kualitas penggunaan suatu aplikasi oleh penggunanya. Pada pengujian ini, peneliti menggunakan kuesioner *Software Usability Measurement Inventory* (SUMI) untuk mengukur kualitas *efficiency*, *affect*, *helpfulness*, *control*, dan *learnability* (Kirakowski, 1994). Kuesioner ini terdiri atas lima puluh pertanyaan dengan tiga buah pilihan jawaban untuk setiap item, sehingga memudahkan pengguna dalam menjawab pertanyaan. Selain itu, jumlah minimal responden yang disarankan untuk pengujian dengan kuesioner SUMI berjumlah dua belas orang. Dengan demikian, proses evaluasi kualitas penggunaan pada aplikasi Statify menu *analisis time series*, dapat dilakukan melalui tahapan yang sederhana namun tetap menghasilkan ukuran kualitas yang komprehensif.

3.2 Tahapan Penelitian

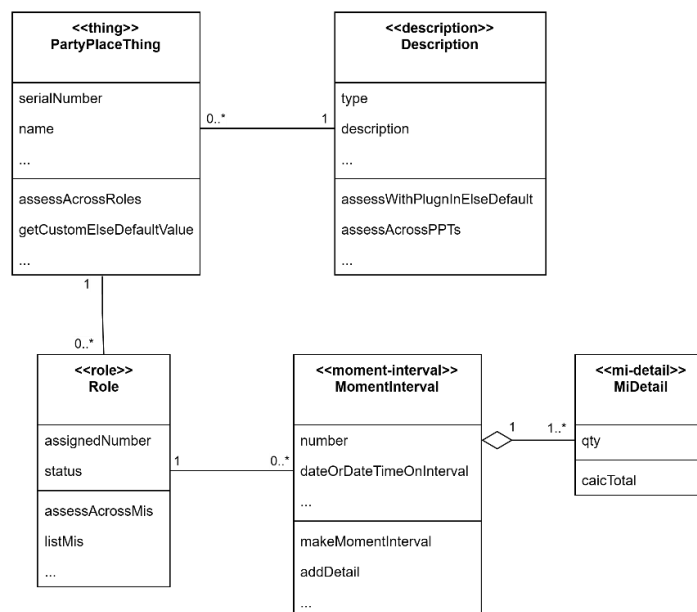
3.2.1 Tahapan Feature-Driven Development (FDD)

Metode FDD terdiri dari 5 tahapan utama, diantaranya *develop an overall model*, *build a features list*, *plan by feature*, *design by feature*, dan *build by feature*. Adapun rincian dari setiap tahapan metode ini adalah sebagai berikut.

1. *Develop an Overall Model*

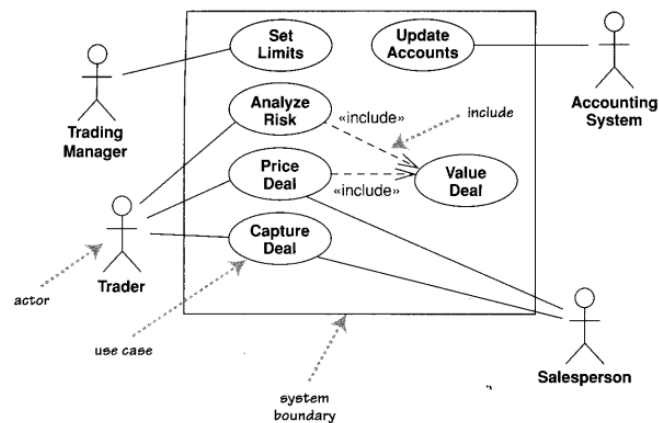
Pada tahapan ini, peneliti bersama seluruh anggota tim pengembang lainnya berdiskusi mengenai gambaran umum proyek yang akan dibangun. Gambaran umum tersebut dibangun melalui pendekatan yang sederhana yaitu *class diagram*. Melalui diagram ini nantinya akan dijelaskan mengenai entitas–entitas yang terlibat dalam sistem aplikasi, meliputi pengguna atau peran, nama

proses kerja, hasil proses, serta objek lain yang memiliki peran penting. Selanjutnya, peneliti dan anggota tim lainnya akan menentukan atribut dan metode yang diperlukan untuk setiap entitas yang terlibat. Pada bagian akhir tahap ini, hubungan antar entitas digambarkan untuk melengkapi *class diagram*. Selain itu, semua kemungkinan proses bisnis yang ada perlu diekspresikan dalam bentuk *use case diagram* agar alur sistem kerja aplikasi mampu tergambar dengan jelas dan menjadi dasar yang baik untuk tahapan pengembangan selanjutnya.



Sumber: Palmer & Felsing (2022)

Gambar 3. Contoh *Class Diagram*



Sumber: Fowler (2003)

Gambar 4. Contoh *Use Case Diagram*

2. *Build a Feature List*

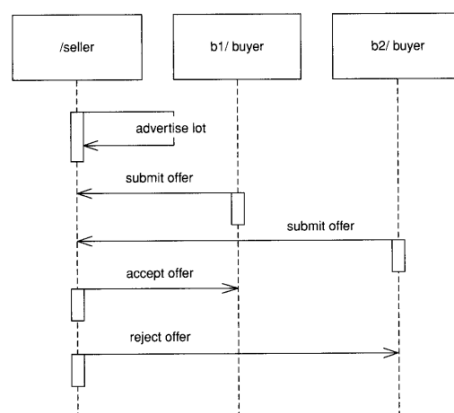
Pada tahapan ini, peneliti dan anggota tim pengembang merancang dan membuat daftar fitur yang akan dibangun berdasarkan proses bisnis yang tercakup pada *class diagram* dan *use case diagram* yang telah dibuat sebelumnya. Hal pertama yang perlu dilakukan pada tahapan ini ialah membagi setiap proses bisnis yang ada menjadi beberapa area domain atau dapat disebut *major feature sets*. Selanjutnya, peneliti dan anggota pengembang bertanggung jawab atas beberapa *major feature sets*. Setiap *major feature sets* yang ada harus dipecah kembali menjadi beberapa kumpulan kegiatan yang dapat disebut *feature sets*. Lalu, setiap *feature sets* yang diperoleh dipecah lagi menjadi beberapa kegiatan kecil yang dapat disebut *features*. Setelah setiap area domain sudah dipecah hingga membentuk kumpulan kegiatan kecil, daftar-daftar fitur tersebut akan saling dievaluasi antar anggota tim pengembang sehingga diperoleh daftar fitur yang terstruktur dan rapi.

3. *Plan by Feature*

Pada tahapan ini, peneliti dan anggota tim pengembang bersama menyepakati urutan pengembangan fitur. Urutan yang disepakati ini disusun atas dasar beberapa pertimbangan, antara lain ketergantungan antar fitur, tingkat prioritas, dan kompleksitas fitur. Dengan demikian, setiap anggota tim pengembang diwajibkan mampu memahami ketergantungan hingga tingkat kompleksitas fitur yang dipegang masing-masing anggota tim pengembang sehingga urutan pengembangan fitur dapat disusun dengan tepat. Hasil dari tahapan ini ialah jadwal yang ditulis dalam *gant chart*.

4. *Design by Feature*

Pada tahapan ini, peneliti dan anggota tim pengembang sudah dapat mengerjakan setiap fitur yang ditanggunggjawab masing-masing berdasarkan jadwal dan aturan proses bisnis yang telah disepakati pada beberapa tahapan sebelumnya. Selanjutnya, peneliti merancang urutan tahapan proses bisnis masing-masing fitur dalam bentuk *sequence diagram*. Selain itu, di tahapan ini juga peneliti mulai merancang *mockup* guna mempermudah proses implementasi di tahapan selanjutnya.



Sumber: Fowler (2003)

Gambar 5. Contoh *Sequence Diagram*

5. ***Build by Feature***

Pada tahapan ini, peneliti memasuki ke tahapan pembuatan kode program sepenuhnya berdasarkan rancangan yang telah dibuat di tahapan sebelumnya. Ketika fitur yang dibangun sudah selesai, peneliti dapat langsung menguji fitur tersebut secara *end-to-end* pada level integrasi. Selanjutnya peneliti dapat mengajukan penggabungan kode program ke bagian utama jika hasil pengujian berjalan dengan baik. Namun, jika masih terdapat kesalahan pada pengujian, peneliti dapat kembali ke tahap 4 untuk memeriksa kembali rancangan yang telah dibuat dan kembali melanjutkan tahap 5 untuk memperbaiki bagian fitur yang masih salah. Sebagai tambahan, tahapan 4 dan 5 dapat dilakukan iterasi sesuai dengan keperluan *programmer* hingga fitur dibangun dapat berjalan dengan dan dapat digabung ke bagian utama.

3.2.2 **Tahapan White-Box Testing**

Berdasarkan penjelasan pada tinjauan umum tahapan yang digunakan pada metode didasarkan pada tahapan teknik *basis path testing* dengan rincian sebagai berikut:

1. ***Flow Graph Notation***

Pada tahapan ini, kode program yang telah ditulis perlu digambarkan dalam bentuk *flow graph* (grafik alir) program. Grafik ini menunjukkan alur kerja program dari satu bagian ke bagian lain. Setiap bagian kode penting (seperti kode *statement* atau *conditional*) menjadi lingkaran (*node*), dan panah (*edge*) menunjukkan bagaimana program bergerak antar bagian tersebut serta keputusan (*predicate node*) yang menunjukkan kode *conditional*. Dengan

pengelompokan ini peneliti dapat membentuk gambaran algoritma kode program secara keseluruhan sebelum dilanjutkan tahapan selanjutnya.

2. *Cyclomatic Complexity*

Pada tahapan ini, kita mengukur seberapa kompleks alur program yang telah digambar. Angka kompleksitas ini membantu kita tahu berapa banyak jalur berbeda yang mungkin ada dalam kode. Angka ini didapatkan dengan menghitung jumlah panah, *node*, dan keputusan dalam grafik. Semakin tinggi angkanya, semakin banyak jalur yang harus diuji.

3. *Deriving Test Cases*

Pada tahapan ini, kita membuat daftar skenario uji berdasarkan jalur-jalur yang paling penting dalam program. Kita menggunakan angka kompleksitas dari tahap sebelumnya untuk menentukan berapa banyak jalur unik yang perlu diuji. Misalnya, jika ada enam jalur unik, kita perlu membuat enam skenario uji. Setiap skenario uji harus mencakup *input* yang akan diberikan dan hasil yang diharapkan.

3.2.3 Tahapan Usability Testing

Berdasarkan penjelasan pada tinjauan umum tahapan yang digunakan pada metode *usability testing* dengan menggunakan kuesioner SUMI adalah sebagai berikut:

1. Analisis Konteks

Pada tahapan ini, peneliti melakukan analisis konteks untuk mendeskripsikan konteks penggunaan sistem, yang terdiri dari pengguna, tugas, lingkungan, dan perangkat keras yang digunakan serta menyiapkan

syarat-syarat yang diperlukan untuk memperoleh izin lisensi kuesioner SUMI. Kegiatan ini akan dilakukan melalui rapat yang melibatkan seluruh anggota tim pengembang guna memastikan persiapan yang matang sebelum melaksanakan *usability testing*.

2. Pengukuran Kepuasan Pengguna

Pada tahapan ini, peneliti meminta pengguna terpilih yang sesuai dengan hasil diskusi pada tahapan sebelumnya untuk melakukan uji coba sesuai panduan penggunaan yang telah dibuat. Selain itu, untuk mempermudah proses pengumpulan data menggunakan kuesioner SUMI, peneliti memindahkan pertanyaan yang terdapat pada *website* resmi SUMI ke dalam *google form* dan memberikan keterangan tambahan pada setiap pertanyaan untuk mempermudah pemahaman responden dalam menjawab pertanyaan. Selanjutnya pengguna menyelesaikan tugas yang ditentukan, mereka diminta mengisi kuesioner yang telah disiapkan sebelumnya.

3. Analisis Hasil

Pada tahapan ini akan dilakukan penghitungan nilai dari setiap dimensi yang diperoleh dari kuesioner SUMI. Dengan mengumpulkan data kuesioner SUMI keseluruhan ke *website* resminya karena dalam aturan lisensi data yang diperoleh hanya dapat diolah oleh pemilik resmi kuesioner SUMI. Adapun dalam *website* resmi kuesioner SUMI secara umum data yang diperoleh melalui beberapa tahapan dasar, yaitu pembobotan setiap item jawaban berdasarkan kontribusi global, dilanjutkan standarisasi nilai menjadi *Z-Score*, lalu dikonversi ke bentuk *T-Score*. Setelah peneliti memperoleh hasil penghitungan dari pemilik resmi SUMI dalam nilai akhir *T-Score*. Peneliti

melanjutkan tahapan ini dengan menginterpretasikan hasil tersebut berdasarkan aturan interpretasi yang dituliskan pada Tabel 1 sebagai bentuk evaluasi akhir *usability testing* Aplikasi Statify Modul Analisis *Time Series*.

Tabel 1. Interpretasi *T-Score* Kuesioner SUMI

No.	<i>T-Score</i>	<i>Grade</i>	Interpretasi
(1)	(2)	(3)	(4)
1	60 +	A	<i>high</i>
2	55-60	B	<i>good</i>
3	50-55	C +	<i>above average</i>
4	45-50	C -	<i>below average</i>
5	40-45	D	<i>poor</i>
6	40 -	E	<i>low</i>

Sumber: Kirakowski (2021)

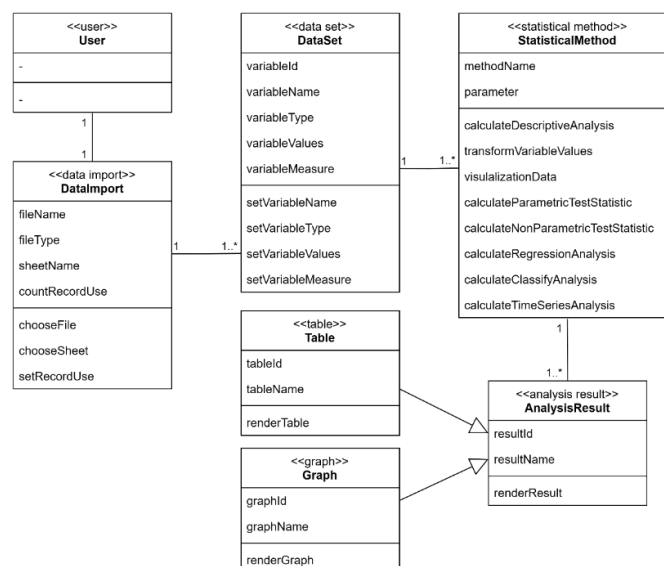
BAB IV

ANALISIS DAN PERANCANGAN

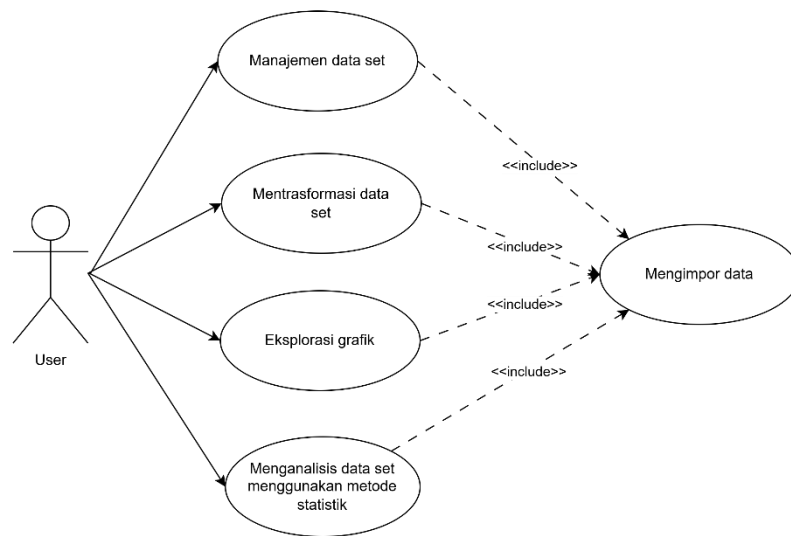
4.1 Analisis

4.1.1 Kebutuhan Umum Pengguna

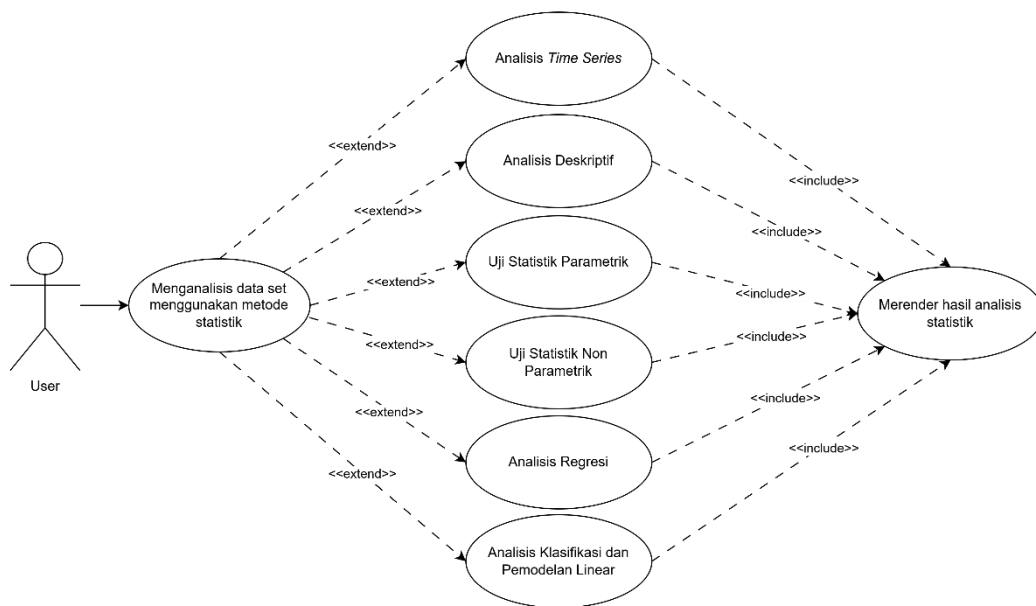
Statify dirancang berdasarkan kebutuhan mahasiswa dan dosen dalam mendukung proses pembelajaran materi statistika di Lingkungan Politeknik Statistika STIS. Aplikasi ini menyediakan berbagai kebutuhan umum dalam pembelajaran statistika seperti eksplorasi data, visualisasi data, transformasi data, hingga analisis data dengan berbagai metode statistik. Melalui pengalaman dan observasi peneliti beserta tim pengembang lainnya yang juga merupakan bagian dari mahasiswa Politeknik Statistika STIS maka diperoleh rancangan proses kerja aplikasi secara umum yang dituangkan pada *Class Diagram* berikut (Gambar 6) beserta beberapa kegiatan umum yang dapat dilakukan pada aplikasi statistik yang dituangkan pada dua gambar *Use Case Diagram* berikut (Gambar 7 dan 8).



Gambar 6. *Class Diagram* Statify



Gambar 7. *Use Case Diagram* Statify Keseluruhan

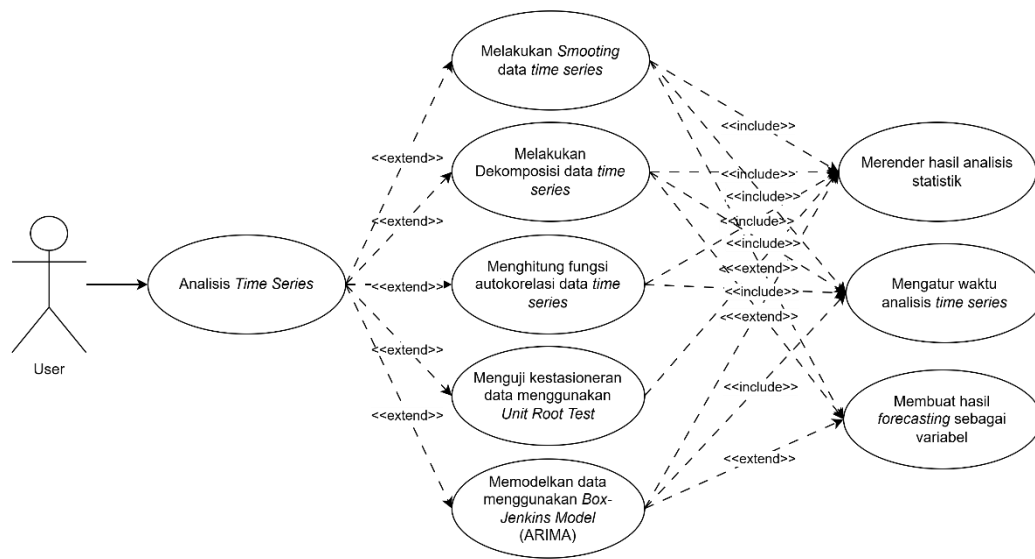


Gambar 8. *Use Case Diagram* Statify Analisis Data

Berdasarkan kedua *Use Case Diagram* di atas maka pengembangan Statify dibagi menjadi enam modul yakni setiap anggota tim pengembang bertanggung

jawab untuk mengembangkan satu modul. Modul yang terdapat pengembangan Statify, antara lain modul manajemen data dan analisis deskriptif, modul transformasi data dan grafik, modul uji statistik parametrik dan non-parametrik, modul analisis regresi, modul analisis klasifikasi dan pemodelan linear, dan modul analisis *time series*. Dari keenam modul yang ada peneliti bertanggung jawab untuk membangun modul analisis *time series*.

Analisis *time series* merupakan salah satu materi pembelajaran yang penting dalam kurikulum Politeknik Statistika STIS. Dalam materi ini secara umum mempelajari analisis tren, dekomposisi deret waktu, stasioneritas, hingga peramalan deret waktu (*forecasting*). Di Politeknik Statistika STIS, aplikasi yang sering digunakan dalam proses pembelajaran adalah Eviews dan R Studio beserta SPSS sebagai aplikasi tambahan. Tentunya seperti yang telah dijelaskan pada bab sebelumnya aplikasi-aplikasi tersebut memiliki kendalanya masing-masing seperti akses penggunaan, kemudahan pemakaian, dan perlunya kemampuan pemrograman pada aplikasi statistik berbasis skrip. Selain itu, diperlukan juga aplikasi statistik yang mampu mengakomodasi materi pembelajaran mata kuliah Analisis Runtun Waktu ke dalam satu aplikasi statistik khususnya di fitur analisis *time series*. Berdasarkan hasil observasi yang diperoleh melalui dokumen Rancangan Pembelajaran Semester Analisis Runtun Waktu di Politeknik Statistika STIS tahun 2024/2025 maka diperoleh kebutuhan umum pengguna Statify Modul Analisis *Time Series* seperti yang tertuang pada Gambar 9.



Gambar 9. Use Case Diagram Statify Modul Analisis Time Series

4.1.2 Kebutuhan Fungsional

Kebutuhan fungsional Statify Modul Analisis *Time Series* mengacu pada *Use Case Diagram* yang telah dibuat yang difokuskan lebih dalam ke spesifikasi teknis dan fitur yang wajib diimplementasikan ke dalam sistem. Pada bagian ini peneliti mengidentifikasi setiap bagian penting dalam pengembangan Statify Modul *Time Series* yang akan dipecah ke lima menu utama, yaitu:

1. *Smoothing*

Menu ini akan menampilkan antarmuka metode *smoothing* yang berfungsi untuk melakukan pemulusan data *time series* guna memperjelas pola yang terdapat pada data *time series*. Selain itu, fungsionalitas yang terdapat pada menu ini antara lain konfigurasi variabel, konfigurasi parameter metode *smoothing*, konfigurasi waktu, penyimpanan otomatis konfigurasi menu yang terakhir kali dipilih, penyimpanan variabel hasil

smoothing sebagai variabel, dan tampilan hasil *smoothing* dalam bentuk tabel evaluasi dan grafik *smoothing*.

2. *Decomposition*

Menu ini akan menampilkan antarmuka metode *decomposition* yang berfungsi untuk mengurai data *time series* menjadi beberapa komponen yaitu komponen siklus-tren (*trend-cycle*), musiman (*seasonal*), dan acak (*irregular*) serta meramal data *time series* menggunakan komponen siklus-tren dan musiman. Selain itu, fungsionalitas menu ini antara lain konfigurasi variabel, konfigurasi opsi metode *decomposition*, konfigurasi waktu, penyimpanan otomatis konfigurasi menu yang terakhir kali dipilih, penyimpanan variabel hasil dekomposisi sebagai variabel, dan tampilan hasil *decomposition* dalam bentuk tabel indeks musiman, tabel evaluasi *forecasting*, grafik tiap komponen hasil *decomposition* dan grafik *forecasting*.

3. *Autocorrelation*

Menu ini akan menampilkan antarmuka metode *autocorrelation* yang berfungsi untuk menghitung fungsi autokorelasi dan fungsi autokorelasi parsial guna mengeksplorasi stasioneritas data *time series* dan menentukan parameter yang tepat sebelum pemodelan *box-jenkins model* (ARIMA). Selain itu, fungsionalitas yang terdapat pada menu ini antara lain konfigurasi variabel, konfigurasi parameter metode *autocorrelation*, konfigurasi waktu, penyimpanan otomatis konfigurasi menu yang terakhir kali dipilih, dan tampilan analisis *autorrelation* dalam bentuk tabel

autorrelation function (acf), bentuk tabel *partial autorrelation function* (pacf) dan grafik koleogram.

4. *Unit Root Test*

Menu ini akan menampilkan antarmuka uji *unit root* yang berfungsi untuk menguji asumsi stasioneritas data *time series* sebelum melakukan pemodelan model *time series* pada data. Selain itu, fungsionalitas yang terdapat pada menu ini antara lain konfigurasi variabel, konfigurasi parameter *unit root test*, konfigurasi waktu, penyimpanan otomatis konfigurasi menu yang terakhir kali dipilih, dan tampilan analisis *unit root test* dalam bentuk tabel hasil uji beserta tabel analisis kriteria model.

5. *Box-Jenkins Model*

Menu ini akan menampilkan antarmuka model *box-jenkins* yang berfungsi untuk memodelkan data *time series* dengan prinsip box-jenkins yang sering disebut dalam literatur sebagai *auto-regressive integrated moving average* (arima) *model*. Selain itu, fungsionalitas yang terdapat pada menu ini antara lain konfigurasi variabel, konfigurasi parameter *box-jenkins model*, konfigurasi waktu, penyimpanan otomatis konfigurasi menu yang terakhir kali dipilih, penyimpanan variabel hasil *forecasting* sebagai variabel, dan tampilan analisis *box-jenkins model* dalam bentuk tabel estimasi parameter, tabel analisis kriteria model, tabel evaluasi *forecasting* dan grafik *forecasting*.

Adapun kumpulan kebutuhan fungsional Statify Modul Analisis *Time Series* diurai dan disusun pada Tabel 2 sebagai berikut.

Tabel 2. Kebutuhan Fungsional Statify Modul Analisis *Time Series*

No.	Bagian Utama	Fungsional	Deskripsi
(1)	(2)	(3)	(4)
1	<i>Smoothing</i>	Akses metode <i>smoothing</i>	Pengguna dapat mengakses metode <i>smoothing</i> melalui tab <i>analyze</i> menu <i>time series</i> yang akan menampilkan antarmuka metode <i>smoothing</i>
2	<i>Smoothing</i>	Konfigurasi parameter <i>smoothing</i>	Pengguna dapat memilih metode <i>smoothing</i> dan mengatur <i>input</i> parameter sesuai dengan kebutuhan
3	<i>Smoothing</i>	Pilih variabel <i>smoothing</i>	Pengguna dapat memilih variabel <i>time series</i> yang ingin dihitung menggunakan metode <i>smoothing</i> yang dipilih sebelumnya
4	<i>Smoothing</i>	Konfigurasi waktu	Pengguna dapat mengatur spesifikasi waktu variabel <i>time series</i> yang sesuai
5	<i>Smoothing</i>	Mencetak hasil <i>smoothing</i>	Aplikasi memproses variabel, metode <i>smoothing</i> , dan parameter yang telah diatur oleh pengguna dalam bentuk tabel analisis, visualisasi grafik, dan variabel baru hasil penghitungan <i>smoothing</i>
6	<i>Decomposition</i>	Akses metode <i>decomposition</i>	Pengguna dapat mengakses metode <i>decomposition</i> melalui tab <i>analyze</i> menu <i>time series</i> yang akan menampilkan antarmuka metode <i>decomposition</i>
7	<i>Decomposition</i>	Konfigurasi parameter <i>decomposition</i>	Pengguna dapat memilih metode <i>decomposition</i> dan mengatur <i>input</i> parameter sesuai dengan kebutuhan
8	<i>Decomposition</i>	Pilih variabel <i>decomposition</i>	Pengguna dapat memilih variabel <i>time series</i> yang ingin didekomposisi
9	<i>Decomposition</i>	Konfigurasi waktu	Pengguna dapat mengatur spesifikasi waktu variabel <i>time series</i> yang sesuai
10	<i>Decomposition</i>	Mencetak hasil <i>decomposition</i>	Aplikasi memproses variabel, metode <i>decomposition</i> , dan parameter yang telah diatur oleh pengguna dalam bentuk tabel analisis, visualisasi grafik, dan variabel baru hasil penghitungan <i>decomposition</i>
11	<i>Autocorrelation</i>	Akses metode <i>autocorrelation</i>	Pengguna dapat mengakses metode <i>autocorrelation</i> melalui tab <i>analyze</i> menu <i>time series</i> yang akan menampilkan antarmuka metode <i>autocorrelation</i>

12	<i>Autocorrelation</i>	Konfigurasi parameter <i>autocorrelation</i>	Pengguna dapat mengatur <i>input</i> parameter metode <i>autocorrelation</i> sesuai dengan kebutuhan
13	<i>Autocorrelation</i>	Pilih variabel <i>autocorrelation</i>	Pengguna dapat memilih variabel <i>time series</i> yang ingin dilakukan <i>autocorrelation</i>
14	<i>Autocorrelation</i>	Konfigurasi tipe periode waktu	Pengguna dapat mengatur spesifikasi tipe periode waktu variabel <i>time series</i> yang sesuai
15	<i>Autocorrelation</i>	Mencetak hasil <i>autocorrelation</i>	Aplikasi memproses variabel dan parameter yang telah diatur oleh pengguna dalam bentuk tabel analisis dan visualisasi grafik
16	<i>Unit Root Test</i>	Akses metode <i>unit root test</i>	Pengguna dapat mengakses metode <i>unit root test</i> melalui tab <i>analyze</i> menu <i>time series</i> yang akan menampilkan antarmuka metode <i>unit root test</i>
17	<i>Unit Root Test</i>	Konfigurasi parameter <i>unit root test</i>	Pengguna dapat memilih metode <i>unit root test</i> dan mengatur <i>input</i> parameter sesuai dengan kebutuhan
18	<i>Unit Root Test</i>	Pilih variabel <i>unit root test</i>	Pengguna dapat memilih variabel <i>time series</i> yang ingin diuji menggunakan <i>unit root test</i>
19	<i>Unit Root Test</i>	Mencetak hasil <i>unit root test</i>	Aplikasi memproses variabel, metode <i>unit root test</i> , dan parameter yang telah diatur oleh pengguna dalam bentuk tabel analisis <i>unit root test</i>
20	<i>Box-Jenkins Model</i>	Akses metode <i>box-jenkins model</i>	Pengguna dapat mengakses metode <i>box-jenkins model</i> melalui tab <i>analyze</i> menu <i>time series</i> yang akan menampilkan antarmuka metode <i>box-jenkins model</i>
21	<i>Box-Jenkins Model</i>	Konfigurasi parameter <i>box-jenkins model</i>	Pengguna dapat mengatur <i>input</i> parameter <i>box-jenkins model</i> sesuai dengan kebutuhan
22	<i>Box-Jenkins Model</i>	Pilih variabel <i>box-jenkins model</i>	Pengguna dapat memilih variabel <i>time series</i> yang ingin dihitung menggunakan <i>box-jenkins model</i>
23	<i>Box-Jenkins Model</i>	Konfigurasi waktu	Pengguna dapat mengatur spesifikasi waktu variabel <i>time series</i> yang sesuai
24	<i>Box-Jenkins Model</i>	Mencetak hasil <i>box-jenkins model</i>	Aplikasi memproses variabel dan parameter yang telah diatur oleh pengguna dalam bentuk tabel analisis, visualisasi grafik, dan variabel baru hasil penghitungan <i>box-jenkins model</i>

Sumber: Dokumen Peneliti

4.1.3 Kebutuhan Non-Fungsional

Spesifikasi kebutuhan non-fungsional Statify ditentukan berdasarkan kualitas aplikasi secara komprehensif. Kebutuhan tersebut dirumuskan dalam beberapa aspek antara lain performa aplikasi yang optimal, usabilitas antarmuka pengguna, serta kualitas tampilan antarmuka berkontribusi terhadap pengalaman pengguna yang optimal. Kerangka kebutuhan non-fungsional ini dirancang dengan tujuan untuk meningkatkan efisiensi operasional sistem, memfasilitasi pemeliharaan aplikasi, dan mengakomodasi kemungkinan pengembangan aplikasi di masa mendatang. Adapun rincian kebutuhan non-fungsional Aplikasi Statify dituliskan pada Tabel 3.

Tabel 3. Kebutuhan Non-fungsional Statify Modul Analisis *Time Series*

No. (1)	Kategori (2)	Nonfungsional (3)	Deskripsi (4)
1	Performa	Kecepatan pemrosesan data	Aplikasi memiliki memproses data dengan cepat dengan bantuan teknologi WASM dengan kemampuan meminimumkan ukuran kode program
2	Aksesibilitas	Dapat diakses pada berbagai peramban web	Aplikasi dirancang sebagai aplikasi web yang memudahkan aksesibilitas pengguna tanpa proses instalasi di awal
3	Usabilitas	Memiliki antarmuka yang mudah dipahami dan digunakan	Aplikasi dirancang dengan tampilan antarmuka yang mudah dipahami dan digunakan oleh pengguna teknis maupun non teknis
4	Responsif	Tampilan antarmuka responsif	Aplikasi memiliki tampilan antarmuka yang mampu menyesuaikan rasio ukuran tampilan sesuai perangkat yang digunakan, yakni tampilan desktop dan <i>mobile</i>
5	Interaktif	Pesan <i>error</i> aktif ketika salah konfigurasi menu	Aplikasi memberikan pesan <i>error</i> yang mudah dipahami pengguna untuk mencegah

			kesalahan dalam memproses data
6	Maintanabilitas	Kode program dibuat secara modular	Kode program aplikasi ditulis secara modular guna mempermudah proses pengembangan aplikasi di masa mendatang

Sumber: Dokumen Peneliti

4.1.4 Daftar Fitur

Berdasarkan analisis kebutuhan yang telah dilakukan pada tahapan sebelumnya maka berikut adalah daftar fitur yang akan dibangun pada Aplikasi Statify Modul Analisis *Time Series* yang dituliskan pada Tabel 4.

Tabel 4. Daftar Fitur Statify Modul Analisis *Time Series*

No.	Major Feature Sets	Feature Sets	Features	Priority
(1)	(2)	(3)	(4)	(5)
1	<i>Smoothing</i>	Akses Menu <i>Smoothing</i>	Akses membuka menu <i>smoothing</i>	Kritis
2		Pengaturan Variabel	Menampilkan daftar variabel tersedia	Kritis
3			Memilih variabel	Kritis
4			Menghapus variabel terpilih	Kritis
5		Pengaturan Menu <i>Smoothing</i>	Memilih metode <i>smoothing</i>	Kritis
6			Manajemen input parameter <i>smoothing</i>	Kritis
7		Pengaturan Waktu	Memilih tipe periode waktu	Kritis
8			Manajemen <i>input</i> parameter waktu	Kritis
9		Histori Pengaturan	Menyimpan histori pengaturan	Rendah

			sesuai dengan penggunaan pengaturan terakhir kali	
10			Menampilkan tabel analisis hasil <i>smoothing</i>	Kritis
11		<i>Output Smoothing</i>	Menampilkan grafik hasil <i>smoothing</i>	Kritis
12			Menyimpan hasil <i>smoothing</i> dalam bentuk data set variabel	Kritis
13		Akses Menu <i>Decomposition</i>	Akses membuka menu <i>decomposition</i>	Kritis
14			Menampilkan daftar variabel tersedia	Kritis
15		Pengaturan Variabel	Memilih variabel	Kritis
16			Menghapus variabel terpilih	Kritis
17		Pengaturan Menu <i>Decomposition</i>	Memilih metode <i>decomposition</i>	Kritis
18			Memilih tipe periode waktu	Kritis
19		Pengaturan Waktu	Manajemen <i>input</i> parameter waktu	Kritis
20		Histori Pengaturan	Menyimpan histori pengaturan sesuai dengan penggunaan pengaturan terakhir kali	Rendah
21			Menampilkan tabel analisis hasil <i>decomposition</i>	Kritis
22		<i>Output Decomposition</i>	Menampilkan grafik hasil <i>decomposition</i>	Kritis

23			Menyimpan hasil <i>decomposition</i> dalam bentuk data set variabel	Kritis
24	<i>Autocorrelation</i>	Akses Menu <i>Autocorrelation</i>	Akses membuka menu <i>autocorrelation</i>	Kritis
25		Pengaturan Variabel	Menampilkan daftar variabel tersedia	Kritis
26			Memilih variabel	Kritis
27			Menghapus variabel terpilih	Kritis
28		Pengaturan Menu <i>Autocorrelation</i>	Manajemen <i>input</i> parameter <i>autocorrelation</i>	Kritis
29		Pengaturan Waktu	Memilih tipe periode waktu	Kritis
30			Manajemen <i>input</i> parameter waktu	Kritis
31		Histori Pengaturan	Menyimpan histori pengaturan sesuai dengan penggunaan pengaturan terakhir kali	Rendah
32		<i>Output Autocorrelation</i>	Menampilkan tabel analisis hasil penghitungan <i>autocorrelation</i>	Kritis
33			Menampilkan grafik hasil penghitungan <i>autocorrelation</i>	Kritis
34	<i>Unit Root Test</i>	Akses Menu <i>Unit Root Test</i>	Akses membuka menu <i>unit root test</i>	Kritis
35		Pengaturan Variabel	Menampilkan daftar variabel tersedia	Kritis

36			Memilih variabel	Kritis
37			Menghapus variabel terpilih	Kritis
38		Pengaturan Menu <i>Unit Root Test</i>	Memilih metode <i>unit root test</i>	Kritis
39			Manajemen <i>input</i> parameter <i>unit root test</i>	Kritis
40		Histori Pengaturan	Menyimpan histori pengaturan sesuai dengan penggunaan pengaturan terakhir kali	Rendah
41		<i>Output Unit Root Test</i>	Menampilkan tabel analisis hasil penghitungan <i>unit root test</i>	Kritis
42	<i>Box-Jenkins Model</i>	Akses Menu <i>Box-Jenkins Model</i>	Akses membuka menu <i>box-jenkins model</i>	Kritis
43		Pengaturan Variabel	Menampilkan daftar variabel tersedia	Kritis
44			Memilih variabel	Kritis
45			Menghapus variabel terpilih	Kritis
46		Pengaturan Menu <i>Box-Jenkins Model</i>	Manajemen <i>input</i> parameter <i>box-jenkins model</i>	Kritis
47		Pengaturan Waktu	Memilih tipe periode waktu	Kritis
48			Manajemen <i>input</i> parameter waktu	Kritis
49		Histori Pengaturan	Menyimpan histori pengaturan sesuai dengan penggunaan	Rendah

			pengaturan terakhir kali	
50			Menampilkan tabel analisis hasil estimasi model	Kritis
51			Menampilkan grafik hasil <i>forecasting</i>	Kritis
52		<i>Output Box-Jenkins Model</i>	Menyimpan hasil <i>forecasting</i> dalam bentuk data set variabel	Kritis

Sumber: Dokumen Peneliti

4.2 Perancangan

4.2.1 Perencanaan Jadwal Pengembangan Fitur

Guna memastikan setiap tahapan pengembangan Aplikasi Statify Modul Analisis *Time Series* dapat berjalan secara sistematis maka perlu dilakukan perancangan jadwal yang jelas di setiap tahapannya. Perancangan ini dituangkan dalam bentuk *Gantt Chart* yang di dalamnya mencakup setiap tahapan yang didasarkan dengan tahapan metode pengembangan FDD. Selain itu, pada perancangan nantinya juga dicantumkan estimasi durasi waktu pengerjaan untuk setiap daftar kegiatan agar proses pengerjaan dapat dilaksanakan dengan optimal. Berikut Gambar 10 sampai 15 menampilkan perencanaan pengembangan Aplikasi Statify Modul Analisis *Time Series* dalam bentuk *Gantt Chart* dengan menguraikan tahapan kegiatan dan estimasi durasi waktu pengerjaannya.

No	Kegiatan	Tanggal Mulai	Durasi Keseluruhan	Durasi Bulan Ini	Desember 2024																														
					1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
1	Develop an Overall Model	01/12/2024	21 Hari	21 Hari																															
1.1	Analisis Kebutuhan Umum	01/12/2024	4 Hari	4 Hari																															
1.2	Analisis Kebutuhan Fungsional	05/12/2024	12 Hari	12 Hari																															
1.3	Analisis Kebutuhan Nonfungsional	17/12/2024	5 Hari	5 Hari																															
2	Build a Feature List	22/12/2024	10 Hari	10 Hari																															
2.1	Membuat daftar fitur	22/12/2024	10 Hari	10 Hari																															

Gambar 10. Gantt Chart Statify Modul Analisis Time Series Desember 2024

No	Kegiatan	Tanggal Mulai	Durasi Keseluruhan	Durasi Bulan Ini	Januari 2025																														
					1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
3	Plan by Feature	01/01/2025	7 Hari	7 Hari																															
3.1	Merancang Jadwal	01/01/2025	7 Hari	7 Hari																															
4	Design by Feature	08/01/2025	24 Hari	24 Hari																															
4.1	Merancang Proses Akses Menu	08/01/2025	4 Hari	4 Hari																															
4.1.1	Membuat Sequence Diagram	08/01/2025	2 Hari	2 Hari																															
4.1.2	Membuat Desain Mockup	10/01/2025	2 Hari	2 Hari																															
4.2	Merancang Proses Pengaturan Variabel	12/01/2025	4 Hari	4 Hari																															
4.2.1	Membuat Sequence Diagram	12/01/2025	2 Hari	2 Hari																															
4.2.2	Membuat Desain Mockup	14/01/2025	2 Hari	2 Hari																															
4.3	Merancang Proses Pengaturan Opsi Menu	16/01/2025	4 Hari	4 Hari																															
4.3.1	Membuat Sequence Diagram	16/01/2025	2 Hari	2 Hari																															
4.3.2	Membuat Desain Mockup	18/01/2025	2 Hari	2 Hari																															
4.4	Merancang Proses Pengaturan Waktu	20/01/2025	4 Hari	4 Hari																															
4.4.1	Membuat Sequence Diagram	20/01/2025	2 Hari	2 Hari																															
4.4.2	Membuat Desain Mockup	22/01/2025	2 Hari	2 Hari																															
4.5	Merancang Proses Penyimpanan Histori Pengaturan	24/01/2025	4 Hari	4 Hari																															
4.5.1	Membuat Sequence Diagram	24/01/2025	2 Hari	2 Hari																															
4.5.2	Membuat Desain Mockup	26/01/2025	2 Hari	2 Hari																															
4.6	Merancang Proses Manajemen Output	28/01/2025	4 Hari	4 Hari																															
4.6.1	Membuat Sequence Diagram	28/01/2025	2 Hari	2 Hari																															
4.6.2	Membuat Desain Mockup	30/01/2025	2 Hari	2 Hari																															

Gambar 11. Gantt Chart Statify Modul Analisis Time Series Januari 2025

No	Kegiatan	Tanggal Mulai	Durasi Keseluruhan	Durasi Bulan Ini	Februari 2025																											
					1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
5	Build by Feature	01/02/2025	100 Hari	28 Hari																												
5.1	Membuat kode program menu smoothing	01/02/2025	20 Hari	20 Hari																												
5.1.1	Membuat kode program UI	01/02/2025	2 Hari	2 Hari																												
5.1.2	Membuat kode program client-side computing	03/02/2025	5 Hari	5 Hari																												
5.1.3	Integrasi kode program level menu	08/02/2025	1 Hari	1 Hari																												
5.1.4	Testing dan perbaikan	09/02/2025	10 Hari	10 Hari																												
5.1.5	Integrasi kode program ke aplikasi utama	19/02/2025	2 Hari	2 Hari																												
5.2	Membuat kode program menu decomposition	21/02/2025	20 Hari	8 Hari																												
5.2.1	Membuat kode program UI	21/02/2025	2 Hari	2 Hari																												
5.2.2	Membuat kode program client-side computing	23/02/2025	5 Hari	5 Hari																												
5.2.3	Integrasi kode program level menu	28/02/2025	1 Hari	1 Hari																												

Gambar 12. *Gantt Chart* Statify Modul Analisis *Time Series* Februari 2025

No	Kegiatan	Tanggal Mulai	Durasi Keseluruhan	Durasi Bulan Ini	Maret 2025																														
					1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
5	Build by Feature	01/02/2025	100 Hari	31 Hari																															
5.2	Membuat kode program menu decomposition	21/02/2025	20 Hari	12 Hari																															
5.2.4	Testing dan perbaikan	01/03/2025	10 Hari	10 Hari																															
5.2.5	Integrasi kode program ke aplikasi utama	11/03/2025	2 Hari	2 Hari																															
5.3	Membuat kode program menu autocorrelation	13/03/2025	20 Hari	19 Hari																															
5.3.1	Membuat kode program UI	13/03/2025	2 Hari	2 Hari																															
5.3.2	Membuat kode program client-side computing	15/03/2025	5 Hari	5 Hari																															
5.3.3	Integrasi kode program level menu	20/03/2025	1 Hari	1 Hari																															
5.3.4	Testing dan perbaikan	21/03/2025	10 Hari	10 Hari																															
5.3.5	Integrasi kode program ke aplikasi utama	31/03/2025	2 Hari	1 Hari																															

Gambar 13. *Gantt Chart* Statify Modul Analisis *Time Series* Maret 2025

No	Kegiatan	Tanggal Mulai	Durasi Keseluruhan	Durasi Bulan Ini	April 2025																												
					1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	Build by Feature	01/02/2025	100 Hari	30 hari																													
5.3	Membuat kode program menu autocorrelation	13/03/2025	20 Hari	1 Hari																													
5.3.5	Integrasi kode program ke aplikasi utama	31/03/2025	2 Hari	1 Hari																													
5.4	Membuat kode program menu unit root test	02/04/2025	20 Hari	20 Hari																													
5.4.1	Membuat kode program UI	02/04/2025	2 Hari	2 Hari																													
5.4.2	Membuat kode program client-side computing	04/04/2025	5 Hari	5 Hari																													
5.4.3	Integrasi kode program level menu	09/04/2025	1 Hari	1 Hari																													
5.4.4	Testing dan perbaikan	10/04/2025	10 Hari	10 Hari																													
5.4.5	Integrasi kode program ke aplikasi utama	20/04/2025	2 Hari	2 Hari																													
5.5	Membuat kode program menu box-jenkins model	22/04/2025	20 Hari	9 Hari																													
5.5.1	Membuat kode program UI	22/04/2025	2 Hari	2 Hari																													
5.5.2	Membuat kode program client-side computing	24/04/2025	5 Hari	5 Hari																													
5.5.3	Integrasi kode program level menu	29/04/2025	1 Hari	1 Hari																													
5.5.4	Testing dan perbaikan	30/04/2025	10 Hari	1 Hari																													

Gambar 14. *Gantt Chart* Statify Modul Analisis *Time Series* April 2025

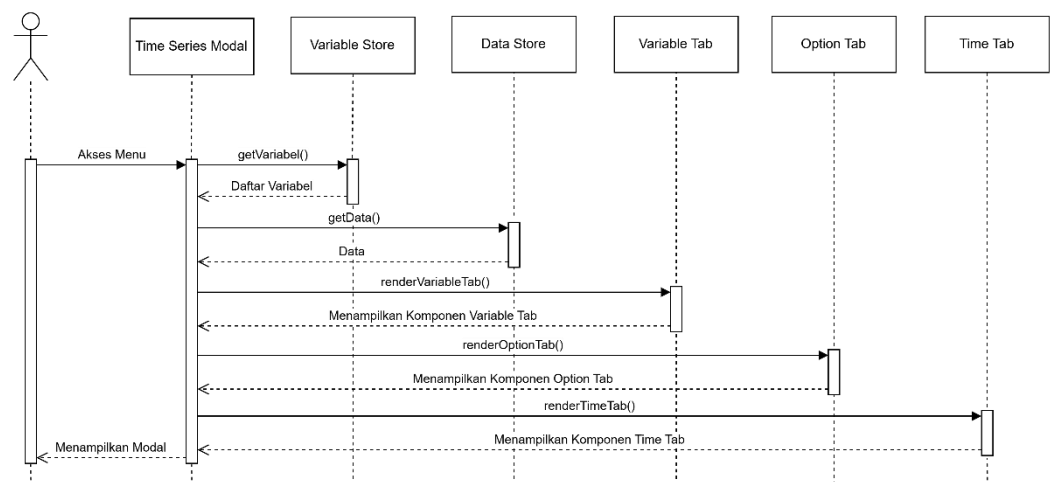
No	Kegiatan	Tanggal Mulai	Durasi Keseluruhan	Durasi Bulan Ini	Mei 2025																														
					1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
5	Build by Feature	01/02/2025	100 Hari	11 Hari																															
5.5	Membuat kode program menu box-jenkins model	22/04/2025	20 Hari	11 Hari																															
5.5.4	Testing dan perbaikan	30/04/2025	10 Hari	9 Hari																															
5.5.5	Integrasi kode program ke aplikasi utama	10/05/2025	2 Hari	2 Hari																															

Gambar 15. *Gantt Chart* Statify Modul Analisis *Time Series* Mei 2025

4.2.2 Sequence Diagram

Perancangan proses bisnis dalam membangun suatu aplikasi merupakan suatu hal yang penting sebelum memasuki tahap desain aplikasi. Pada penelitian ini perancangan proses bisnis Aplikasi Statify Modul Analisis *Time Series* didasarkan pada kegiatan (*feature sets*) yang diperoleh dari daftar fitur tahapan sebelumnya. Dikarenakan setiap area domain (*major feature sets*) pada modul ini memiliki kesamaan daftar kegiatan maka kegiatan yang ada akan dirancang secara umum yang ditampilkan dalam bentuk *Sequence Diagram* beserta dengan deskripsi proses kegiatannya.

1. Mengakses Menu *Time Series*



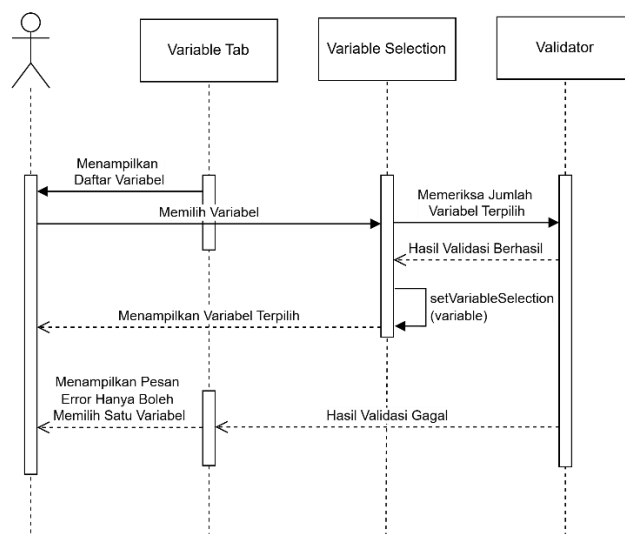
Gambar 16. *Sequence Diagram* Kegiatan Mengakses Menu *Time Series*

Deskripsi Proses:

- 1) Pengguna mengakses menu *time series* untuk membuka fitur *Time Series Modal* pada antarmuka aplikasi.
- 2) *Time series modal* memanggil fungsi *getVariabel()* pada *Variable Store* untuk memperoleh informasi variabel yang tersedia pada data set.

- 3) *Time Series Modal* memanggil fungsi *getData()* pada *Data Store* untuk memperoleh seluruh data set yang tersedia.
- 4) *Time Series Modal* memanggil fungsi *renderVariableTab()* untuk menampilkan komponen *Variable Tab*.
- 5) *Time Series Modal* memanggil fungsi *renderOptionTab()* untuk menampilkan komponen *Option Tab*.
- 6) *Time Series Modal* memanggil fungsi *renderTimeTab()* untuk menampilkan komponen *Time Tab*.
- 7) Setelah seluruh komponen telah dibentuk maka *Time Series Modal* akan menampilkan antarmuka *Time Series Modal* secara keseluruhan.

2. Mengatur Variabel



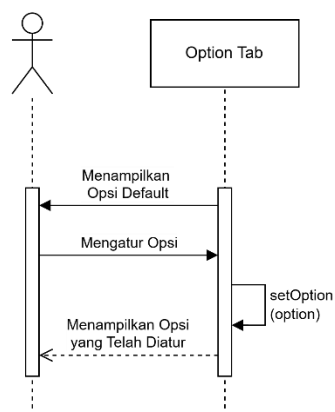
Gambar 17. *Sequence Diagram* Kegiatan Mengatur Variabel

Deskripsi Proses:

- 1) Komponen *Variable Tab* menampilkan daftar variabel yang tersedia pada data set

- 2) Pengguna memilih variabel yang akan digunakan pada menu *time series* ke komponen *Variabel Selection*.
- 3) Validator memeriksa apakah sudah terdapat variabel terpilih pada komponen *Variabel Selection*.
- 4) Jika tidak terdapat variabel terpilih maka *Validator* akan memberikan hasil validasi berhasil ke komponen *Variabel Selection*.
- 5) Komponen *Variabel Selection* menjalankan fungsi *setVariableSelection(variable)* untuk mengubah nilai variabel terpilih sesuai pilihan pengguna.
- 6) Komponen *Variabel Selection* menampilkan variabel terpilih.
- 7) Namun jika terdapat variabel terpilih maka *Validator* akan memberikan hasil validasi gagal yang langsung diberikan ke komponen *Variabel Tab*.
- 8) Komponen *Variabel Tab* menampilkan pesan *error* hanya boleh memilih satu variabel.

3. Mengatur Opsi Menu *Time Series*

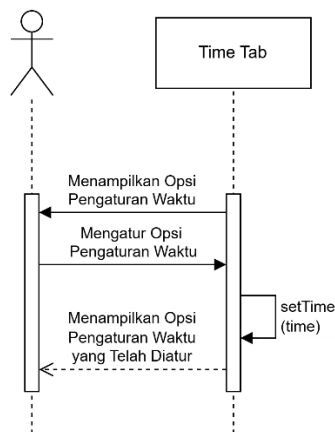


Gambar 18. *Sequence Diagram* Kegiatan Mengatur Opsi Menu *Time Series*

Deskripsi Proses:

- 1) Komponen *Option Tab* menampilkan daftar opsi menu *time series* yang nilai sudah terisi secara *default*.
- 2) Pengguna mengatur daftar opsi yang tersedia.
- 3) Komponen *Option Tab* menjalankan fungsi *setOption(option)* untuk mengubah nilai opsi pengaturan menu *time series* sesuai dengan pilihan pengguna.
- 4) Komponen *Option Tab* menampilkan daftar opsi menu *time series* yang telah diatur oleh pengguna.

4. Mengatur Opsi Pengaturan Waktu



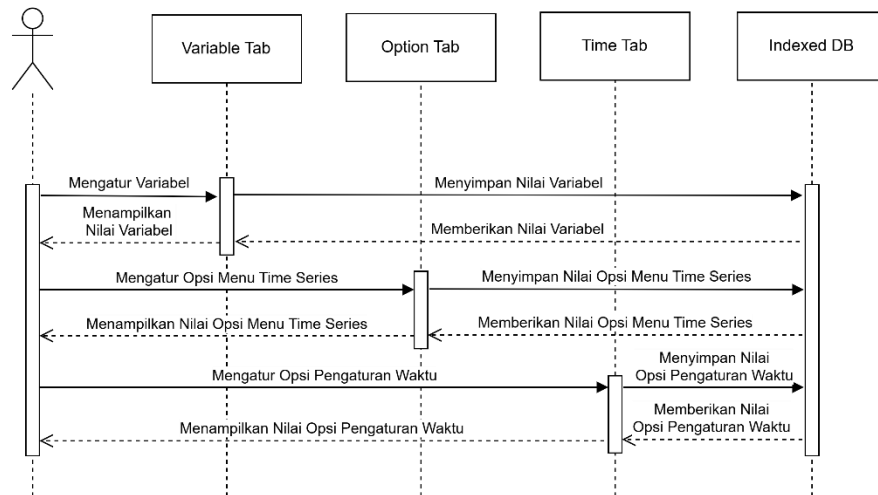
Gambar 19. *Sequence Diagram* Kegiatan Mengatur Opsi Pengaturan Waktu

Deskripsi Proses:

- 1) Komponen *Time Tab* menampilkan opsi pengaturan waktu.
- 2) Pengguna mengatur opsi pengaturan waktu sesuai dengan keperluan penggunaan.
- 3) Komponen *Time Tab* menjalankan fungsi *setTime(time)* untuk mengubah nilai opsi pengaturan waktu sesuai dengan pilihan pengguna.

- 4) Komponen *Time Tab* menampilkan daftar opsi yang telah diatur oleh pengguna.

5. Menyimpan Histori Pengaturan

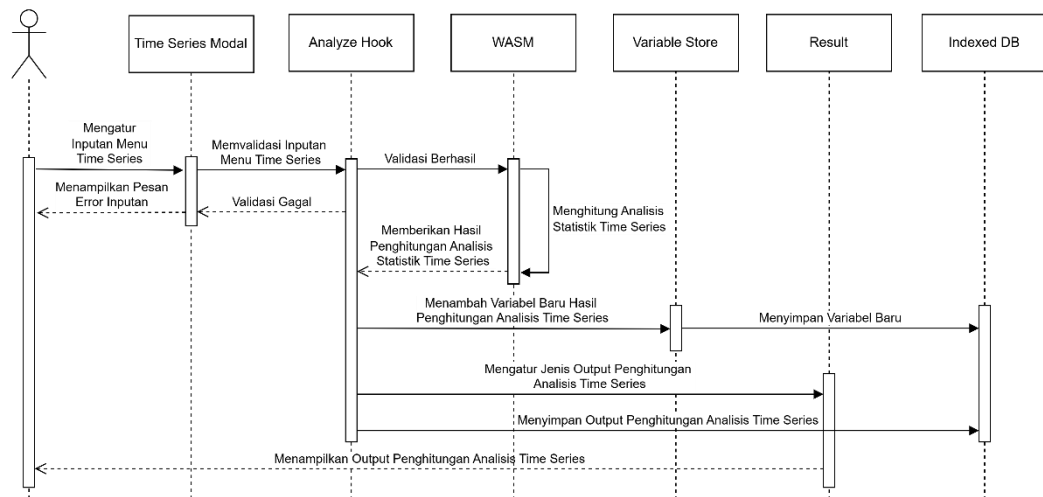


Gambar 20. *Sequence Diagram* Kegiatan Menyimpan Histori Pengaturan

Deskripsi Proses:

- 1) Pengguna mengatur pengaturan penggunaan setiap komponen *tab*, yaitu *Variable Tab*, *Option Tab*, dan *Time Tab*.
- 2) Setiap komponen *tab* menyimpan nilai hasil pengaturan pengguna ke *Indexed DB*.
- 3) *Indexed DB* mengembalikan nilai pengaturan yang telah disimpan ke masing-masing komponen *tab*.
- 4) Setiap komponen *tab* menampilkan nilai yang telah disimpan di *Indexed DB*.

6. Menghitung *Output Menu Time Series*



Gambar 21. *Sequence Diagram* Kegiatan Menghitung *Output Menu Time Series*

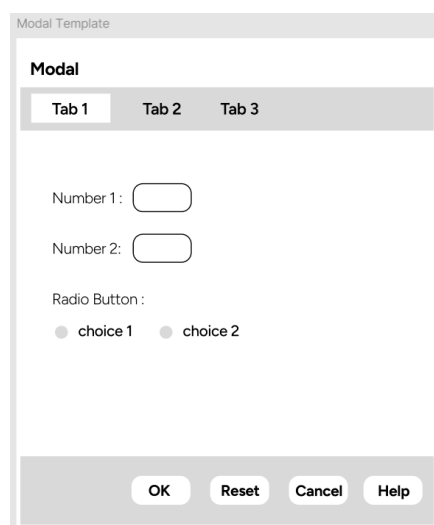
Deskripsi Proses:

- 1) Pengguna mengatur inputan menu *time series* pada *Time Series Modal*.
- 2) *Time Series Modal* memvalidasi inputan menu *time series* pada *Analyze Hook*.
- 3) Jika inputan tidak valid, *Analyze Hook* mengirim nilai validasi gagal ke *Time Series Modal*.
- 4) *Time Series Modal* menampilkan pesan *error* inputan yang menunjukkan letak inputan yang tidak valid, sehingga pengguna dapat memperbaiki inputan sebelum dilanjutkan ke proses penghitungan.
- 5) Jika inputan valid, *Analyze Hook* mengirim inputan ke WASM dalam bentuk kumpulan parameter.
- 6) WASM menghitung analisis *time series* sesuai dengan permintaan dan parameter pilihan pengguna lalu memberikan hasilnya ke *Analyze Hook* kembali.

- 7) *Analyze Hook* menambahkan variabel baru hasil penghitungan analisis *time series* ke *Variable Store*.
- 8) *Variable Store* menyimpan variabel baru ke dalam *Indexed DB*.
- 9) Selanjutnya *Analyze Hook* mengatur jenis *output* yang digunakan pada halaman *Result* dan menyimpan output tersebut ke *Indexed DB*.
- 10) Halaman *Result* menampilkan output penghitungan analisis statistik *time series*.

4.2.3 Desain Antarmuka Pengguna

Tampilan antarmuka Aplikasi Statify Modul Analisis *Time Series* dirancang menyesuaikan dengan rancangan dasar desain yang telah dirancang bersama-sama di awal perancangan. Perancangan dasar desain ini ditujukan untuk memberikan tampilan visual yang konsisten untuk setiap antarmuka menu yang terdapat di aplikasi ini. Adapun rancangan dasar desain untuk tampilan manajemen menu seperti yang ditampilkan pada Gambar 22.



The image shows a mockup of a 'Modal Template' window. It has a title bar 'Modal Template' and a header 'Modal'. Below the header are three tabs: 'Tab 1', 'Tab 2', and 'Tab 3'. The main content area contains the following elements: 'Number 1:' followed by a text input field; 'Number 2:' followed by a text input field; 'Radio Button:' followed by two radio buttons labeled 'choice 1' and 'choice 2'. At the bottom, there is a footer bar with four buttons: 'OK', 'Reset', 'Cancel', and 'Help'.

Gambar 22. *Mockup* Rancangan Dasar Desain Modal Statify

Seperti yang tertera pada Gambar 18 rancangan dasar desain antarmuka setiap menu dirancang dalam bentuk pengelompokan pengaturan yang terpisah dalam bentuk tampilan tab. Dengan demikian peneliti membuat rancangan desain untuk setiap menu Statify Modul Analisis *Time Series* terdiri tiga tab pengaturan yaitu tab *variable*, tab *option*, dan tab *time*. Tab *time* memiliki desain yang sama untuk setiap menu analisis *time series* kecuali pada menu *unit root test* tidak terdapat tab *time* karena penghitungannya tidak melibatkan parameter waktu. Begitu juga dengan tab variabel memiliki desain yang hampir sama untuk setiap menu kecuali pada menu *autocorrelation* dan *unit root test*. Sedangkan tab *option* memiliki desain yang berbeda untuk setiap menu analisis *time series*. Gambar 23 sampai 30 menampilkan desain antarmuka untuk setiap tampilan menu analisis *time series* dalam bentuk *Mockup*.

Variable Tab With Save

Time Series Modal

Variables Option Tab Time

Available Variables

variable 1
variable 2
variable 3
variable 4

Selected Variable :

variable 5

☐ Save Forecasting as Variable

OK Reset Cancel Help

Gambar 23. *Mockup Variable Tab Menu Time Series Dengan Checkbox Menyimpan Forecasting Sebagai Variabel*

Variable Tab Without Save

Time Series Modal

Variables
Option Tab
Time

Available Variables
Selected Variable :

variable 1
variable 2
variable 3
variable 4

variable 5

OK
Reset
Cancel
Help

Gambar 24. Mockup Variable Tab Menu Time Series Tanpa Checkbox Menyimpan Forecasting Sebagai Variabel

Time Tab

Time Series Modal

Variables
Option Tab
Time

Time Option:

time spesification : week-work days(5) ▼

year : 2025

month : 1

day : 1

periodicity: 5

start time : Wednesday, 1 Jan 2025

OK
Reset
Cancel
Help

Gambar 25. Mockup Time Tab Menu Time Series

Smoothing Tab

Smoothing

Variables Smoothing Option Time

Smoothing Method

method : Holt's Method Exponential Smo.....▼

alpha : 0.2

beta : 0.2

OK Reset Cancel Help

Gambar 26. Mockup Option Tab Menu Smoothing

Decomposition Tab

Decomposition

Variables Decomposition Option Time

Decomposition Method

☐ Additive ☒ Multiplicative

trend

☐ Linear ☒ Exponential

OK Reset Cancel Help

Gambar 27. Mockup Option Tab Menu Decomposition

Autocorrelation Tab

Autocorrelation

Variables Autocorrelation Option Time

Autocorrelation Options:

maximum lags :

☐ seasonally

autocorrelate on:

☐ level ☒ first difference ☐ second difference

OK Reset Cancel Help

Gambar 28. Mockup Option Tab Menu Autocorrelation

Unit Root Test Tab

Unit Root Test

Variables Unit Root Test Option Time

Methods:

☐ [dickey-fuller](#)

☒ augmented dickey-fuller

Unit Root Test Options:

calculate on :

☐ level ☒ first difference ☐ second difference

equation :

☐ none ☒ intercept ☐ trend and intercept

lag length :

OK Reset Cancel Help

Gambar 29. Mockup Option Tab Menu Unit Root Test

Box-Jenkins Model Tab

Box-Jenkins Model

VariablesBox-Jenkins Model OptionTime

Parameter Order:
p: 0q: 1d: 0
Note: Method used is Conditional Least Square

OKResetCancelHelp

Gambar 30. Mockup Option Tab Menu Box-Jenkins Model

BAB V

IMPLEMENTASI DAN EVALUASI

5.1 Implementasi

5.1.1 Lingkungan Implementasi

Berdasarkan hasil analisis kebutuhan dan kesepakatan bersama seluruh anggota tim, Statify dibangun menggunakan *framework* next.js dengan beberapa komponen pendukung, antara lain Tailwind CSS, Shadcn UI, dan WASM. *Framework* Next.js adalah salah satu *framework* pengembangan web dengan bahasa pemrograman JavaScript atau TypeScript. *Framework* ini dipilih oleh peneliti dan tim pengembang lainnya karena memiliki struktur folder kode program yang fleksibel untuk disusun dan diubah sesuai dengan kebutuhan pengembangan. Dengan fleksibilitas ini tim dapat menyusun kerangka awal struktur folder dan menyepakati aturan pembuatan struktur folder sehingga pengembangan kode program dapat dilakukan secara modular dan terstruktur serta mempermudah proses integrasi kode program antar modul.

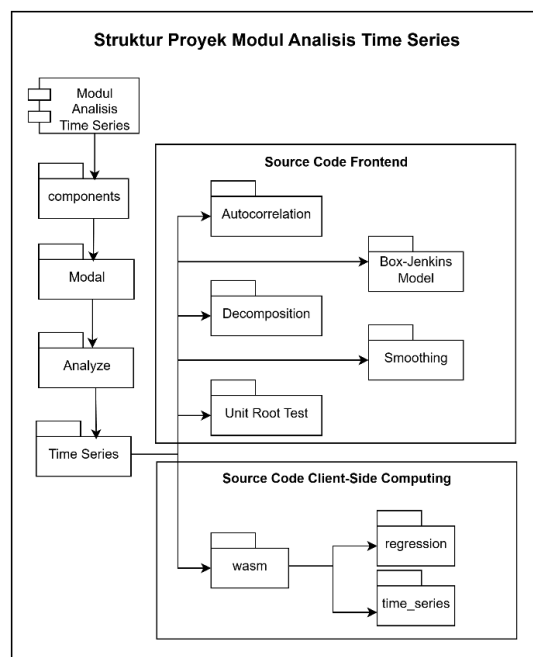
Selain itu, Tailwind CSS juga digunakan dalam pengembangan Statify yang berfungsi sebagai *framework front-end* aplikasi. *Framework* ini juga memerlukan kerangka desain komponen antarmuka untuk mempermudah proses desain antarmuka aplikasi. Desain komponen antarmuka yang digunakan pada Statify adalah Shadcn UI yang bersesuaian dengan desain setiap aplikasi web di lingkungan Politeknik Statistika STIS.

Terakhir, Statify menggunakan teknologi WASM dalam proses kalkulasi data. Teknologi ini adalah suatu mesin kalkulasi yang mengompilasi bahasa

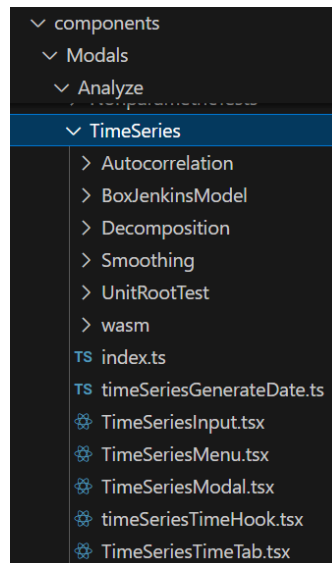
pemrograman C, C++, dan Rust ke dalam format instruksi biner. Teknologi ini digunakan dalam Statify dengan tujuan untuk memberikan performa kalkulasi data yang cepat di sisi klien. Selain itu, Statify bahasa pemrograman yang digunakan dalam WASM adalah Rust. Rust digunakan karena memiliki kemampuan proses kalkulasi yang cepat dan penggunaan memori yang efisien yang bersesuaian dengan kebutuhan non-fungsional Statify yang membutuhkan performa yang baik.

5.1.2 Struktur Proyek

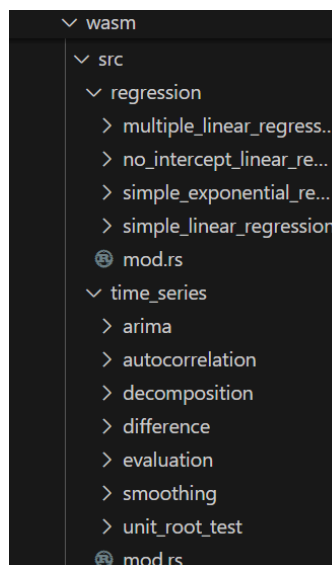
Struktur proyek Statify terdiri atas beberapa folder utama, antara lain *folder app, components, hooks, public, types, dan stores*. Peneliti pada penelitian ini berfokus membangun kode program antarmuka modal menu analisis *time series* yang ditulis pada folder *components* dan membangun kode program penghitungan analisis *time series* menggunakan WASM. Adapun struktur proyek Statify Modul Analisis *Time Series* seperti pada Gambar 31 sampai 33.



Gambar 31. Struktur Proyek Statify Modul Analisis *Time Series*



Gambar 32. Struktur Folder *Source Code* Antarmuka Modal Menu *Time Series*



Gambar 33. Struktur Folder *Source Code* Penghitungan Menu *Time Series*

5.1.3 Implementasi Fitur

Berdasarkan analisis kebutuhan dan rancangan yang telah dilakukan pada tahapan sebelumnya, diimplementasikan lima fitur menu *time series* yang bersesuaian dengan RPS mata kuliah Analisis Runtun Waktu yaitu Sub-CPMK (Capaian Pembelajaran Mata Kuliah) Pertemuan 2 sampai 6. Sub-CPMK setiap pertemuan masing-masing membahas metode-metode yang digunakan pada proses analisis *time series*, seperti pertemuan 2 membahas metode *decomposition*, pertemuan 3 membahas metode *smoothing*, pertemuan 4 membahas metode *autocorrelation* dan *unit root test*, serta pertemuan 5 dan 6 membahas model ARMA pada data stasioner dan model ARIMA pada data tidak stasioner yang dicakup dalam *box-jenkins model*. Namun, Sub-CPMK Pertemuan 1 tidak dimasukkan ke dalam implementasi fitur menu *time series* dikarenakan pertemuan tersebut berfokus pada pengenalan konsep-konsep dasar *time series*. Adapun untuk dapat menggunakan Statify, pengguna dapat mengaksesnya melalui Lampiran 2. Berikut ditampilkan implementasi fitur yang terdapat pada setiap menu Statify Modul Analisis *Time Series*.

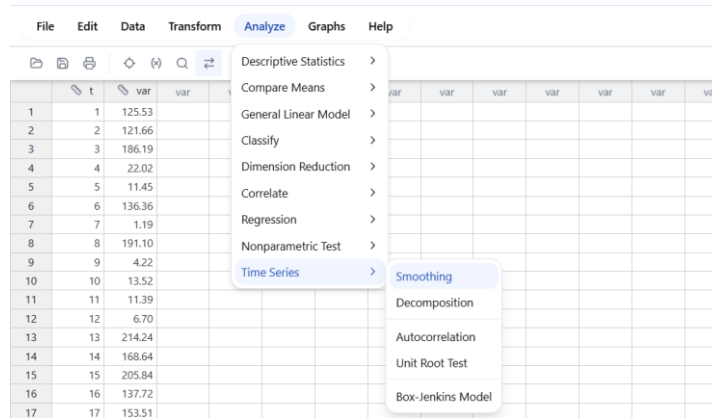
1. Implementasi Menu *Smoothing*

Menu *smoothing* mengimplementasikan materi pembelajaran Analisis Runtun Waktu Sub-CPMK Pertemuan 3. Adapun untuk menggunakan menu *smoothing* pada aplikasi ini, pengguna dapat melakukan tahapan-tahapan sebagai berikut.

1) Mengakses menu *smoothing*

Untuk mengakses menu *smoothing* pengguna dapat memilih tab *analyze* pada *toolbar* lalu memilih menu *time series* yang selanjutnya

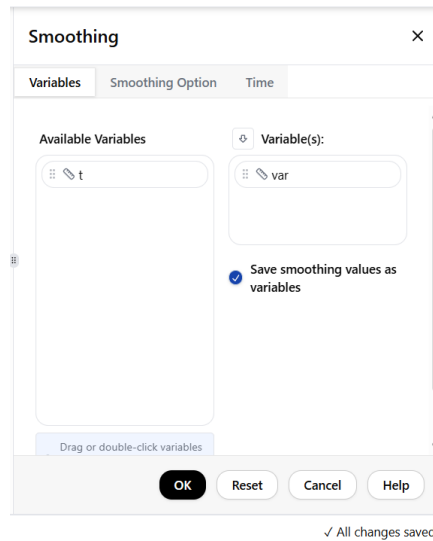
akan terdapat pilihan untuk mengakses menu *smoothing* seperti yang ditampilkan pada Gambar 34.



Gambar 34. Implementasi Mengakses Menu *Smoothing*

2) Mengonfigurasi variabel

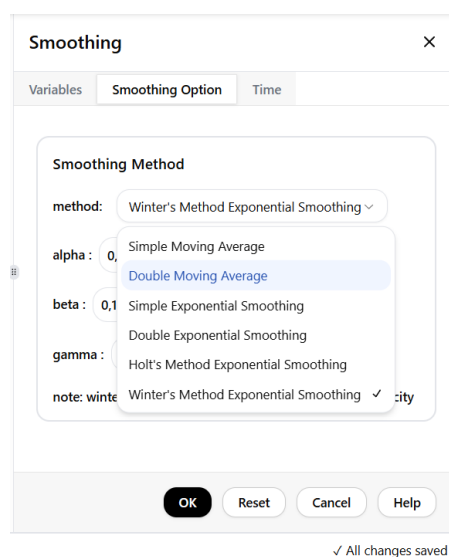
Pada modal menu *smoothing* pengguna dapat mengonfigurasi variabel pada tab *variable*. Pada tab tersebut pengguna dapat memilih satu variabel yang ingin digunakan, menghapus variabel terpilih, hingga menyimpan hasil *smoothing* sebagai variabel atau tidak pada *checkbox* yang tersedia. Berikut Gambar 35 menampilkan antarmuka tab *variable* menu *smoothing*.



Gambar 35. Implementasi Mengonfigurasi Variabel Menu *Smoothing*

3) Mengonfigurasi opsi menu *smoothing*

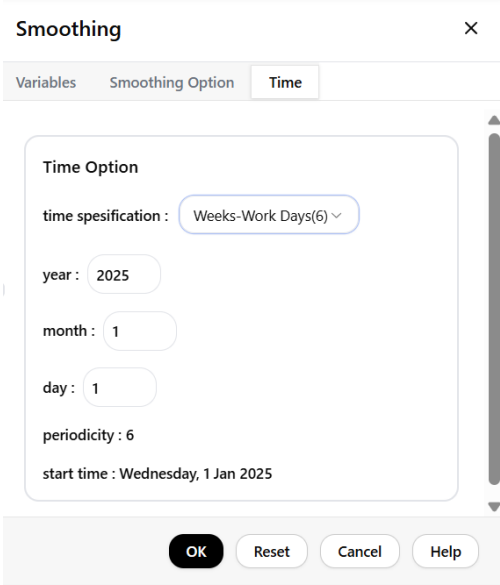
Untuk mengakses opsi menu *smoothing* pengguna dapat memilih tab *smoothing option*. Selanjutnya pengguna dapat mengonfigurasi opsi menu *smoothing* sesuai dengan kebutuhan seperti yang tampilkan pada Gambar 36.



Gambar 36. Implementasi Mengonfigurasi Opsi Menu *Smoothing*

4) Mengonfigurasi waktu

Untuk mengakses pengaturan waktu menu *smoothing* pengguna dapat memilih tab *time*. Selanjutnya pengguna dapat mengonfigurasi waktu menu *smoothing* sesuai dengan kebutuhan seperti yang ditampilkan pada Gambar 37.



The image shows a 'Smoothing' dialog box with a close button (X) in the top right corner. It has three tabs: 'Variables', 'Smoothing Option', and 'Time'. The 'Time' tab is selected. Inside the 'Time' tab, there is a 'Time Option' section with the following fields: 'time specification' (a dropdown menu showing 'Weeks-Work Days(6)'), 'year' (a text input field with '2025'), 'month' (a text input field with '1'), 'day' (a text input field with '1'), 'periodicity' (a text input field with '6'), and 'start time' (a text input field with 'Wednesday, 1 Jan 2025'). At the bottom of the dialog box, there are four buttons: 'OK', 'Reset', 'Cancel', and 'Help'. Below the dialog box, there is a status message: '✓ All changes saved'.

Gambar 37. Implementasi Mengonfigurasi Waktu Menu *Smoothing*

5) Menghitung *smoothing*

Untuk menghitung *smoothing* pengguna dapat menekan tombol *ok* pada modal menu *smoothing*. Selanjutnya aplikasi akan menghitung *smoothing* pada sisi klien menggunakan kode program hasil kompilasi WASM. Adapun potongan kode program penghitungan *smoothing* ditampilkan pada Gambar 38.

```

frontend > src > wasm > src > time_series > smoothing > @ advance_exponential_smoothing.rs > ...
5 impl Smoothing{
26 pub fn calculate_winter(&self, alpha:f64, beta:f64, gamma:f64, period:usize) -> Vec<f64> {
27     let mut level: Vec<f64> = Vec::new();
28     let mut trend: Vec<f64> = Vec::new();
29     let mut seasonal: Vec<f64> = Vec::new();
30     let data: Vec<f64> = self.get_data();
31     let mut winter_values: Vec<f64> = Vec::new();
32     for i in 0..data.len(){
33         if i == 0{
34             level.push(data[i]);
35             trend.push(0.0);
36             seasonal.push(1.0);
37             winter_values.push(data[i]);
38         }
39         else if i < period{
40             level.push(alpha * (data[i] / seasonal[0]) + (1.0 - alpha) * (level[i-1] + trend[i-1]));
41             trend.push(beta * (level[i] - level[i-1]) + (1.0 - beta) * trend[i-1]);
42             seasonal.push(gamma * (data[i] / level[i]) + (1.0 - gamma) * seasonal[0]);
43             winter_values.push(data[0]);
44         }
45         else{
46             level.push(alpha * (data[i] / seasonal[i-period]) + (1.0 - alpha) * (level[i-1] + trend[i-1]));
47             trend.push(beta * (level[i] - level[i-1]) + (1.0 - beta) * trend[i-1]);
48             seasonal.push(gamma * (data[i] / level[i]) + (1.0 - gamma) * seasonal[i-period]);
49             winter_values.push((level[i-1] + trend[i-1]) * seasonal[i-period]);

```

Gambar 38. Implementasi Potongan Kode Program Menu *Smoothing*

6) Melihat variabel baru

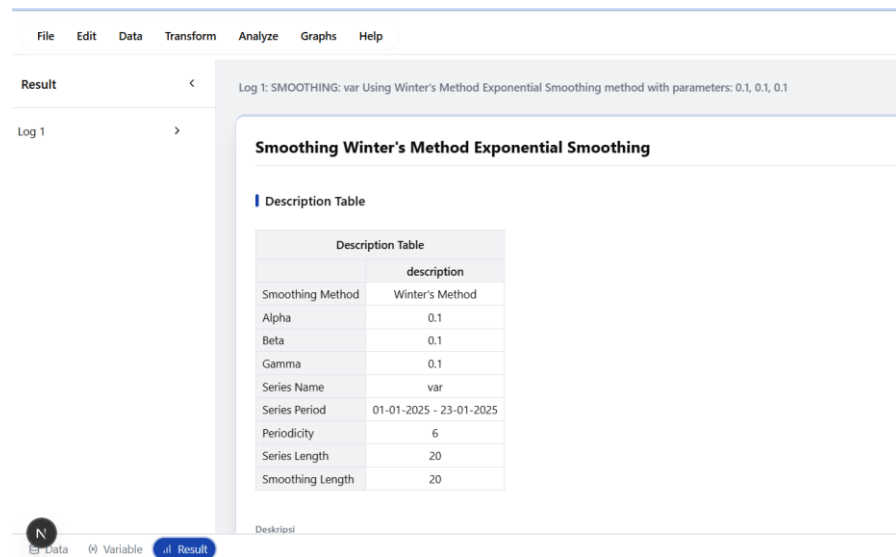
Untuk melihat variabel baru hasil *smoothing* pengguna dapat memilih halaman Data seperti yang ditampilkan pada Gambar 39.

File Edit Data Transform Analyze Graphs Help												
📁 📄 🔍 ↺												
	t	var	var_winter_	var	var	var	var	var	var	var	var	var
1	1	125.53	125.53									
2	2	121.66	125.53									
3	3	186.19	125.53									
4	4	22.02	125.53									
5	5	11.45	125.53									
6	6	136.36	125.53									
7	7	1.19	109.32									
8	8	191.10	95.83									
9	9	4.22	108.56									
10	10	13.52	84.22									
11	11	11.39	73.56									
12	12	6.70	71.69									
13	13	214.24	53.32									
14	14	168.64	80.09									
15	15	205.84	75.88									
16	16	137.72	79.01									
17	17	153.51	84.22									
18	18	156.15	102.60									
19	19	4.63	127.93									
20	20	2.07	125.19									
21												
22												
23												

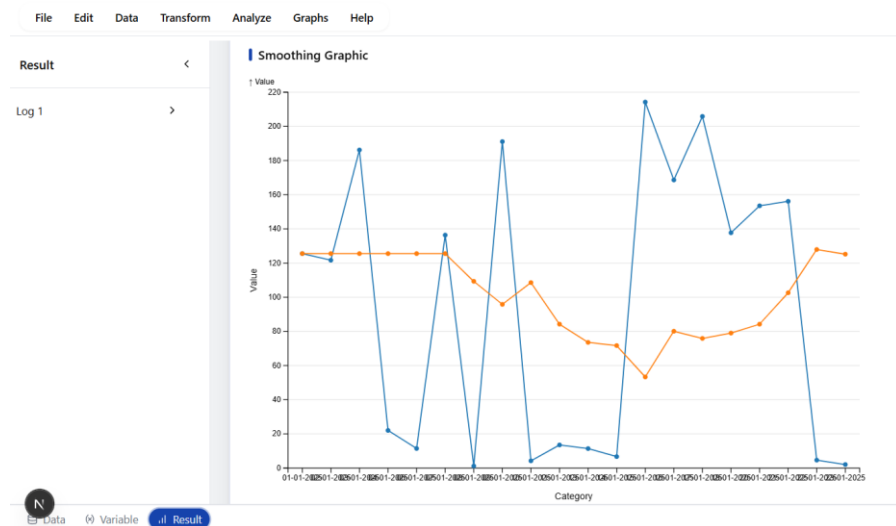
Gambar 39. Implementasi Melihat Variabel Baru Menu *Smoothing*

7) Melihat hasil analisis *smoothing*

Untuk melihat tabel analisis dan grafik *smoothing* pengguna dapat memilih halaman *Result* seperti yang ditampilkan pada Gambar 40 dan 41.



Gambar 40. Implementasi Tabel Analisis Menu *Smoothing*



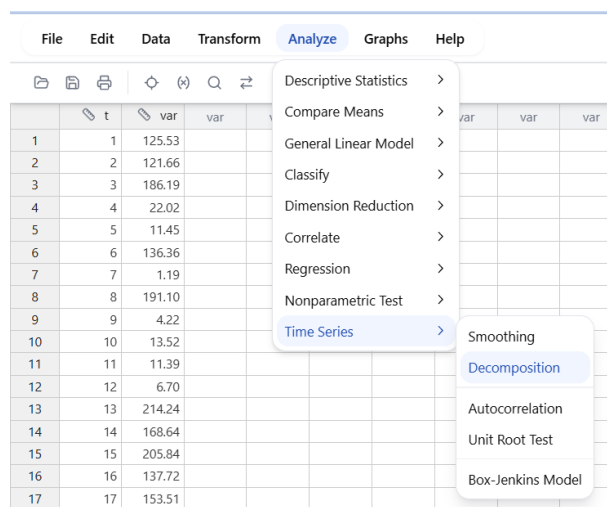
Gambar 41. Implementasi Grafik *Smoothing*

2. Implementasi Menu *Decomposition*

Menu *decomposition* mengimplementasikan materi pembelajaran Analisis Runtun Waktu Sub-CPMK Pertemuan 2. Adapun untuk menggunakan menu *decomposition* pada aplikasi ini, pengguna dapat melakukan tahapan-tahapan sebagai berikut.

1) Mengakses menu *decomposition*

Untuk mengakses menu *decomposition* pengguna dapat memilih tab *analyze* pada *toolbar* lalu memilih menu *time series* yang selanjutnya akan terdapat pilihan untuk mengakses menu *decomposition* seperti yang ditampilkan pada Gambar 42.

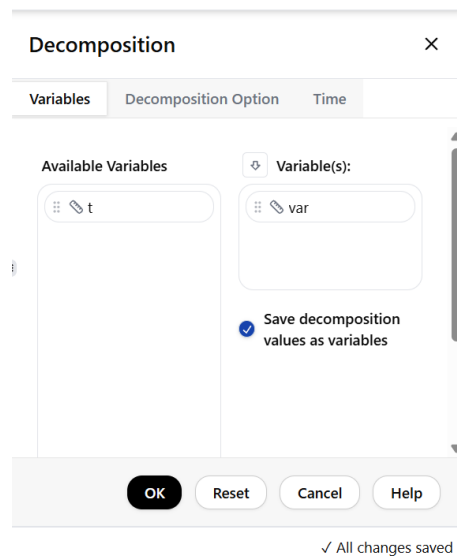


Gambar 42. Implementasi Mengakses Menu *Decomposition*

2) Mengonfigurasi variabel

Pada modal menu *decomposition* pengguna dapat mengonfigurasi variabel pada tab *variable*. Pada tab tersebut pengguna dapat memilih satu variabel yang ingin digunakan, menghapus variabel terpilih,

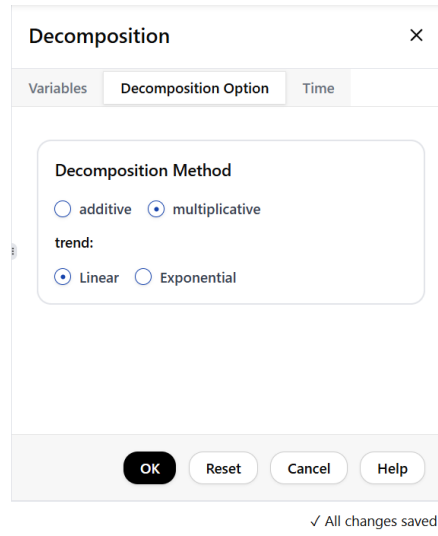
hingga menyimpan hasil *decomposition* sebagai variabel atau tidak pada *checkbox* yang tersedia. Berikut Gambar 43 menampilkan antarmuka tab *variable* menu *decomposition*.



Gambar 43. Implementasi Mengonfigurasi Variabel Menu *Decomposition*

3) Mengonfigurasi opsi menu *decomposition*

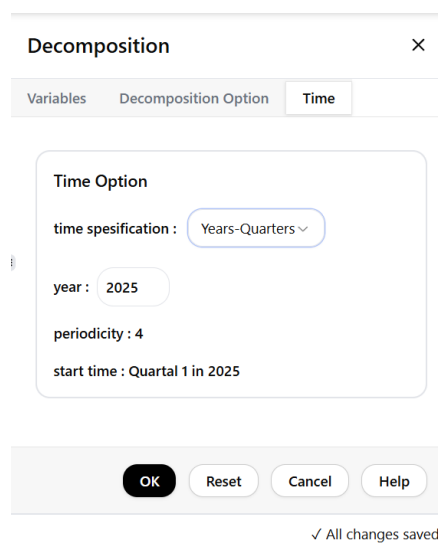
Untuk mengakses opsi menu *decomposition* pengguna dapat memilih tab *decomposition option*. Selanjutnya pengguna dapat mengonfigurasi opsi menu *decomposition* sesuai dengan kebutuhan seperti yang tampilkan pada Gambar 44.



Gambar 44. Implementasi Mengonfigurasi Opsi Menu *Decomposition*

4) Mengonfigurasi waktu

Untuk mengakses pengaturan waktu menu *decomposition* pengguna dapat memilih tab *time*. Selanjutnya pengguna dapat mengonfigurasi waktu menu *decomposition* sesuai dengan kebutuhan seperti yang tampilkan pada Gambar 45.



Gambar 45. Implementasi Mengonfigurasi Waktu Menu *Decomposition*

5) Menghitung *decomposition*

Untuk menghitung *decomposition* pengguna dapat menekan tombol *ok* pada modal menu *decomposition*. Selanjutnya aplikasi akan menghitung *decomposition* pada sisi klien menggunakan kode program hasil kompilasi WASM. Adapun potongan kode program penghitungan *decomposition* ditampilkan pada Gambar 46.

```
frontend > src > wasm > src > time_series > decomposition > multiplicative > multiplicative_season.rs > ...
1 use wasm_bindgen::prelude::*;
2 use crate::Decomposition;
3
4 #[wasm_bindgen]
5 impl Decomposition{
6     // Calculate seasonal index
7     pub fn calculate_multiplicative_seasonal_component(&mut self, centered_ma: Vec<f64>)->Vec<f64>{
8         // Initialize the variables
9         let mut seasonal_indices: Vec<f64> = Vec::new(); // seasonal indices
10        let mut seasonal_component: Vec<f64> = Vec::new(); // seasonal components
11        let centered_ma_values: Vec<f64> = centered_ma.clone();
12
13        // Create vector index
14        let mut index: Vec<usize> = Vec::new();
15        let mut index_values = 0;
16        for _i in 0..self.get_data().len(){
17            if index_values == self.get_period(){
18                index_values = 0;
19            }
20            index.push(index_values as usize);
21            index_values += 1;
22        }
23
24        // Calculate seasonal indices
25        let mut centered_index = Vec::new();
```

Gambar 46. Implementasi Potongan Kode Program Menu *Decomposition*

6) Melihat variabel baru

Untuk melihat variabel baru hasil dekomposisi pengguna dapat memilih halaman Data seperti yang ditampilkan pada Gambar 47.

File Edit Data Transform Analyze Graphs Help									
	t	var	var_SC	var_TC	var_IC	var_Foreca	var	var	var
1	1	125.53	0.787	94.727	1.683	74.593			
2	2	121.66	1.329	94.867	0.965	126.051			
3	3	186.19	0.866	95.006	2.264	82.249			
4	4	22.02	1.018	95.146	0.227	96.869			
5	5	11.45	0.787	95.285	0.153	75.032			
6	6	136.36	1.329	95.425	1.075	126.793			
7	7	1.19	0.866	95.564	0.014	82.732			
8	8	191.10	1.018	95.704	1.961	97.438			
9	9	4.22	0.787	95.844	0.056	75.472			
10	10	13.52	1.329	95.983	0.106	127.534			
11	11	11.39	0.866	96.123	0.137	83.215			
12	12	6.70	1.018	96.262	0.068	98.006			
13	13	214.24	0.787	96.402	2.822	75.911			
14	14	168.64	1.329	96.541	1.315	128.276			
15	15	205.84	0.866	96.681	2.459	83.699			
16	16	137.72	1.018	96.821	1.397	98.574			
17	17	153.51	0.787	96.960	2.011	76.351			
18	18	156.15	1.329	97.100	1.210	129.018			
19	19	4.63	0.866	97.239	0.055	84.182			

Gambar 47. Implementasi Melihat Variabel Baru Hasil Dekomposisi

7) Melihat hasil analisis *decomposition*

Untuk melihat tabel analisis dan grafik *decomposition* pengguna dapat memilih halaman *Result* seperti yang ditampilkan pada Gambar 48 dan 49.

File

Edit

Data

Transform

Analyze

Graphs

Help

Result

<

Log 1

>

Seasonal Indices

Seasonal Indices Years-Quarters

	value
period 1 of 4	0.787
period 2 of 4	1.329
period 3 of 4	0.866
period 4 of 4	1.018

Deskripsi

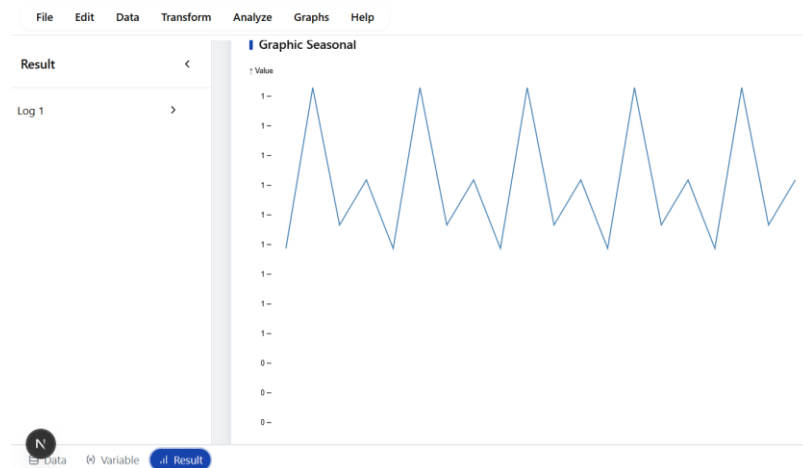
Tidak ada deskripsi.

Equation Trend

Linear Trend Equation

$y = 94.587 + 0.14t$

Gambar 48. Implementasi Tabel Analisis Menu *Decomposition*



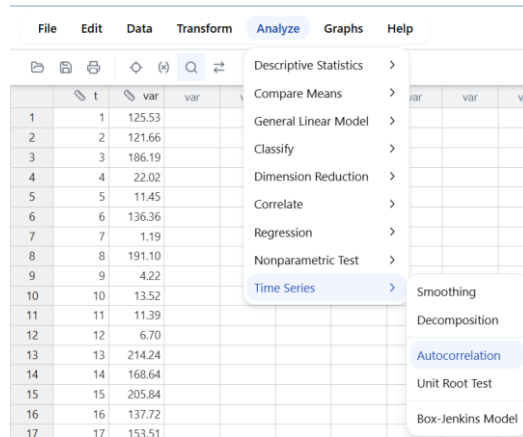
Gambar 49. Implementasi Grafik Komponen Seasonal Hasil *Decomposition*

3. Implementasi Menu *Autocorrelation*

Menu *autocorrelation* mengimplementasikan materi pembelajaran Analisis Runtun Waktu Sub-CPMK Pertemuan 4. Adapun untuk menggunakan menu *autocorrelation* pada aplikasi ini, pengguna dapat melakukan tahapan-tahapan sebagai berikut.

1) Mengakses menu *autocorrelation*

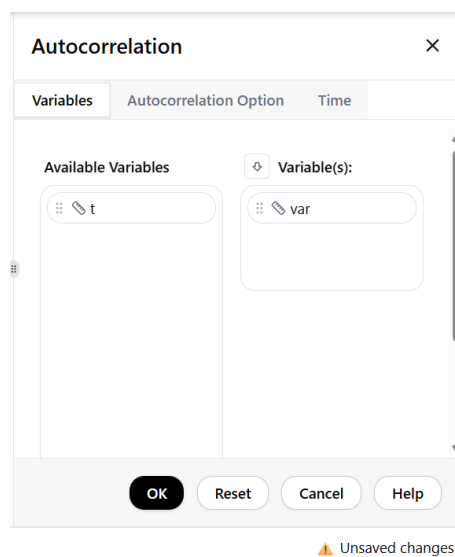
Untuk mengakses menu *autocorrelation* pengguna dapat memilih tab *analyze* pada *toolbar* lalu memilih menu *time series* yang selanjutnya akan terdapat pilihan untuk mengakses menu *autocorrelation* seperti yang ditampilkan pada Gambar 50.



Gambar 50. Implementasi Mengakses Menu *Autocorrelation*

2) Mengonfigurasi variabel

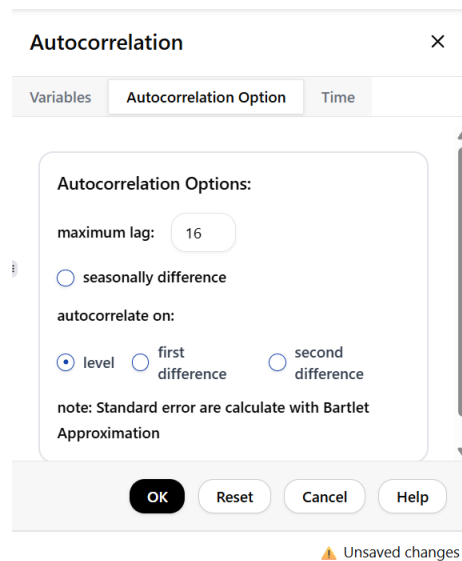
Pada modal menu *autocorrelation* pengguna dapat mengonfigurasi variabel pada tab *variable*. Pada tab tersebut pengguna dapat memilih satu variabel yang ingin digunakan dan menghapus variabel terpilih sesuai dengan kebutuhan. Berikut Gambar 51 menampilkan antarmuka tab *variable* menu *autocorrelation*.



Gambar 51. Implementasi Mengonfigurasi Variabel Menu *Autocorrelation*

3) Mengonfigurasi opsi menu *autocorrelation*

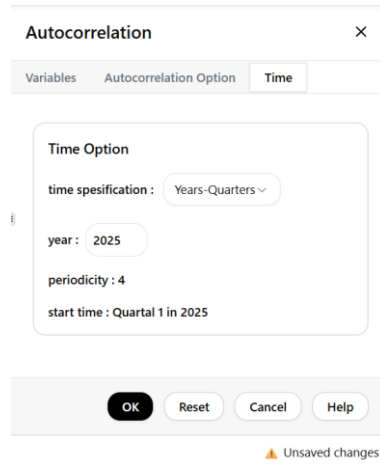
Untuk mengakses opsi menu *autocorrelation* pengguna dapat memilih tab *autocorrelation option*. Selanjutnya pengguna dapat mengonfigurasi opsi menu *autocorrelation* sesuai dengan kebutuhan seperti yang tampilkan pada Gambar 52.



Gambar 52. Implementasi Mengonfigurasi Opsi Menu *Autocorrelation*

4) Mengonfigurasi waktu

Untuk mengakses pengaturan waktu menu *autocorrelation* pengguna dapat memilih tab *time*. Selanjutnya pengguna dapat mengonfigurasi waktu menu *autocorrelation* sesuai dengan kebutuhan seperti yang tampilkan pada Gambar 53.



Gambar 53. Implementasi Mengonfigurasi Waktu Menu *Autocorrelation*

5) Menghitung autokorelasi

Untuk menghitung autokorelasi pengguna dapat menekan tombol *ok* pada modal menu *autocorrelation*. Selanjutnya aplikasi akan menghitung autokorelasi pada sisi klien menggunakan kode program hasil kompilasi WASM. Adapun potongan kode program penghitungan autokorelasi ditampilkan pada Gambar 54.

```

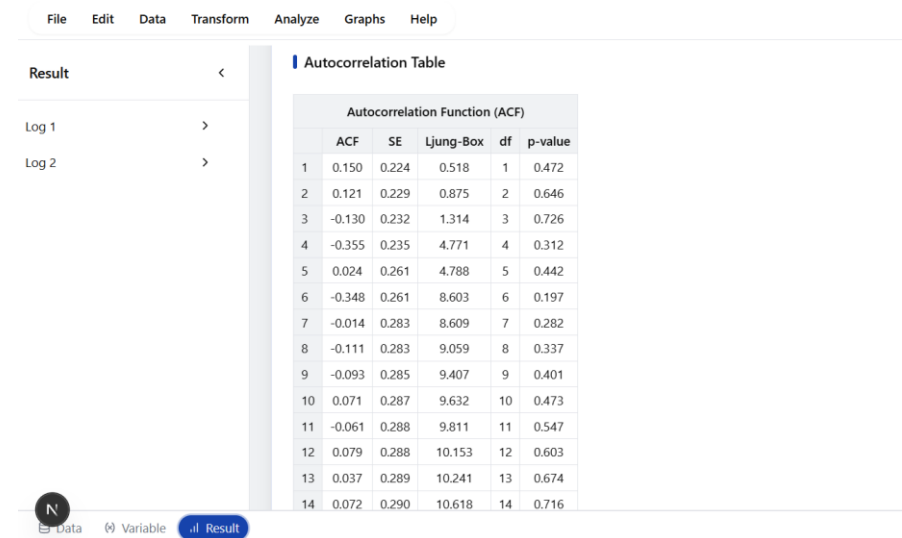
frontend > src > wasm > src > time_series > autocorrelation > @ autocorrelate.rs > ...
5  #[wasm_bindgen]
6  impl Autocorrelation{
7      pub fn autocorrelate(&mut self, difference: String, use_seasonal: bool, seasonally: i32){
8          let data = self.get_data();
9          let fix_diff: Vec<f64>;
10         if use_seasonal{
11             let season_diff = seasonal_difference(data, seasonally);
12             match difference.as_str(){
13                 "level" => {
14                     fix_diff = season_diff;
15                 },
16                 "first-difference" => {
17                     fix_diff = first_difference(season_diff);
18                 },
19                 "second-difference" => {
20                     fix_diff = second_difference(season_diff);
21                 },
22                 _ => {
23                     fix_diff = vec![];
24                 }
25             }
26         } else {
27             match difference.as_str(){
28                 "level" => {
29                     fix_diff = data;

```

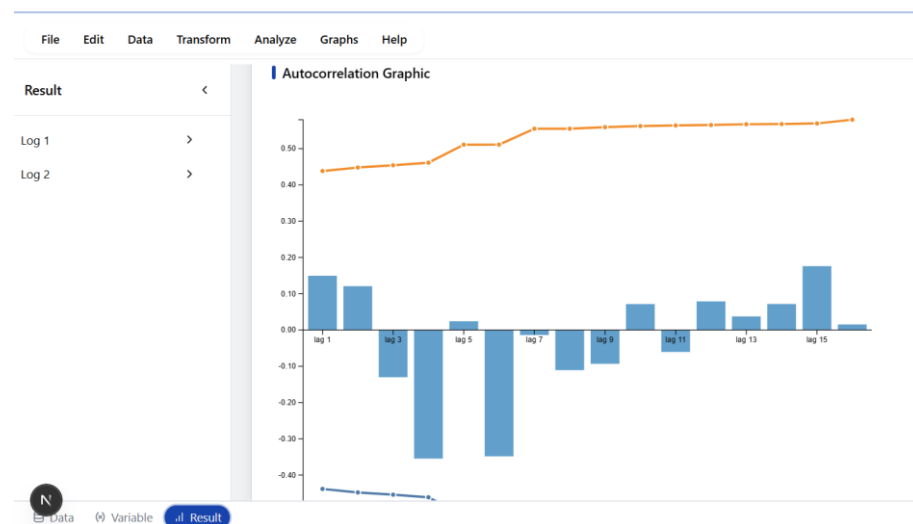
Gambar 54. Implementasi Potongan Kode Program Menu *Autocorrelation*

6) Melihat hasil analisis autokorelasi

Untuk melihat tabel analisis autokorelasi dan grafik koleogram pengguna dapat memilih halaman *Result* seperti yang ditampilkan pada Gambar 55 dan 56.



Gambar 55. Implementasi Tabel Analisis Menu *Autocorrelation*



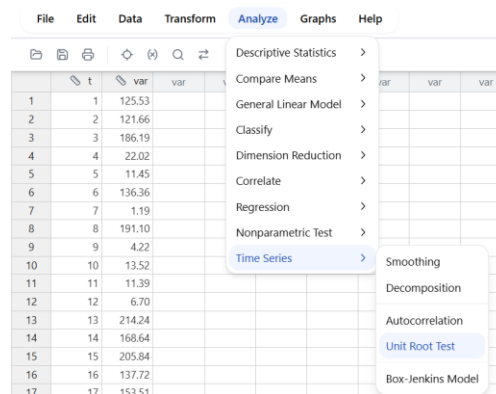
Gambar 56. Implementasi Grafik Koleogram Menu *Autocorrelation*

4. Implementasi Menu *Unit Root Test*

Menu *unit root test* mengimplementasikan materi pembelajaran Analisis Runtun Waktu Sub-CPMK Pertemuan 4. Adapun untuk menggunakan menu *unit root test* pada aplikasi ini, pengguna dapat melakukan tahapan-tahapan sebagai berikut.

1) Mengakses menu *unit root test*

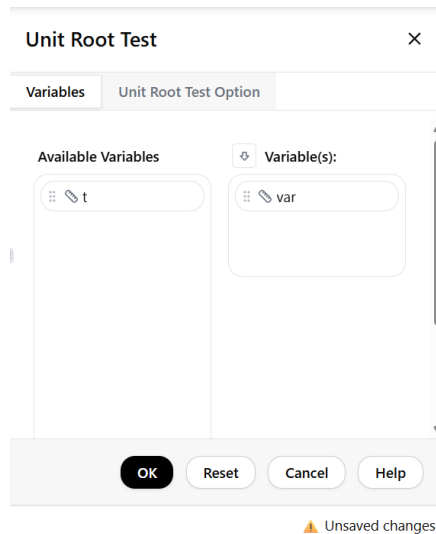
Untuk mengakses menu *unit root test* pengguna dapat memilih tab *analyze* pada *toolbar* lalu memilih menu *time series* yang selanjutnya akan terdapat pilihan untuk mengakses menu *unit root test* seperti yang ditampilkan pada Gambar 57.



Gambar 57. Implementasi Mengakses Menu *Unit Root Test*

2) Mengonfigurasi variabel

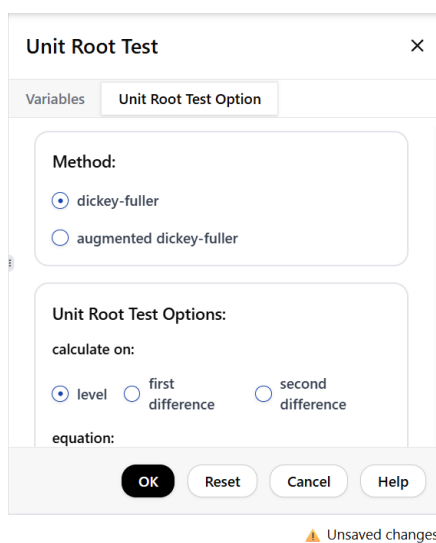
Pada modal menu *unit root test* pengguna dapat mengonfigurasi variabel pada tab *variable*. Pada tab tersebut pengguna dapat memilih satu variabel yang ingin digunakan dan menghapus variabel terpilih sesuai dengan kebutuhan. Berikut Gambar 58 menampilkan antarmuka tab *variable* menu *unit root test*.



Gambar 58. Implementasi Mengonfigurasi Variabel Menu *Unit Root Test*

3) Mengonfigurasi opsi menu *unit root test*

Untuk mengakses opsi menu *unit root test* pengguna dapat memilih tab *unit root test option*. Selanjutnya pengguna dapat mengonfigurasi opsi menu *unit root test* sesuai dengan kebutuhan seperti yang tampilkan pada Gambar 59.



Gambar 59. Implementasi Mengonfigurasi Opsi Menu *Unit Root Test*

4) Menghitung *unit root test*

Untuk menghitung *unit root test* pengguna dapat menekan tombol *ok* pada modal menu *unit root test*. Selanjutnya aplikasi akan menghitung *unit root test* pada sisi klien menggunakan kode program hasil kompilasi WASM. Adapun potongan kode program penghitungan *unit root test* ditampilkan pada Gambar 60.

```
frontend > src > wasm > src > time_series > unit_root_test > dickey_fuller > @ calculate_statistics > ...
7  impl DickeyFuller{
26      pub fn calculate_test_stat(&mut self) -> f64 {
27          // Initialize the variables
28          let mut t: Vec<f64> = Vec::new();
29          let mut x: Vec<f64> = Vec::new();
30          let mut y: Vec<f64> = Vec::new();
31          let mut data_prepare = self.get_data();
32
33          match self.get_level().as_str() {
34              "first-difference" => {
35                  let diff_data = first_difference(data_prepare.clone());
36                  data_prepare = diff_data;
37              },
38              "second-difference" => {
39                  let diff_data = second_difference(data_prepare.clone());
40                  data_prepare = diff_data;
41              },
42              _ => {}
43          }
44
45          let data = data_prepare;
46          let difference = first_difference(data.clone());
47          for i in 0..difference.len(){
48              t.push(i as f64 + 2.0);
49              x.push(data[i]);
```

Gambar 60. Implementasi Potongan Kode Program Menu *Unit Root Test*

5) Melihat hasil analisis *unit root test*

Untuk melihat tabel analisis *unit root test* pengguna dapat memilih halaman *Result* seperti yang ditampilkan pada Gambar 61.

File	Edit	Data	Transform	Analyze	Graphs	Help
------	------	------	-----------	---------	--------	------

Result	<
Log 1	>
Log 2	>
Log 3	>

Dickey-Fuller Test Statistic			
		t-statistic	p-value
Dickey-Fuller Test		-3.392	0.011
Critical Value	1%	-3.833	
	5%	-3.031	
	10%	-2.656	

Deskripsi

Tidak ada deskripsi.

Coefficient Regression Test				
	coefficient	standard error	t-statistic	p-value
Constant	76.088	31.311	2.430	0.026

N	Data	Variable	Result
---	------	----------	--------

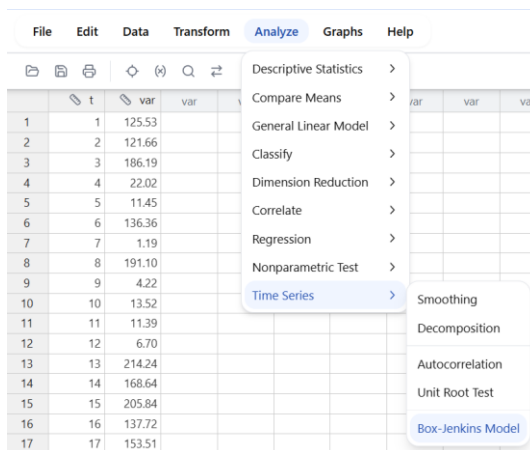
Gambar 61. Implementasi Tabel Analisis Menu *Unit Root Test*

5. Implementasi Menu *Box-Jenkins Model*

Menu *box-jenkins model* mengimplementasikan materi pembelajaran Analisis Runtun Waktu Sub-CPMK Pertemuan 5 dan 6. Adapun untuk menggunakan menu *box-jenkins model* pada aplikasi ini, pengguna dapat melakukan tahapan-tahapan sebagai berikut.

1) Mengakses menu *box-jenkins model*

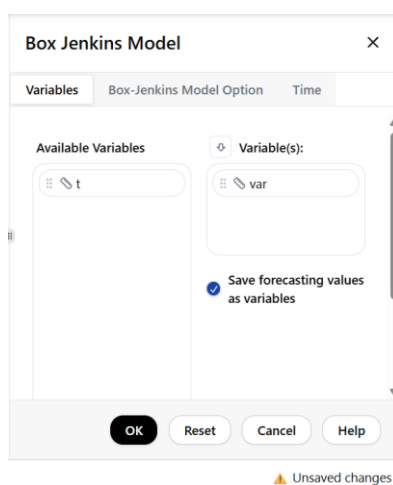
Untuk mengakses menu *box-jenkins model* pengguna dapat memilih tab *analyze* pada *toolbar* lalu memilih menu *time series* yang selanjutnya akan terdapat pilihan untuk mengakses menu *box-jenkins model* seperti yang ditampilkan pada Gambar 62.



Gambar 62. Implementasi Mengakses Menu *Box-Jenkins Model*

2) Mengonfigurasi variabel

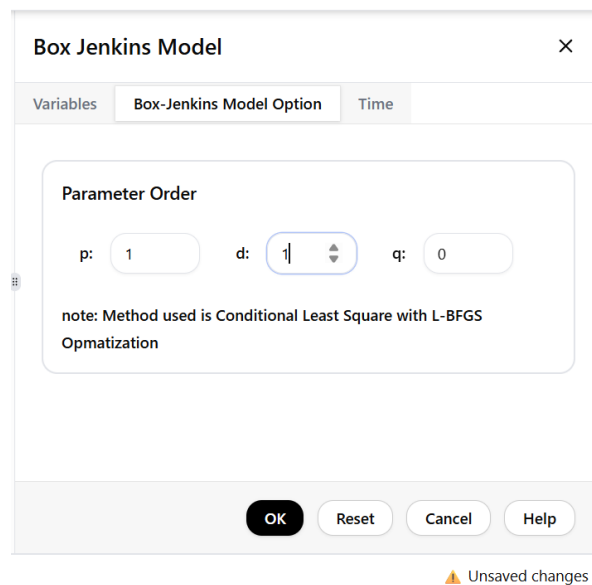
Pada modal menu *box-jenkins model* pengguna dapat mengonfigurasi variabel pada tab *variable*. Pada tab tersebut pengguna dapat memilih satu variabel yang ingin digunakan, menghapus variabel terpilih, hingga menyimpan hasil *box-jenkins model* sebagai variabel atau tidak pada *checkbox* yang tersedia. Berikut Gambar 63 menampilkan antarmuka tab *variable* menu *box-jenkins model*.



Gambar 63. Implementasi Mengonfigurasi Variabel Menu *Box-Jenkins Model*

3) Mengonfigurasi opsi menu *box-jenkins model*

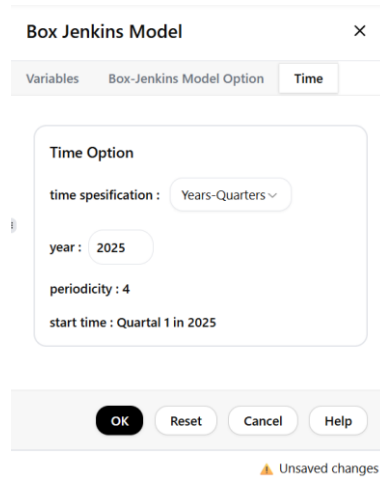
Untuk mengakses opsi menu *box-jenkins model* pengguna dapat memilih tab *box-jenkins model option*. Selanjutnya pengguna dapat mengonfigurasi parameter menu *box-jenkins model* sesuai dengan kebutuhan seperti yang tampilkan pada Gambar 64.



Gambar 64. Implementasi Mengonfigurasi Opsi Menu *Box-Jenkins Model*

4) Mengonfigurasi waktu

Untuk mengakses pengaturan waktu menu *box-jenkins model* pengguna dapat memilih tab *time*. Selanjutnya pengguna dapat mengonfigurasi waktu menu *box-jenkins model* sesuai dengan kebutuhan seperti yang tampilkan pada Gambar 65.



Gambar 65. Implementasi Mengonfigurasi Waktu Menu *Box-Jenkins Model*

5) Menghitung model *box-jenkins*

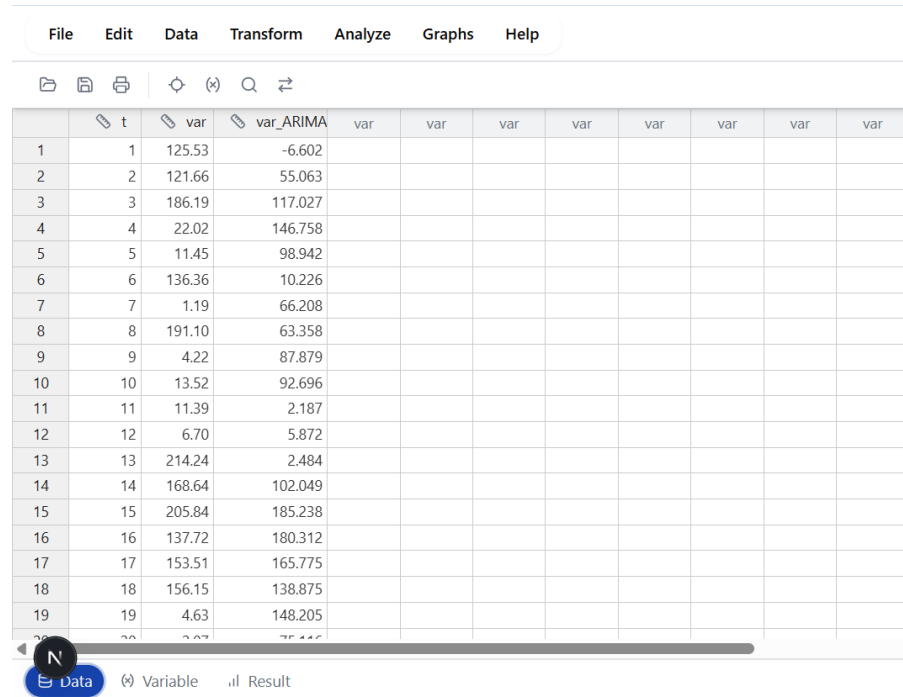
Untuk menghitung model *box-jenkins* pengguna dapat menekan tombol *ok* pada modal menu *box-jenkins model*. Selanjutnya aplikasi akan menghitung *box-jenkins model* pada sisi klien menggunakan kode program hasil kompilasi WASM. Adapun potongan kode program penghitungan model *box-jenkins* ditampilkan pada Gambar 66.

```
frontend > src > wasm > src > time_series > arima > @ arima_est_res.rs > ...
4  #[wasm_bindgen]
5  impl Arima {
6      pub fn est_res(&self, intercept: f64, ar: Vec<f64>, ma: Vec<f64>, data: Vec<f64>) -> Vec<f64> {
7          let mut residuals = Vec::new();
8          let p = self.get_ar_order() as usize;
9          let q = self.get_ma_order() as usize;
10         if p > 0 && q > 0 {
11             for i in 0..data.len() {
12                 let mut sum_ar = 0.0;
13                 for j in 1..=p as usize {
14                     if i + 1 > j {
15                         sum_ar += ar[j - 1] * (data[i - j] - intercept);
16                     }
17                 }
18                 let mut sum_ma = 0.0;
19                 for j in 1..=q as usize {
20                     if i + 1 > j {
21                         sum_ma += ma[j - 1] * residuals[i - j];
22                     }
23                 }
24                 residuals.push(data[i] - intercept - sum_ar + sum_ma);
25             }
26         } else if p > 0 {
27             for i in 0..data.len() {
28                 let mut sum_ar = 0.0;
```

Gambar 66. Implementasi Potongan Kode Program Menu *Box-Jenkins Model*

6) Melihat variabel baru

Untuk melihat variabel baru hasil *forecasting* model *box-jenkins* pengguna dapat memilih halaman Data seperti yang ditampilkan pada Gambar 67.

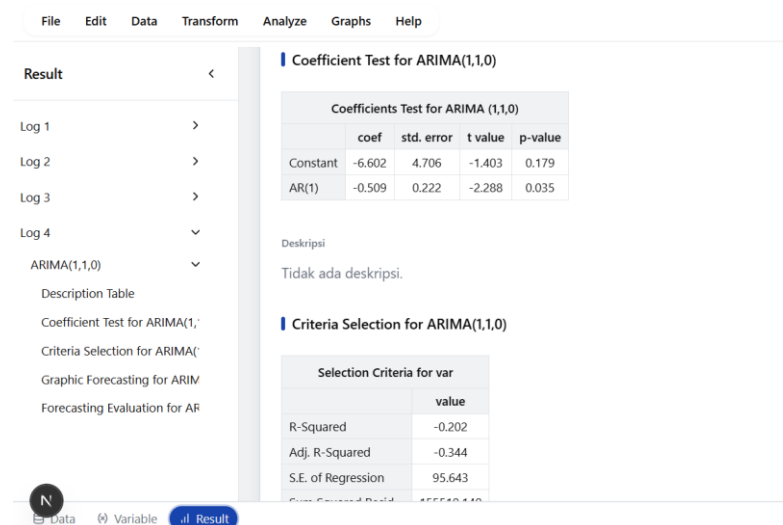


	t	var	var_ARIMA	var	var	var	var	var	var	var	var
1	1	125.53	-6.602								
2	2	121.66	55.063								
3	3	186.19	117.027								
4	4	22.02	146.758								
5	5	11.45	98.942								
6	6	136.36	10.226								
7	7	1.19	66.208								
8	8	191.10	63.358								
9	9	4.22	87.879								
10	10	13.52	92.696								
11	11	11.39	2.187								
12	12	6.70	5.872								
13	13	214.24	2.484								
14	14	168.64	102.049								
15	15	205.84	185.238								
16	16	137.72	180.312								
17	17	153.51	165.775								
18	18	156.15	138.875								
19	19	4.63	148.205								

Gambar 67. Implementasi Melihat Variabel Baru Menu *Box-Jenkins Model*

7) Melihat hasil analisis *box-jenkins model*

Untuk melihat tabel analisis dan grafik *forecasting box-jenkins model* pengguna dapat memilih halaman *Result* seperti yang ditampilkan pada Gambar 68 dan 69.



Gambar 68. Implementasi Tabel Analisis *Box-Jenkins Model*



Gambar 69. Implementasi Grafik *Forecasting Box-Jenkins Model*

5.2 Evaluasi

5.2.1 *White-Box Testing*

Berdasarkan pembahasan di bab sebelumnya, pada tahapan evaluasi akan dilakukan pengujian *white-box testing* menggunakan teknik *basis path* pada level *integration testing* melalui tiga tahapan utama, yaitu membentuk *flow graph*, menghitung *cyclomatic complexity*, dan membuat *test cases* berdasarkan hasil dua

tahapan sebelumnya. Setelah melakukan tahapan-tahapan tersebut peneliti akan melakukan perbandingan *output* statistik Statify Modul Analisis *Time Series* dengan *output* statistik aplikasi Eviews untuk memvalidasi *output* yang dihasilkan. Aplikasi Eviews dijadikan dasar perbandingan karena aplikasi ini merupakan aplikasi yang sering digunakan dalam proses pembelajaran Mata Kuliah Analisis Runtun Waktu di Politeknik Statistika STIS. Berikut rincian proses *white-box testing* pada masing-masing menu Statify Modul Analisis *Time Series*.

1. Menu *Smoothing*

Berikut tahapan *white-box testing* menu *smoothing*.

1) *Flow Graph Notation*

Pada tahapan ini, dilakukan pendaftaran simpul untuk membentuk dasar-dasar *flow graph notation* menu *smoothing*. Peneliti mengklasifikasi setiap bagian kode program menu *smoothing* menjadi kumpulan *statement* sederhana lalu diubah menjadi kumpulan notasi simpul. Selain itu, pada proses ini terdapat beberapa penyingkatan nama metode untuk mempermudah penulisan nama simpul, antara lain *simple moving average* (sma), *double moving average* (dma), *simple exponential smoothing* (ses), *double exponential smoothing* (des), *holt's method exponential smoothing* (holt), dan *winter's method exponential smoothing* (winter). Berikut kumpulan *statement* menu *smoothing* yang dijadikan kumpulan notasi simpul yang ditampilkan pada Tabel 5.

Tabel 5. Daftar Simpul Menu *Smoothing*

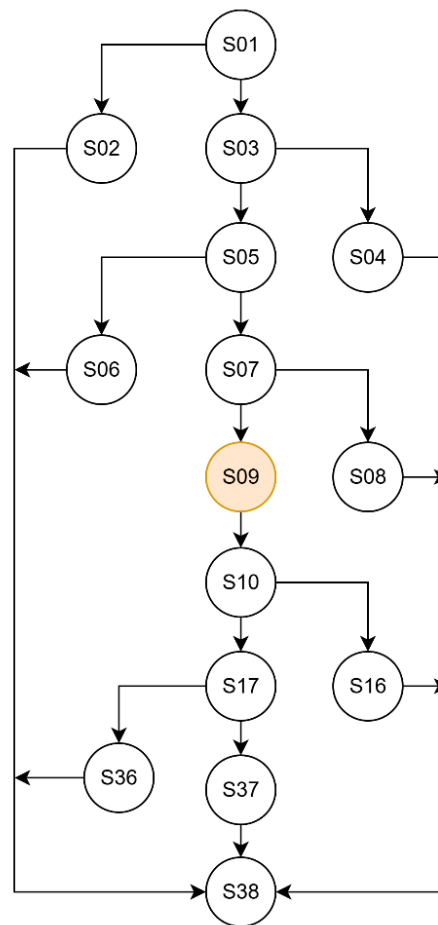
Kode Simpul	Nama Simpul
(1)	(2)
S01	Menjalankan menu <i>smoothing</i>

S02	Variabel terpilih kosong
S03	Variabel terpilih berisi
S04	Variabel terpilih bukan numerik
S05	Variabel terpilih numerik
S06	Data kosong
S07	Data berisi
S08	Observasi data kurang dari 20
S09	Observasi data lebih dari sama dengan 20
S10	Metode sma
S11	Metode dma
S12	Metode ses
S13	Metode des
S14	Metode holt
S15	Metode winter
S16	Parameter metode sma di luar batas
S17	Parameter metode sma di dalam batas
S18	Parameter metode dma di luar batas
S19	Parameter metode dma di dalam batas
S20	Observasi data kurang dari kelipatan tiga parameter metode dma
S21	Observasi data lebih dari sama dengan kelipatan tiga parameter metode dma
S22	Parameter alfa metode ses di luar batas
S23	Parameter alfa metode ses di dalam batas
S24	Parameter alfa metode des di luar batas
S25	Parameter alfa metode des di dalam batas
S26	Parameter alfa metode holt di luar batas
S27	Parameter alfa metode holt di dalam batas
S28	Parameter beta metode holt di luar batas
S29	Parameter beta metode holt di dalam batas
S30	Parameter alfa metode winter di luar batas
S31	Parameter alfa metode winter di dalam batas
S32	Parameter beta metode winter di luar batas
S33	Parameter beta metode winter di dalam batas
S34	Parameter gama metode winter di luar batas
S35	Parameter gama metode winter di dalam batas
S36	Belum mengatur konfigurasi waktu
S37	Sudah mengatur konfigurasi waktu
S38	Hasil

Sumber: Dokumen Peneliti

Setelah mendaftarkan nama-nama simpul beserta kodenya, peneliti melanjutkan ke tahapan pembentukan *flow graph*. *Flow graph* pada menu

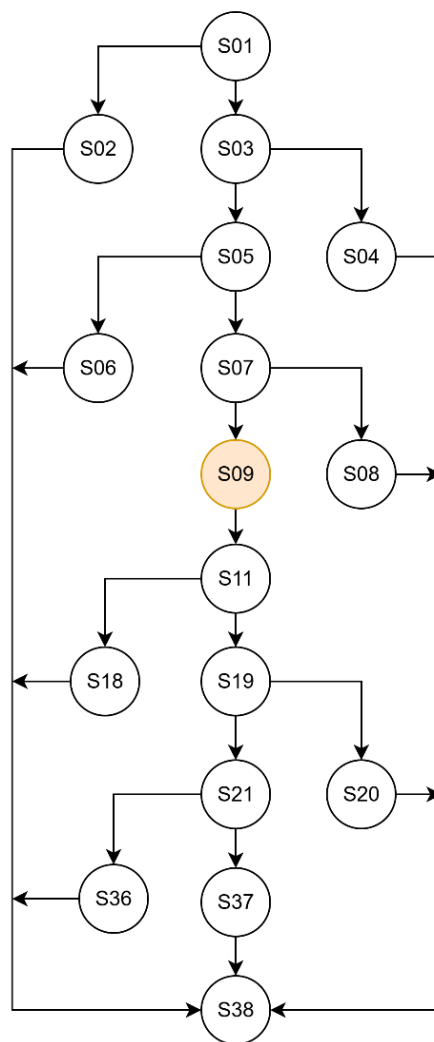
ini dibentuk menjadi enam bagian karena simpul S09 memiliki enam cabang jalur yang perlu disederhanakan untuk mempermudah pembacaan *flow graph* di tahapan selanjutnya. Berikut kumpulan *flow graph* menu *smoothing* yang ditampilkan pada Gambar 70 sampai 75.



Keterangan :

 Simpul memiliki 6 cabang dan dipisah menjadi 6 flow chart

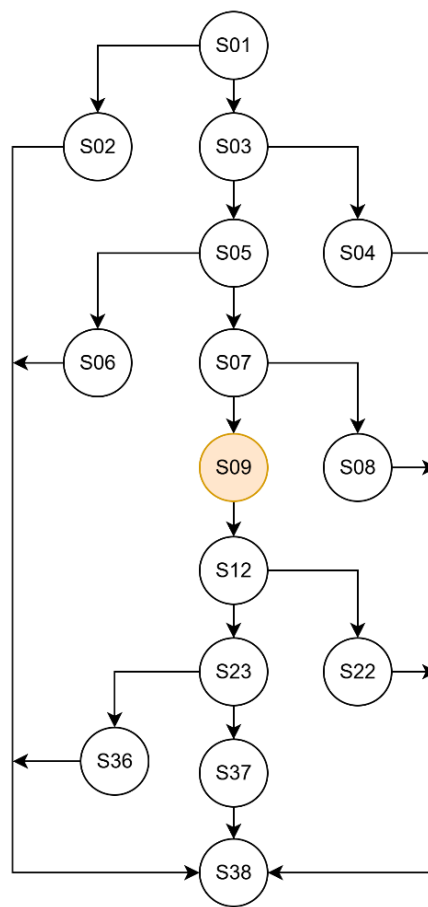
Gambar 70. *Flow Graph* Menu *Smoothing* Metode *Simple Moving Average*



Keterangan :

● Simpul memiliki 6 cabang dan
dipisah menjadi 6 flow chart

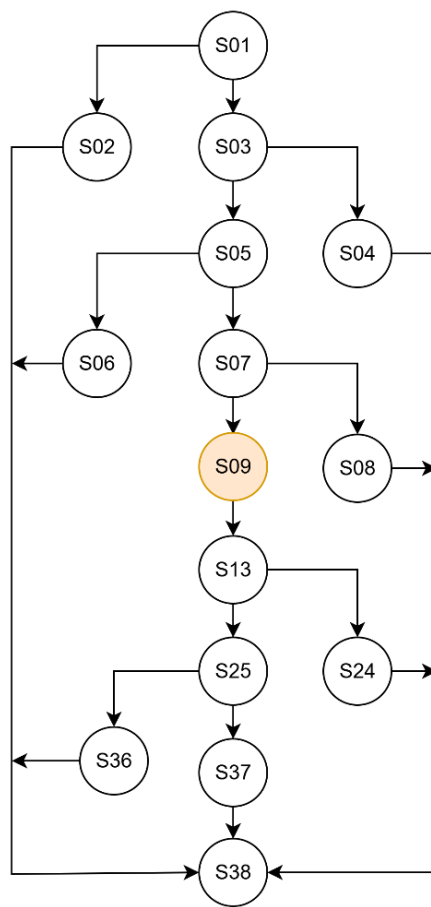
Gambar 71. Flow Graph Menu Smoothing Metode Double Moving Average



Keterangan :

○ Simpul memiliki 6 cabang dan dipisah menjadi 6 flow chart

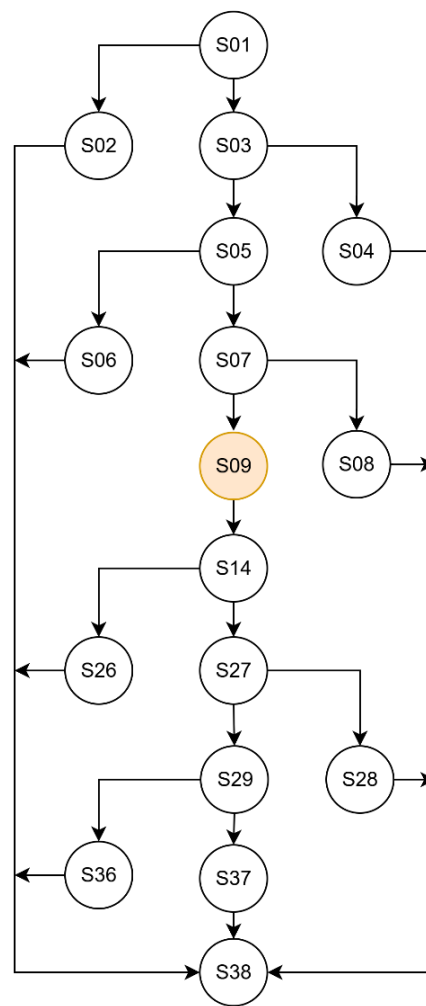
Gambar 72. *Flow Graph Menu Smoothing Metode Simple Exponential Smoothing*



Keterangan :

● Simpul memiliki 6 cabang dan dipisah menjadi 6 flow chart

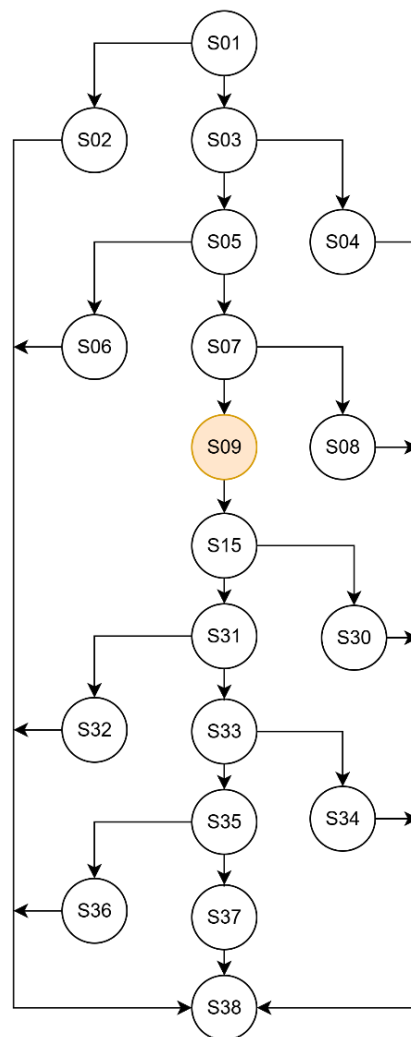
Gambar 73. *Flow Graph Menu Smoothing Metode Double Exponential Smoothing*



Keterangan :

 Simpul memiliki 6 cabang dan dipisah menjadi 6 flow chart

Gambar 74. *Flow Graph Menu Smoothing Metode Holt's Method Exponential Smoothing*



Keterangan :

● Simpul memiliki 6 cabang dan dipisah menjadi 6 flow chart

Gambar 75. *Flow Graph Menu Smoothing Metode Winter's Method Exponential Smoothing*

2) *Cyclomatic Complexity*

Pada tahapan ini, peneliti menghitung nilai *cyclomatic complexity* yang dinotasikan sebagai $V(G)$. Nilai tersebut dapat dihitung menggunakan rumus $V(G) = E - N + 2$. Notasi E melambangkan jumlah garis (*edge*) yang terdapat pada *flow graph* dan notasi N melambangkan jumlah simpul (*node*) yang terdapat pada *flow graph*. Pada menu

smoothing jumlah garis dan simpul yang diperoleh pada *flow graph* masing-masing berjumlah 62 dan 38 sehingga diperoleh nilai *cyclomatic complexity* sebesar 26. Nilai tersebut memiliki arti bahwa jumlah jalur yang terdapat pada kode program menu *smoothing* berjumlah dua puluh enam jalur yang selanjutnya akan dijadikan sebagai dasar jumlah jalur untuk tahapan *derriving test cases*.

3) *Derriving Test Cases*

Berdasarkan kedua tahapan sebelumnya, jumlah *test cases* yang dibentuk oleh peneliti pada menu *smoothing* berjumlah dua puluh enam *test cases*. Selanjutnya, setiap *test cases* akan diuji dengan membandingkan hasil ekspektasi dan hasil aktual untuk menentukan keberhasilan suatu *test case*. Berikut Tabel 6 menampilkan hasil pengujian seluruh *test cases* menu *smoothing*.

Tabel 6. Hasil *Test Cases* Menu *Smoothing*

Kode <i>Test Case</i>	Jalur <i>Test Case</i>	Hasil Eskpektasi	Hasil Aktual	Hasil Pengujian
(1)	(2)	(3)	(4)	(5)
TCS01	S01-S02-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS02	S01-S03-S04-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS03	S01-S03-S05-S06-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS04	S01-S03-S05-S07-S08-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS05	S01-S03-S05-S07-S09-S10-S16-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil

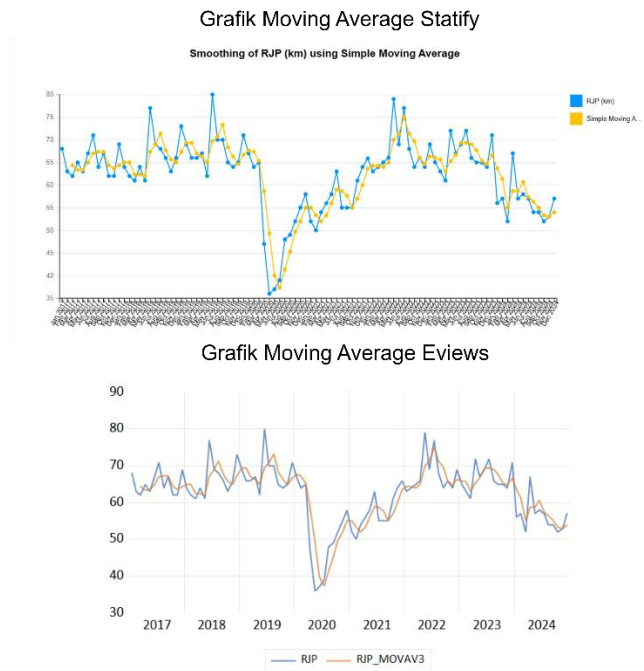
TCS06	S01-S03-S05-S07-S09-S10-S17-S36-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS07	S01-S03-S05-S07-S09-S10-S17-S37-S38	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil
TCS08	S01-S03-S05-S07-S09-S11-S18-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS09	S01-S03-S05-S07-S09-S11-S19-S20-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS10	S01-S03-S05-S07-S09-S11-S19-S21-S36-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS11	S01-S03-S05-S07-S09-S11-S19-S21-S37-S38	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil
TCS12	S01-S03-S05-S07-S09-S12-S22-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS13	S01-S03-S05-S07-S09-S12-S23-S36-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS14	S01-S03-S05-S07-S09-S12-S23-S37-S38	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil
TCS15	S01-S03-S05-S07-S09-S13-S24-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS16	S01-S03-S05-S07-S09-S13-S25-S36-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS17	S01-S03-S05-S07-S09-S13-S25-S37-S38	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil
TCS18	S01-S03-S05-S07-S09-S14-S26-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS19	S01-S03-S05-S07-S09-S14-S27-S28-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS20	S01-S03-S05-S07-S09-S14-S27-S29-S36-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil

TCS21	S01-S03-S05-S07-S09-S14-S27-S29-S37-S38	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil
TCS22	S01-S03-S05-S07-S09-S15-S30-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS23	S01-S03-S05-S07-S09-S15-S31-S32-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS24	S01-S03-S05-S07-S09-S15-S31-S33-S34-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS25	S01-S03-S05-S07-S09-S15-S31-S33-S35-S36-S38	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCS26	S01-S03-S05-S07-S09-S15-S31-S33-S35-S37-S38	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil

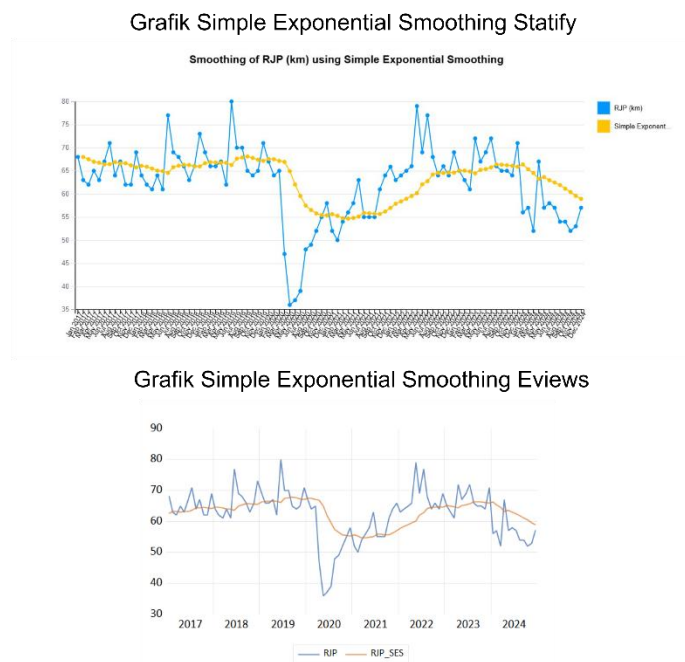
Sumber: Dokumen Peneliti

4) Perbandingan *Output* Statistik

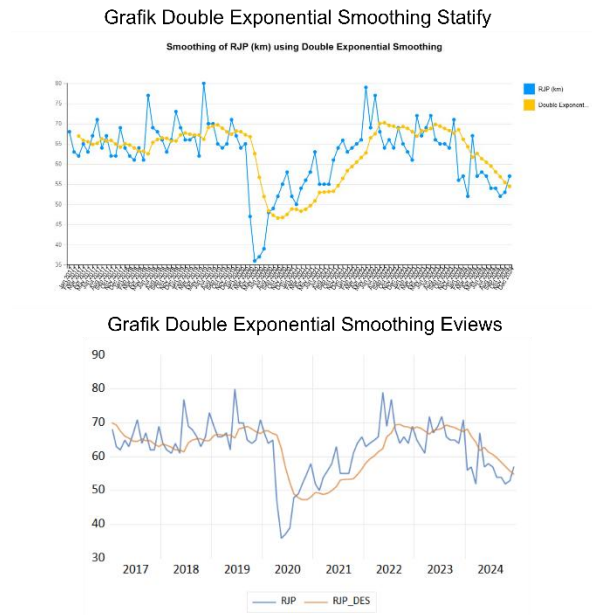
Pada tahapan ini, peneliti membandingkan *output* statistik antara Statify dan Eviews. *Case* yang digunakan peneliti pada tahapan ini adalah metode-metode *smoothing* yang sama yang tersedia pada masing-masing aplikasi, yaitu metode *simple moving average*, *simple exponential smoothing*, *double exponential smoothing*, *holt's method exponential smoothing*, dan *winter's method exponential smoothing*. Selain itu, data yang digunakan pada tahapan ini adalah data penumpang kereta api bulanan tahun 2017 hingga 2024 yang diperoleh dari *website* resmi BPS. Adapun variabel yang digunakan adalah Rerata Jarak Perjalanan (RJP). Berikut ditampilkan perbandingan *output* dari setiap metode *smoothing* aplikasi Statify dan Eviews pada Gambar 76 sampai 80.



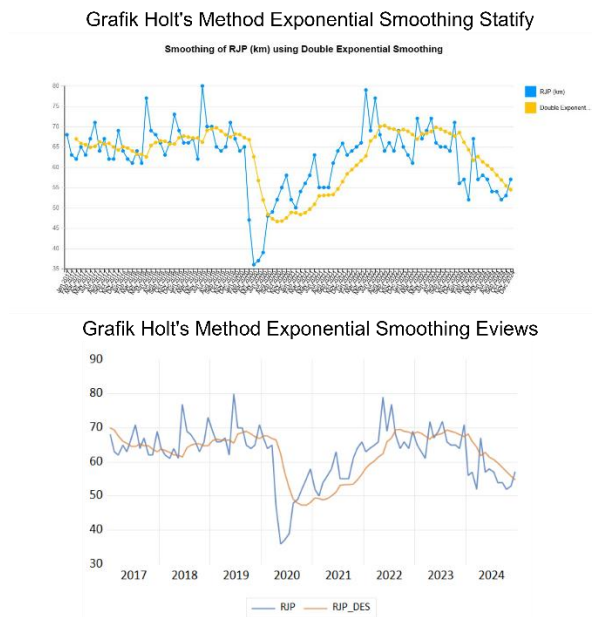
Gambar 76. Grafik *Moving Average* dengan Periode 3



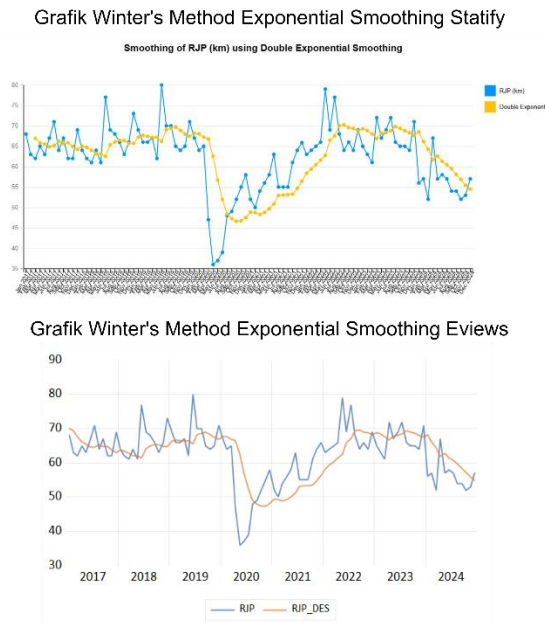
Gambar 77. Grafik *Simple Exponential Smoothing* dengan Alpa 0.1



Gambar 78. Grafik *Double Exponential Smoothing* dengan Alpa 0.1



Gambar 79. Grafik *Holt's Method Exponential Smoothing* dengan Alpa 0.1 dan Beta 0.1



Gambar 80. Grafik *Winter's Method Exponential Smoothing* dengan Alpa 0.1, Beta 0.1, dan Gama 0.1.

Berdasarkan *output* grafik pada Gambar 76 sampai 80 terlihat bahwa terdapat sedikit perbedaan hasil pada metode *exponential smoothing*. Hal ini dikarenakan perbedaan metode inisiasi nilai awal, yakni Statify menggunakan dasar inisiasi nilai awal yang tertulis pada buku *Business Forecasting* (Hanke & Reitsch, 1995).

2. Menu *Decomposition*

Berikut tahapan *white-box testing* menu *decomposition*.

1) *Flow Graph Notation*

Pada tahapan ini, dilakukan pendaftaran simpul untuk membentuk dasar-dasar *flow graph notation* menu *decomposition*. Peneliti mengklasifikasi setiap bagian kode program menu *decomposition* menjadi kumpulan *statement* sederhana lalu diubah menjadi kumpulan notasi

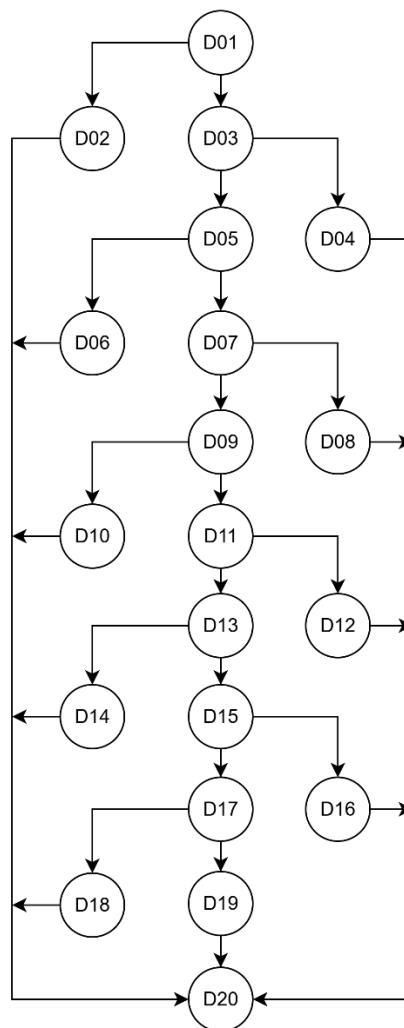
simpul. Berikut kumpulan *statement* menu *decomposition* yang dijadikan kumpulan notasi simpul yang ditampilkan pada Tabel 7.

Tabel 7. Daftar Simpul Menu *Decomposition*

Kode Simpul	Nama Simpul
(1)	(2)
D01	Menjalankan menu <i>decomposition</i>
D02	Variabel terpilih kosong
D03	Variabel terpilih satu
D04	Variabel terpilih bukan numerik
D05	Variabel terpilih numerik
D06	Data kosong
D07	Data berisi
D08	Belum mengatur konfigurasi waktu
D09	Sudah mengatur konfigurasi waktu
D10	Spesifikasi waktu tidak memiliki panjang periode
D11	Spesifikasi waktu memiliki panjang periode
D12	Observasi data kurang dari kelipatan 4 panjang periode waktu
D13	Observasi data lebih dari sama dengan kelipatan 4 panjang periode waktu
D14	Observasi data tidak sama dengan kelipatan panjang periode waktu
D15	Observasi data sama dengan kelipatan panjang periode waktu
D16	Metode <i>Additive</i>
D17	Metode <i>Multiplicative</i>
D18	Tren linear
D19	Tren eksponensial
D20	Hasil

Sumber: Dokumen Peneliti

Setelah mendaftarkan nama-nama simpul beserta kodenya, peneliti melanjutkan ke tahapan pembentukan *flow graph*. Berikut *flow graph* menu *decomposition* yang ditampilkan pada Gambar 81.



Gambar 81. *Flow Graph Menu Decomposition*

2) *Cyclomatic Complexity*

Pada tahapan ini, peneliti menghitung nilai *cyclomatic complexity* yang dinotasikan sebagai $V(G)$. Nilai tersebut dapat dihitung menggunakan rumus $V(G) = E - N + 2$. Notasi E melambangkan jumlah garis (*edge*) yang terdapat pada *flow graph* dan notasi N melambangkan jumlah simpul (*node*) yang terdapat pada *flow graph*. Pada menu *decomposition* jumlah garis dan simpul yang diperoleh pada *flow graph* masing-masing berjumlah 28 dan 20 sehingga diperoleh nilai *cyclomatic*

complexity sebesar 10. Nilai tersebut memiliki arti bahwa jumlah jalur yang terdapat pada kode program menu *decomposition* berjumlah sepuluh jalur yang selanjutnya akan dijadikan sebagai dasar jumlah jalur untuk tahapan *derriving test cases*.

3) *Derriving Test Cases*

Berdasarkan kedua tahapan sebelumnya, jumlah *test cases* yang dibentuk oleh peneliti pada menu *decomposition* berjumlah sepuluh *test cases*. Selanjutnya, setiap *test cases* akan diuji dengan membandingkan hasil ekspektasi dan hasil aktual untuk menentukan keberhasilan suatu *test case*. Berikut Tabel 8 menampilkan hasil pengujian seluruh *test cases* menu *decomposition*.

Tabel 8. Hasil *Test Cases* Menu *Decomposition*

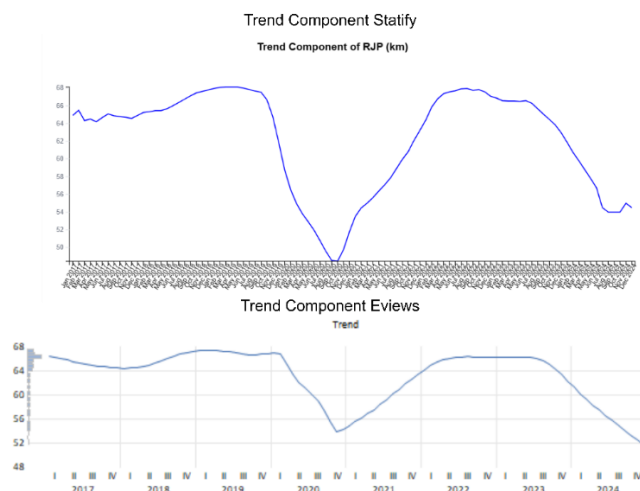
Kode Test Case	Jalur Test Case	Hasil Eskpektasi	Hasil Aktual	Hasil Pengujian
(1)	(2)	(3)	(4)	(5)
TCD01	D01-D02-D20	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCD02	D01-D03-D04-D20	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCD03	D01-D03-D05-D06-D20	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCD04	D01-D03-D05-D07-D08-D20	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCD05	D01-D03-D05-D07-D09-D10-D20	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCD06	D01-D03-D05-D07-D09-D11-D12-D20	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCD07	D01-D03-D05-D07-D09-D11-D13-D14-D20	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil

TCD08	D01-D03-D05-D07-D09-D11-D13-D15-D16-D20	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil
TCD09	D01-D03-D05-D07-D09-D11-D13-D15-D17-D18-D20	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil
TCD10	D01-D03-D05-D07-D09-D11-D13-D15-D17-D19-D20	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil

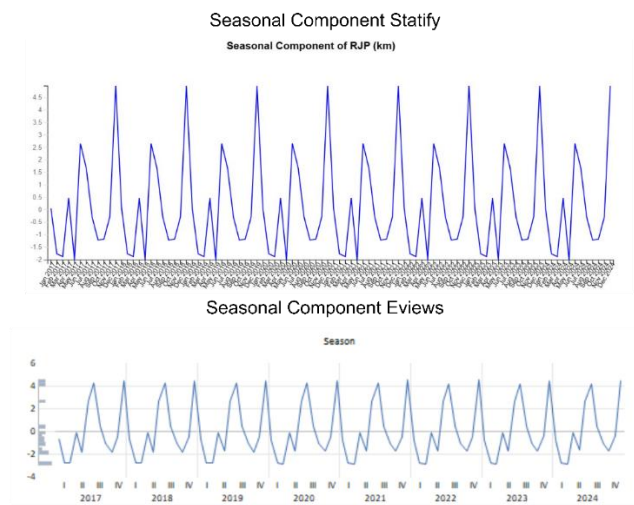
Sumber: Dokumen Peneliti

4) Perbandingan *Output* Statistik

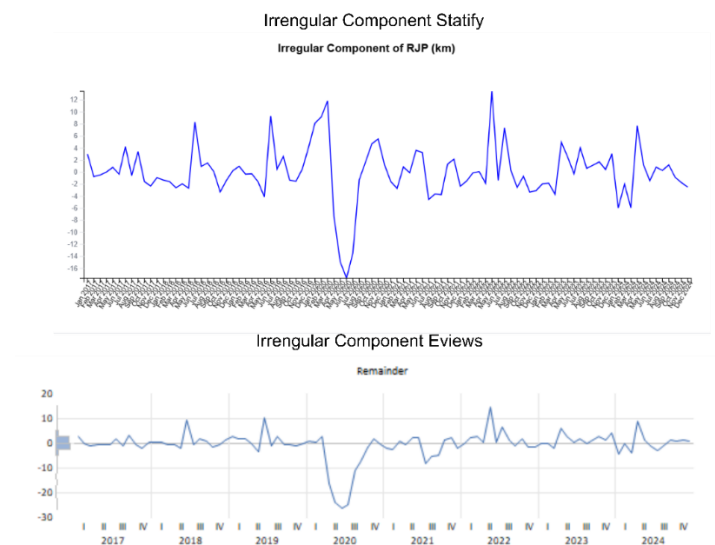
Pada tahapan ini, peneliti membandingkan *output* statistik antara Statify dan Eviews. *Case* yang digunakan peneliti pada tahapan ini adalah metode *decomposition* yang sama yang tersedia pada masing-masing aplikasi, yaitu metode *additive decomposition*. Selain itu, data yang digunakan pada tahapan ini adalah data penumpang kereta api bulanan tahun 2017 hingga 2024 yang diperoleh dari *website* resmi BPS. Adapun variabel yang digunakan adalah Rerata Jarak Perjalanan (RJP). Berikut ditampilkan perbandingan *output* dari setiap komponen data hasil dekomposisi aplikasi Statify dan Eviews pada Gambar 82 sampai 84.



Gambar 82. Grafik *Trend Component*



Gambar 83. Grafik *Seasonal Component*



Gambar 84. Grafik *Irregular Component*

Berdasarkan *output* grafik pada Gambar 82 sampai 84 terlihat bahwa terdapat sedikit perbedaan hasil pada setiap komponen hasil dekomposisi. Hal ini dikarenakan perbedaan dasar penghitungan dekomposisi, yakni

Statify menggunakan metode penghitungan dekomposisi klasik yang tertulis pada buku *Forecasting: Methods and Applications* (Makridakis et al., 1997).

3. Menu *Autocorrelation*

Berikut tahapan *white-box testing* menu *autocorrelation*.

1) *Flow Graph Notation*

Pada tahapan ini, dilakukan pendaftaran simpul untuk membentuk dasar-dasar *flow graph notation* menu *autocorrelation*. Peneliti mengklasifikasi setiap bagian kode program menu *autocorrelation* menjadi kumpulan *statement* sederhana lalu diubah menjadi kumpulan notasi simpul. Berikut kumpulan *statement* menu *autocorrelation* yang dijadikan kumpulan notasi simpul yang ditampilkan pada Tabel 9.

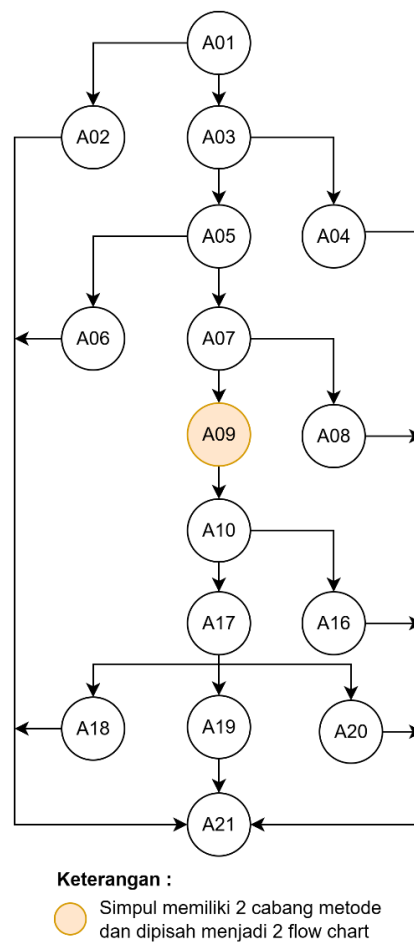
Tabel 9. Daftar Simpul Menu *Autocorrelation*

Kode Simpul	Nama Simpul
(1)	(2)
A01	Menjalankan menu <i>autocorrelation</i>
A02	Variabel terpilih kosong
A03	Variabel terpilih berisi
A04	Variabel terpilih bukan numerik
A05	Variabel terpilih numerik
A06	Data kosong
A07	Data berisi
A08	Observasi data kurang dari 20
A09	Observasi data lebih dari sama dengan 20
A10	Tidak memiliki pengaruh musiman
A11	Memiliki pengaruh musiman
A12	Belum mengatur konfigurasi waktu
A13	Sudah mengatur konfigurasi waktu
A14	Spesifikasi waktu tidak memiliki panjang periode
A15	Spesifikasi waktu memiliki panjang periode
A16	Jumlah lag maksimum di luar batas
A17	Jumlah lag maksimum di dalam batas
A18	Tanpa diferensiasi

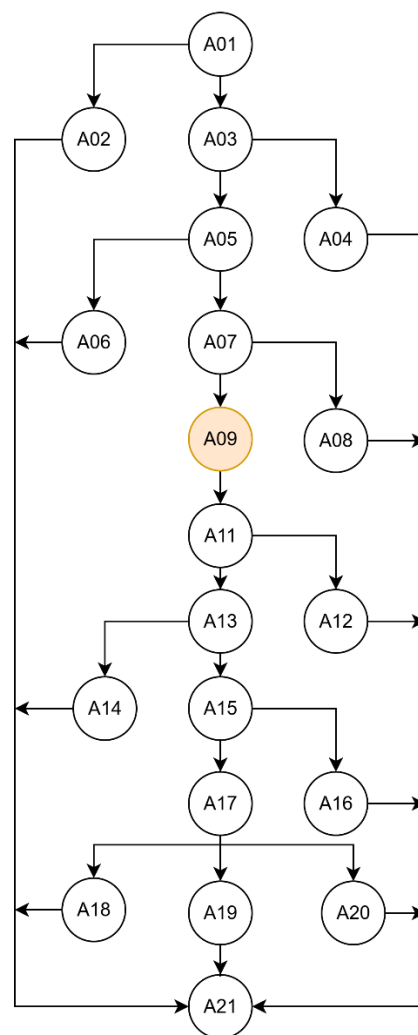
A19	Diferensiasi pertama
A20	Diferensiasi kedua
A21	Hasil

Sumber: Dokumen Peneliti

Setelah mendaftarkan nama-nama simpul beserta kodenya, peneliti melanjutkan ke tahapan pembentukan *flow graph*. *Flow graph* pada menu ini dibentuk menjadi dua bagian karena terdapat dua metode yang perlu disederhanakan menjadi dua cabang jalur untuk mempermudah pembacaan *flow graph* di tahapan selanjutnya. Berikut *flow graph* menu *autocorrelation* yang ditampilkan pada Gambar 85 dan 86.



Gambar 85. *Flow Graph* Menu *Autocorrelation* Tanpa Musim



Keterangan :

○ Simpul memiliki 2 cabang metode dan dipisah menjadi 2 flow chart

Gambar 86. Flow Graph Menu Autocorrelation Dengan Musim

2) Cyclomatic Complexity

Pada tahapan ini, peneliti menghitung nilai *cyclomatic complexity* yang dinotasikan sebagai $V(G)$. Nilai tersebut dapat dihitung menggunakan rumus $V(G) = E - N + 2$. Notasi E melambangkan jumlah garis (*edge*) yang terdapat pada *flow graph* dan notasi N melambangkan jumlah simpul (*node*) yang terdapat pada *flow graph*. Pada menu

autocorrelation jumlah garis dan simpul yang diperoleh pada *flow graph* masing-masing berjumlah 33 dan 21 sehingga diperoleh nilai *cyclomatic complexity* sebesar 14. Nilai tersebut memiliki arti bahwa jumlah jalur yang terdapat pada kode program menu *autocorrelation* berjumlah empat belas jalur yang selanjutnya akan dijadikan sebagai dasar jumlah jalur untuk tahapan *derriving test cases*.

3) *Derriving Test Cases*

Berdasarkan kedua tahapan sebelumnya, jumlah *test cases* yang dibentuk oleh peneliti pada menu *autocorrelation* berjumlah empat belas *test cases*. Selanjutnya, setiap *test cases* akan diuji dengan membandingkan hasil ekspektasi dan hasil aktual untuk menentukan keberhasilan suatu *test case*. Berikut Tabel 10 menampilkan hasil pengujian seluruh *test cases* menu *autocorrelation*.

Tabel 10. Hasil *Test Cases* Menu *Autocorrelation*

Kode <i>Test Case</i>	Jalur <i>Test Case</i>	Hasil Eskpektasi	Hasil Aktual	Hasil Pengujian
(1)	(2)	(3)	(4)	(5)
TCA01	A01-A02-A21	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCA02	A01-A03-A04-A21	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCA03	A01-A03-A05-A06-A21	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCA04	A01-A03-A05-A07-A08-A21	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCA05	A01-A03-A05-A07-A09-A10-A16-A21	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil

TCA06	A01-A03-A05-A07-A09-A10-A17-A18-A21	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil
TCA07	A01-A03-A05-A07-A09-A10-A17-A19-A21	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil
TCA08	A01-A03-A05-A07-A09-A10-A17-A20-A21	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil
TCA09	A01-A03-A05-A07-A09-A11-A12-A21	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCA10	A01-A03-A05-A07-A09-A11-A13-A14-A21	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCA11	A01-A03-A05-A07-A09-A11-A13-A15-A16-A21	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCA12	A01-A03-A05-A07-A09-A11-A13-A15-A17-A18-A21	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil
TCA13	A01-A03-A05-A07-A09-A11-A13-A15-A17-A19-A21	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil
TCA14	A01-A03-A05-A07-A09-A11-A13-A15-A17-A20-A21	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil

Sumber: Dokumen Peneliti

4) Perbandingan *Output* Statistik

Pada tahapan ini, peneliti membandingkan *output* statistik antara Statify dan Eviews. *Case* yang digunakan peneliti pada tahapan ini adalah metode *autocorrelation* yang sama yang tersedia pada masing-masing aplikasi, yaitu autokorelasi pada data aktual, data diferensiasi pertama, dan data diferensiasi kedua. Selain itu, data yang digunakan pada tahapan ini adalah data penumpang kereta api bulanan tahun 2017 hingga 2024 yang diperoleh dari *website* resmi BPS. Adapun variabel yang digunakan adalah

Rerata Jarak Perjalanan (RJP). Berikut ditampilkan perbandingan *output* autokorelasi setiap level diferensiasi data aplikasi Statify dan Eviews pada Gambar 87 sampai 89.

Autocorrelation Statify							Autocorrelation Eviews					
	ACF	SE	Ljung-Box	df	p-value	PACF	SE	AC	PAC	Q-Stat	Prob	
1	0.727	0.102	52.379	1	0.000	0.727	0.102	1	0.727	0.727	52.379	0.000
2	0.626	0.146	91.557	2	0.000	0.205	0.102	2	0.626	0.205	91.557	0.000
3	0.460	0.172	112.986	3	0.000	-0.112	0.102	3	0.460	-0.112	112.99	0.000
4	0.373	0.184	127.235	4	0.000	0.021	0.102	4	0.373	0.021	127.23	0.000
5	0.352	0.192	140.051	5	0.000	0.153	0.102	5	0.352	0.153	140.05	0.000
6	0.348	0.199	152.704	6	0.000	0.093	0.102	6	0.348	0.093	152.70	0.000
7	0.259	0.205	159.815	7	0.000	-0.172	0.102	7	0.259	-0.172	159.81	0.000
8	0.178	0.208	163.199	8	0.000	-0.105	0.102	8	0.178	-0.105	163.20	0.000
9	0.125	0.210	164.880	9	0.000	0.069	0.102	9	0.125	0.069	164.88	0.000
10	-0.002	0.211	164.880	10	0.000	-0.187	0.102	10	-0.002	-0.187	164.88	0.000
11	0.002	0.211	164.881	11	0.000	0.041	0.102	11	0.002	0.041	164.88	0.000
12	0.036	0.211	165.024	12	0.000	0.186	0.102	12	0.036	0.186	165.02	0.000
13	-0.073	0.211	165.630	13	0.000	-0.308	0.102	13	-0.073	-0.308	165.63	0.000
14	-0.073	0.211	166.240	14	0.000	-0.002	0.102	14	-0.073	-0.002	166.24	0.000
15	-0.153	0.211	168.973	15	0.000	0.018	0.102	15	-0.153	0.018	168.97	0.000
16	-0.192	0.212	173.297	16	0.000	-0.038	0.102	16	-0.192	-0.038	173.30	0.000
17	-0.214	0.214	178.762	17	0.000	-0.104	0.102	17	-0.214	-0.104	178.76	0.000
18	-0.230	0.216	185.165	18	0.000	-0.079	0.102	18	-0.230	-0.079	185.17	0.000
19	-0.281	0.219	194.812	19	0.000	-0.002	0.102	19	-0.281	-0.002	194.81	0.000
20	-0.339	0.223	209.044	20	0.000	-0.231	0.102	20	-0.339	-0.231	209.04	0.000

Gambar 87. Tabel Autokorelasi pada Data Aktual dengan 20 Lag Data

Autocorrelation Statify								Autocorrelation Eviews				
	ACF	SE	Ljung-Box	df	p-value	PACF	SE	AC	PAC	Q-Stat	Prob	
1	-0.318	0.103	9.905	1	0.002	-0.318	0.103	1	-0.318	-0.318	9.9045	0.002
2	0.119	0.112	11.304	2	0.004	0.020	0.103	2	0.119	0.020	11.304	0.004
3	-0.149	0.114	13.520	3	0.004	-0.117	0.103	3	-0.149	-0.117	13.520	0.004
4	-0.120	0.116	14.973	4	0.005	-0.227	0.103	4	-0.120	-0.227	14.973	0.005
5	-0.034	0.117	15.092	5	0.010	-0.151	0.103	5	-0.034	-0.151	15.092	0.010
6	0.152	0.117	17.481	6	0.008	0.110	0.103	6	0.152	0.110	17.481	0.008
7	-0.003	0.119	17.482	7	0.015	0.046	0.103	7	-0.003	0.046	17.482	0.015
8	-0.042	0.119	17.667	8	0.024	-0.117	0.103	8	-0.042	-0.117	17.667	0.024
9	0.120	0.119	19.217	9	0.023	0.109	0.103	9	0.120	0.109	19.217	0.023
10	-0.235	0.121	25.183	10	0.005	-0.118	0.103	10	-0.235	-0.118	25.183	0.005
11	-0.070	0.125	25.725	11	0.007	-0.249	0.103	11	-0.070	-0.249	25.725	0.007
12	0.299	0.126	35.653	12	0.000	0.292	0.103	12	0.299	0.292	35.653	0.000
13	-0.211	0.133	40.677	13	0.000	-0.068	0.103	13	-0.211	-0.068	40.677	0.000
14	0.154	0.137	43.378	14	0.000	-0.103	0.103	14	0.154	-0.103	43.378	0.000
15	-0.084	0.138	44.187	15	0.000	-0.015	0.103	15	-0.084	-0.015	44.187	0.000
16	-0.023	0.139	44.248	16	0.000	0.055	0.103	16	-0.023	0.055	44.248	0.000
17	-0.030	0.139	44.351	17	0.000	-0.017	0.103	17	-0.030	-0.017	44.351	0.000
18	0.073	0.139	44.987	18	0.000	-0.074	0.103	18	0.073	-0.074	44.987	0.000
19	0.012	0.139	45.006	19	0.001	0.151	0.103	19	0.012	0.151	45.006	0.001
20	-0.105	0.139	46.349	20	0.001	-0.157	0.103	20	-0.105	-0.157	46.349	0.001

Gambar 88. Tabel Autokorelasi pada Data Diferensiasi Pertama dengan 20 Lag Data

Autocorrelation Statify							Autocorrelation Eviews			
	ACF	SE	Ljung-Box	df	p-value	PACF	SE	AC	PAC	Q-Stat Prob
1	-0.665	0.103	42.941	1	0.000	-0.665	0.103	1	-0.665	-0.665 42.941 0.000
2	0.272	0.142	50.209	2	0.000	-0.306	0.103	2	0.272	-0.306 50.209 0.000
3	-0.117	0.147	51.562	3	0.000	-0.166	0.103	3	-0.117	-0.166 51.562 0.000
4	-0.017	0.148	51.590	4	0.000	-0.228	0.103	4	-0.017	-0.228 51.590 0.000
5	-0.039	0.148	51.745	5	0.000	-0.363	0.103	5	-0.039	-0.363 51.745 0.000
6	0.123	0.148	53.293	6	0.000	-0.201	0.103	6	0.123	-0.201 53.293 0.000
7	-0.034	0.149	53.409	7	0.000	-0.004	0.103	7	-0.034	-0.004 53.409 0.000
8	-0.093	0.149	54.316	8	0.000	-0.217	0.103	8	-0.093	-0.217 54.316 0.000
9	0.209	0.150	58.970	9	0.000	0.026	0.103	9	0.209	0.026 58.970 0.000
10	-0.197	0.153	63.130	10	0.000	0.117	0.103	10	-0.197	0.117 63.130 0.000
11	-0.077	0.156	63.778	11	0.000	-0.369	0.103	11	-0.077	-0.369 63.778 0.000
12	0.327	0.156	75.514	12	0.000	0.046	0.103	12	0.327	0.046 75.514 0.000
13	-0.330	0.163	87.618	13	0.000	0.041	0.103	13	-0.330	0.041 87.618 0.000
14	0.232	0.170	93.680	14	0.000	-0.036	0.103	14	0.232	-0.036 93.680 0.000
15	-0.117	0.173	95.240	15	0.000	-0.072	0.103	15	-0.117	-0.072 95.240 0.000
16	0.039	0.174	95.416	16	0.000	0.020	0.103	16	0.039	0.020 95.416 0.000
17	-0.059	0.174	95.831	17	0.000	0.063	0.103	17	-0.059	0.063 95.831 0.000
18	0.066	0.175	96.356	18	0.000	-0.157	0.103	18	0.066	-0.157 96.356 0.000
19	0.024	0.175	96.427	19	0.000	0.146	0.103	19	0.024	0.146 96.427 0.000
20	-0.157	0.175	99.414	20	0.000	-0.002	0.103	20	-0.157	-0.002 99.414 0.000

Gambar 89. Tabel Autokorelasi pada Data Diferensiasi Kedua dengan 20 Lag Data

Berdasarkan *output* tabel pada Gambar 87 sampai 89 terlihat bahwa menu *autocorrelation* sudah memberikan hasil yang sama antara Statify dan Eviews untuk setiap level diferensiasi data. Pada Statify, nilai autokorelasi dihitung menggunakan rumus-rumus yang tertulis pada buku *Time Series Analysis With Application in R* (Crycer & Chan, 2008).

4. Menu Unit Root Test

Berikut tahapan *white-box testing* menu *unit root test*.

1) Flow Graph Notation

Pada tahapan ini, dilakukan pendaftaran simpul untuk membentuk dasar-dasar *flow graph notation* menu *unit root test*. Peneliti mengklasifikasi setiap bagian kode program menu *unit root test* menjadi kumpulan *statement* sederhana lalu diubah menjadi kumpulan notasi

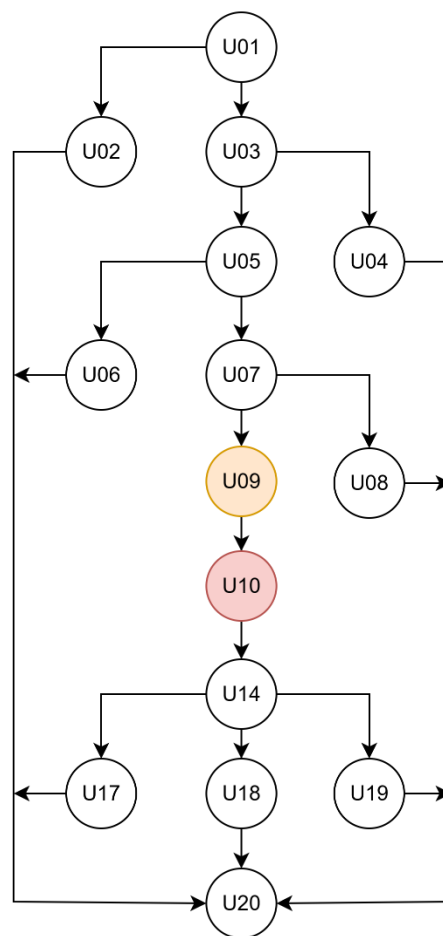
simpul. Berikut kumpulan *statement* menu *unit root test* yang dijadikan kumpulan notasi simpul yang ditampilkan pada Tabel 11.

Tabel 11. Daftar Simpul Menu *Unit Root Test*

Kode Simpul	Nama Simpul
(1)	(2)
U01	Menjalankan menu <i>unit root test</i>
U02	Variabel terpilih kosong
U03	Variabel terpilih berisi
U04	Variabel terpilih bukan numerik
U05	Variabel terpilih numerik
U06	Data kosong
U07	Data berisi
U08	Observasi data kurang dari 20
U09	Observasi data lebih dari sama dengan 20
U10	metode <i>dickey-fuller</i>
U11	metode <i>augmented dickey-fuller</i>
U12	Jumlah <i>lag</i> di luar batas
U13	Jumlah <i>lag</i> di dalam batas
U14	Tanpa diferensiasi
U15	Diferensiasi pertama
U16	Diferensiasi kedua
U17	Tanpa intersep
U18	Dengan intersep
U19	Dengan tren dan intersep
U20	Hasil

Sumber: Dokumen Peneliti

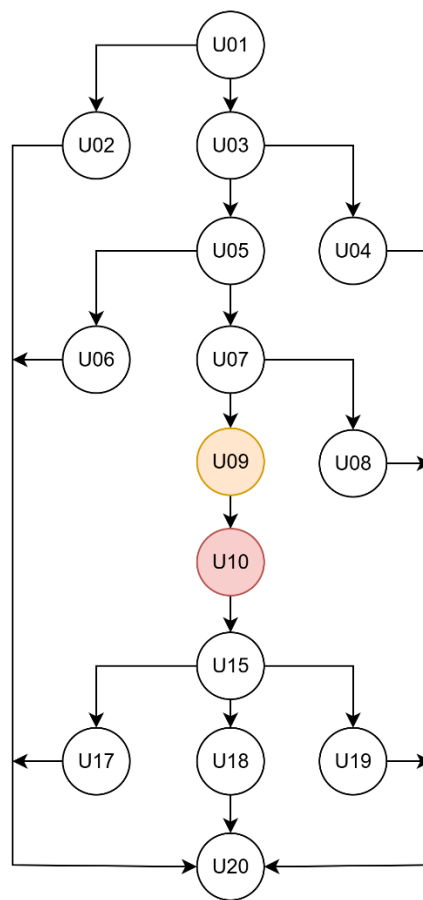
Setelah mendaftarkan nama-nama simpul beserta kodenya, peneliti melanjutkan ke tahapan pembentukan *flow graph*. *Flow graph* pada menu ini dibentuk menjadi enam bagian karena terdapat pembagian metode *unit root test* sebanyak dua cabang dan tingkat diferensiasi data sebanyak tiga cabang sehingga terbentuk enam kemungkinan yang perlu disederhanakan untuk mempermudah pembacaan *flow graph* di tahapan selanjutnya. Berikut kumpulan *flow graph* menu *unit root test* yang ditampilkan pada Gambar 90 sampai 95.



Keterangan :

- Simpul memiliki 2 cabang metode unit root test yang terpisah
- Simpul memiliki 3 cabang tingkat differensiasi data yang terpisah

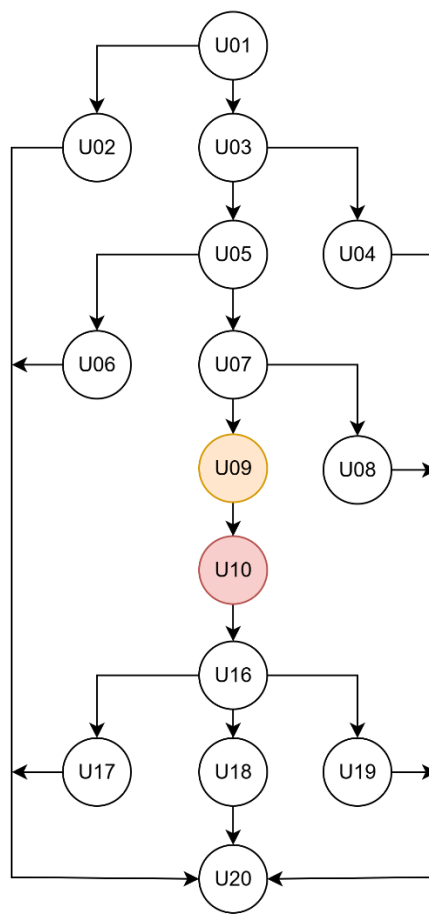
Gambar 90. *Flow Graph Menu Unit Root Test Metode Dickey Fuller Tanpa Diferensiasi*



Keterangan :

- Simpul memiliki 2 cabang metode unit root test yang terpisah
- Simpul memiliki 3 cabang tingkat differensiasi data yang terpisah

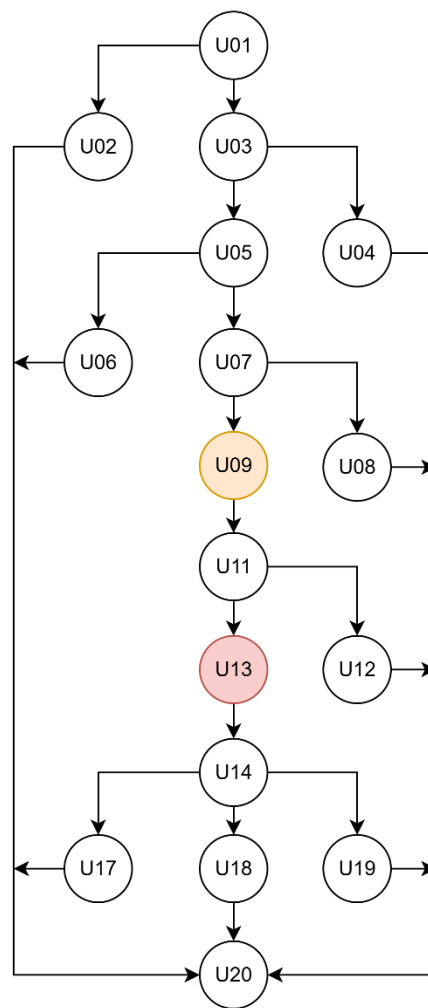
Gambar 91. *Flow Graph Menu Unit Root Test Metode Dickey-Fuller Diferensiasi Pertama*



Keterangan :

- Simpul memiliki 2 cabang metode unit root test yang terpisah
- Simpul memiliki 3 cabang tingkat differensiasi data yang terpisah

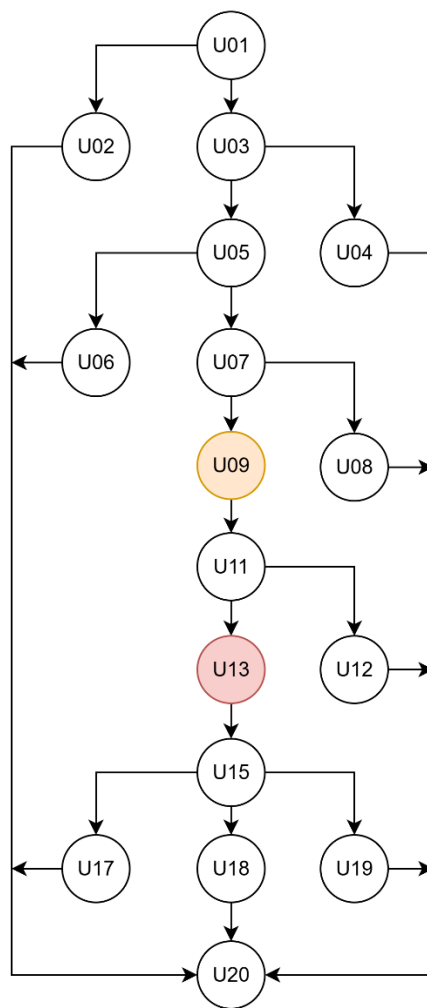
Gambar 92. *Flow Graph Menu Unit Root Test Metode Dickey-Fuller Diferensiasi Kedua*



Keterangan :

- Simpul memiliki 2 cabang metode unit root test yang terpisah
- Simpul memiliki 3 cabang tingkat differensiasi data yang terpisah

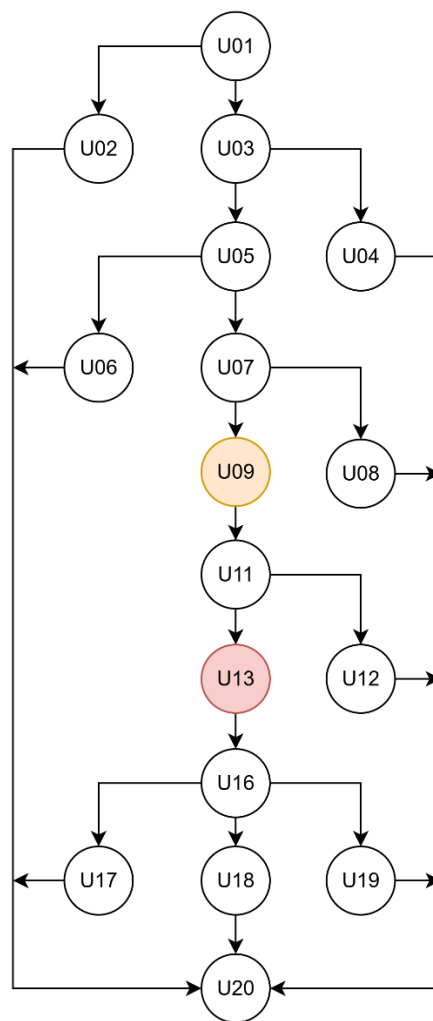
Gambar 93. *Flow Graph Menu Unit Root Test Metode Augmented Dickey-Fuller Tanpa Diferensiasi*



Keterangan :

- Simpul memiliki 2 cabang metode unit root test yang terpisah
- Simpul memiliki 3 cabang tingkat differensiasi data yang terpisah

Gambar 94. *Flow Graph Menu Unit Root Test Metode Augmented Dickey-Fuller Diferensiasi Pertama*



Keterangan :

- Simpul memiliki 2 cabang metode unit root test yang terpisah
- Simpul memiliki 3 cabang tingkat differensiasi data yang terpisah

Gambar 95. *Flow Graph Menu Unit Root Test Metode Augmented Dickey-Fuller Diferensiasi Kedua*

2) *Cyclomatic Complexity*

Pada tahapan ini, peneliti menghitung nilai *cyclomatic complexity* yang dinotasikan sebagai $V(G)$. Nilai tersebut dapat dihitung menggunakan rumus $V(G) = E - N + 2$. Notasi E melambangkan jumlah garis (*edge*) yang terdapat pada *flow graph* dan notasi N melambangkan

jumlah simpul (*node*) yang terdapat pada *flow graph*. Pada menu *smoothing* jumlah garis dan simpul yang diperoleh pada *flow graph* masing-masing berjumlah 41 dan 20 sehingga diperoleh nilai *cyclomatic complexity* sebesar 23. Nilai tersebut memiliki arti bahwa jumlah jalur yang terdapat pada kode program menu *unit root test* berjumlah dua puluh tiga jalur yang selanjutnya akan dijadikan sebagai dasar jumlah jalur untuk tahapan *derriving test cases*.

3) *Derriving Test Cases*

Berdasarkan kedua tahapan sebelumnya, jumlah *test cases* yang dibentuk oleh peneliti pada menu *unit root test* berjumlah dua puluh tiga *test cases*. Selanjutnya, setiap *test cases* akan diuji dengan membandingkan hasil ekspektasi dan hasil aktual untuk menentukan keberhasilan suatu *test case*. Berikut Tabel 12 menampilkan hasil pengujian seluruh *test cases* menu *unit root test*.

Tabel 12. Hasil *Test Cases* Menu *Unit Root Test*

Kode Test Case	Jalur Test Case	Hasil Eskpektasi	Hasil Aktual	Hasil Pengujian
(1)	(2)	(3)	(4)	(5)
TCU01	U01-U02-U20	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCU02	U01-U03-U04-U20	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCU03	U01-U03-U05-U06-U20	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCU04	U01-U03-U05-U07-U08-U20	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCU05	U01-U03-U05-U07-U09-U10-U14-U17-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan	Berhasil

			<i>output statistik</i>	
TCU06	U01-U03-U05-U07-U09-U10-U14-U18-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU07	U01-U03-U05-U07-U09-U10-U14-U19-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU08	U01-U03-U05-U07-U09-U10-U15-U17-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU09	U01-U03-U05-U07-U09-U10-U15-U18-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU10	U01-U03-U05-U07-U09-U10-U15-U19-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU11	U01-U03-U05-U07-U09-U10-U16-U17-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU12	U01-U03-U05-U07-U09-U10-U16-U18-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU13	U01-U03-U05-U07-U09-U10-U16-U19-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU14	U01-U03-U05-U07-U09-U11-U12-U20	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCU15	U01-U03-U05-U07-U09-U11-U13-U14-U17-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU16	U01-U03-U05-U07-U09-U11-U13-U14-U18-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU17	U01-U03-U05-U07-U09-U11-U13-U14-U19-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU18	U01-U03-U05-U07-U09-U11-	Menghasilkan <i>output statistik</i>	Berhasil menampilkan	Berhasil

	U13-U15-U17-U20		<i>output statistik</i>	
TCU19	U01-U03-U05-U07-U09-U11-U13-U15-U18-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU20	U01-U03-U05-U07-U09-U11-U13-U15-U19-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU21	U01-U03-U05-U07-U09-U11-U13-U16-U17-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU22	U01-U03-U05-U07-U09-U11-U13-U16-U18-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil
TCU23	U01-U03-U05-U07-U09-U11-U13-U16-U19-U20	Menghasilkan <i>output statistik</i>	Berhasil menampilkan <i>output statistik</i>	Berhasil

Sumber: Dokumen Peneliti

4) Perbandingan *Output Statistik*

Pada tahapan ini, peneliti membandingkan *output statistik* antara Statify dan Eviews. *Case* yang digunakan peneliti pada tahapan ini adalah metode *unit root test* yang sama yang tersedia pada masing-masing aplikasi, yaitu uji *dickey-fuller* dan *augmented dickey-fuller*. Namun, pada Eviews uji *dickey-fuller* dianggap sama dengan uji *augmented dickey-fuller* dengan masukan parameter *lag* bernilai 0. Selain itu, data yang digunakan pada tahapan ini adalah data penumpang kereta api bulanan tahun 2017 hingga 2024 yang diperoleh dari *website* resmi BPS. Adapun variabel yang digunakan adalah Rerata Jarak Perjalanan (RJP). Berikut ditampilkan perbandingan *output uji dickey-fuller* dan *augmented dickey-fuller* aplikasi Statify dan Eviews pada Gambar 96 sampai 97.

Dickey-Fuller Test Statify

Dickey-Fuller Test Statistic				
		t-statistic	p-value	
Dickey-Fuller Test		-3.806	0.003	
	1%	-3.501		
Critical Value	5%	-2.892		
	10%	-2.583		

Coefficients Test				
	coefficient	standard error	t-statistic	p-value
Constant	16.689	4.452	3.749	0.000
RJP (km)	-0.269	0.071	-3.806	0.000

Selection Criterion	
	value
R-Squared	0.135
Adj. R-Squared	0.125
S.E. of Regression	5.556
Sum Squared Resid	2871.242
Log Likelihood	-296.709
F-Statistic	14.485
Prob(F-Stat)	0.000
Mean Dependent Var	-0.116
S.D. Dependent Var	5.942
Akaike Info Crit	6.289
Schwarz Criterion	6.342
Hannan-Quinn	6.310
Durbin-Watson	2.298

Dickey-Fuller Test Eviews

Augmented Dickey-Fuller Unit Root Test on RJP				
Null Hypothesis: RJP has a unit root				
Exogenous: Constant				
Lag Length: 0 (Fixed)				
			t-Statistic	Prob.*
Augmented Dickey-Fuller test statistic			-3.805980	0.0040
Test critical values:				
	1% level		-3.500669	
	5% level		-2.892200	
	10% level		-2.583192	
Variable	Coefficient	Std. Error	t-Statistic	Prob.
RJP(-1)	-0.269268	0.070749	-3.805980	0.0003
C	16.68901	4.452018	3.748640	0.0003
R-squared	0.134767			-0.115789
Adjusted R-squared	0.125463			5.941613
S.E. of regression	5.556399			6.288605
Sum squared resid	2871.242			6.342371
Log likelihood	-296.7088			6.310331
F-statistic	14.48548			2.298191
Prob(F-statistic)	0.000253			

Gambar 96. Tabel Hasil *Dickey-Fuller Test* Data Aktual dengan Intersep

Dickey-Fuller Test Statify

Augmented Dickey-Fuller Test Statistic

	t-statistic	p-value
Augmented Dickey-Fuller Test	-2.910	0.044
Critical Value	1%	-3.503
	5%	-2.893
	10%	-2.584

Coefficients Test

	coefficient	standard error	t-statistic	p-value
Constant	14.344	4.992	2.874	0.005
RJP (km) Diff Lag (1)	-0.169	0.114	-1.490	0.140
RJP (km) Diff Lag (2)	0.109	0.106	1.033	0.304
RJP (km)	-0.231	0.079	-2.910	0.005

Selection Criterion

	value
R-Squared	0.182
Adj. R-Squared	0.154
S.E. of Regression	5.502
Sum Squared Resid	2694.002
Log Likelihood	-288.489
F-Statistic	6.601
Prob(F-Stat)	0.000
Mean Dependent Var	-0.054
S.D. Dependent Var	5.983
Akaike Info Crit	6.290
Schwarz Criterion	6.399
Hannan-Quinn	6.334
Durbin-Watson	1.984

Dickey-Fuller Test Eviews

Augmented Dickey-Fuller Unit Root Test on RJP				
Null Hypothesis: RJP has a unit root				
Exogenous: Constant				
Lag Length: 2 (Fixed)				
			t-Statistic	Prob.*
Augmented Dickey-Fuller test statistic			-2.909669	0.0480
Test critical values:		1% level	-3.502238	
		5% level	-2.892879	
		10% level	-2.583553	
*MacKinnon (1996) one-sided p-values.				
Variable	Coefficient	Std. Error	t-Statistic	Prob.
RJP(-1)	-0.230936	0.079369	-2.909669	0.0046
D(RJP(-1))	-0.169197	0.113575	-1.489737	0.1398
D(RJP(-2))	0.109481	0.105967	1.033159	0.3043
C	14.34406	4.991715	2.873574	0.0051
R-squared	0.182016	Mean dependent var	-0.053763	
Adjusted R-squared	0.154443	S.D. dependent var	5.983187	
S.E. of regression	5.501789	Akaike info criterion	6.290082	
Sum squared resid	2694.002	Schwarz criterion	6.399011	
Log likelihood	-288.4888	Hannan-Quinn criter.	6.334065	
F-statistic	6.601360	Durbin-Watson stat	1.983759	
Prob(F-statistic)	0.000444			

Gambar 97. Tabel Hasil *Augmented Dickey-Fuller Test* Data Aktual dengan Intersep dan 2 Lag Data

Berdasarkan *output* tabel pada Gambar 96 dan 97 terlihat bahwa penghitungan uji *dickey-fuller* dan *augmented dickey-fuller* sudah memberikan hasil yang sama antara Statify dan Eviews, namun masih terdapat sedikit perbedaan pada nilai *p-value*. Hal ini dikarenakan Pada Statify, nilai *p-value* dihitung menggunakan rumus-rumus yang tertulis pada jurnal *Approximate Asymptotic Distribution Functions for Unit-Root and Cointegration Tests* (Mackinnon, 1994).

5. Menu *Box-Jenkins Model*

Berikut tahapan *white-box testing* menu *box-jenkins model*.

1) *Flow Graph Notation*

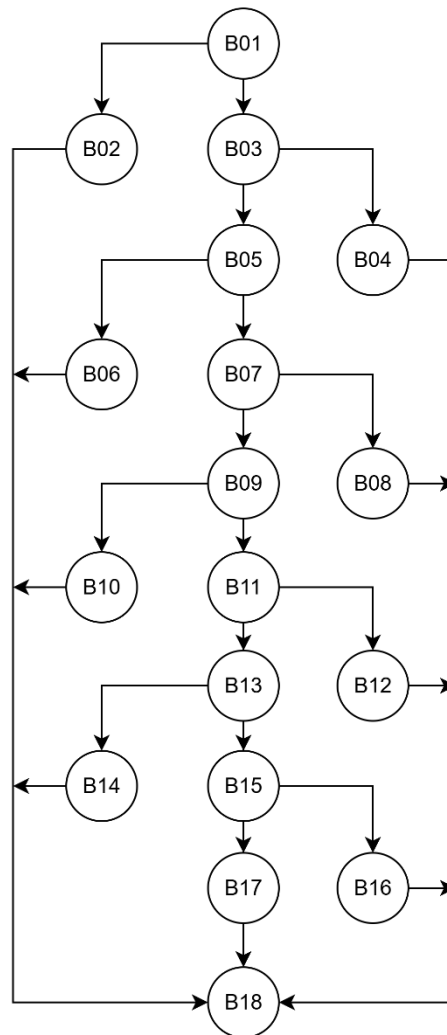
Pada tahapan ini, dilakukan pendaftaran simpul untuk membentuk dasar-dasar *flow graph notation* menu *box-jenkins model*. Peneliti mengklasifikasi setiap bagian kode program menu *box-jenkins model* menjadi kumpulan *statement* sederhana lalu diubah menjadi kumpulan notasi simpul. Berikut kumpulan *statement* menu *box-jenkins model* yang dijadikan kumpulan notasi simpul yang ditampilkan pada Tabel 13.

Tabel 13. Daftar Simpul Menu *Box-Jenkins Model*

Kode Simpul	Nama Simpul
(1)	(2)
B01	Menjalankan menu <i>box-jenkins model</i>
B02	Variabel terpilih kosong
B03	Variabel terpilih berisi
B04	Variabel terpilih bukan numerik
B05	Variabel terpilih numerik
B06	Data kosong
B07	Data berisi
B08	Observasi data kurang dari 20
B09	Observasi data lebih dari sama dengan 20
B10	Belum mengatur konfigurasi waktu
B11	Sudah mengatur konfigurasi waktu
B12	Parameter p di luar batas
B13	Parameter p di dalam batas
B14	Parameter d di luar batas
B15	Parameter d di dalam batas
B16	Parameter q di luar batas
B17	Parameter q di dalam batas
B18	Hasil

Sumber: Dokumen Peneliti

Setelah mendaftarkan nama-nama simpul beserta kodenya, peneliti melanjutkan ke tahapan pembentukan *flow graph*. Berikut kumpulan *flow graph* menu *box-jenkins model* yang ditampilkan pada Gambar 98.



Gambar 98. *Flow Graph Menu Box-Jenkins Model*

2) Cyclomatic Complexity

Pada tahapan ini, peneliti menghitung nilai *cyclomatic complexity* yang dinotasikan sebagai $V(G)$. Nilai tersebut dapat dihitung

menggunakan rumus $V(G) = E - N + 2$. Notasi E melambangkan jumlah garis (*edge*) yang terdapat pada *flow graph* dan notasi N melambangkan jumlah simpul (*node*) yang terdapat pada *flow graph*. Pada menu *box-jenkins model* jumlah garis dan simpul yang diperoleh pada *flow graph* masing-masing berjumlah 25 dan 18 sehingga diperoleh nilai *cyclomatic complexity* sebesar 9. Nilai tersebut memiliki arti bahwa jumlah jalur yang terdapat pada kode program menu *box-jenkins model* berjumlah sembilan jalur yang selanjutnya akan dijadikan sebagai dasar jumlah jalur untuk tahapan *derriving test cases*.

3) *Derriving Test Cases*

Berdasarkan kedua tahapan sebelumnya, jumlah *test cases* yang dibentuk oleh peneliti pada menu *box-jenkins model* berjumlah sembilan *test cases*. Selanjutnya, setiap *test cases* akan diuji dengan membandingkan hasil ekspektasi dan hasil aktual untuk menentukan keberhasilan suatu *test case*. Berikut Tabel 14 menampilkan hasil pengujian seluruh *test cases* menu *box-jenkins model*.

Tabel 14. Hasil *Test Cases* Menu *Box-Jenkins Model*

Kode <i>Test Case</i>	Jalur <i>Test Case</i>	Hasil Eskpektasi	Hasil Aktual	Hasil Pengujian
(1)	(2)	(3)	(4)	(5)
TCB01	B01-B02-B18	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCB02	B01-B03-B04-B18	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCB03	B01-B03-B05-B06-B18	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil

TCB04	B01-B03-B05-B07-B08-B18	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCB05	B01-B03-B05-B07-B09-B10-B18	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCB06	B01-B03-B05-B07-B09-B11-B12-B18	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCB07	B01-B03-B05-B07-B09-B11-B13-B14-B18	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCB08	B01-B03-B05-B07-B09-B11-B13-B15-B16-B18	Menampilkan pesan <i>error</i>	Berhasil menampilkan pesan <i>error</i>	Berhasil
TCB09	B01-B03-B05-B07-B09-B11-B13-B15-B17-B18	Menghasilkan <i>output</i> statistik	Berhasil menampilkan <i>output</i> statistik	Berhasil

Sumber: Dokumen Peneliti

4) Perbandingan *Output* Statistik

Pada tahapan ini, peneliti membandingkan *output* statistik antara Statify dan Eviews. *Case* yang digunakan peneliti pada tahapan ini adalah model *box-jenkins* yang umum digunakan dalam materi Mata Kuliah Analisis Runtun Waktu, yaitu model *Auto-Regressive* (AR), *Moving Average* (MA), *Auto-Regressive Moving Average* (ARMA), dan *Auto-Regressive Intergrated Moving Average* (ARIMA). Selain itu, data yang digunakan pada tahapan ini adalah data penumpang kereta api bulanan tahun 2017 hingga 2024 yang diperoleh dari *website* resmi BPS. Adapun variabel yang digunakan adalah Rerata Jarak Perjalanan (RJP). Berikut ditampilkan perbandingan *output* model AR, MA, ARMA, dan ARIMA aplikasi Statify dan Eviews pada Gambar 99 sampai 102.

Output Model AR Statify

Coefficients Test for ARIMA (2,0,0)					Selection Criteria for RJP (km)		F-Statistic	115.204
	coef	std. error	t value	p-value		value	Prob(F-Stat)	0.000
Constant	62.852	1.582	39.735	0.000	R-Squared	0.553	Mean Dependent Var	62.353
AR(1)	0.575	0.092	6.243	0.000	Adj. R-Squared	0.544	S.D. Dependent Var	8.077
AR(2)	0.217	0.244	0.892	0.375	S.E. of Regression	5.456	Akaike Info Crit	6.262
					Sum Squared Resid	2768.069	Schwarz Criterion	6.342
					Log Likelihood	-297.573	Hannan-Quinn	6.294
							Durbin-Watson	1.944

Output Model AR Eviews

Variable	Coefficient	Std. Error	t-Statistic	Prob.
C	62.28240	2.650818	23.49554	0.0000
AR(1)	0.572322	0.080079	7.146936	0.0000
AR(2)	0.210894	0.110841	1.902671	0.0602
SIGMASQ	28.71761	2.821893	10.17672	0.0000
R-squared	0.555127	Mean dependent var	62.35281	
Adjusted R-squared	0.540620	S.D. dependent var	8.076625	
S.E. of regression	5.474139	Akaike info criterion	6.287446	
Sum squared resid	2756.891	Schwarz criterion	6.394294	
Log likelihood	-297.7974	Hannan-Quinn criter.	6.330636	
F-statistic	38.26685	Durbin-Watson stat	1.943534	
Prob(F-statistic)	0.000000			

Gambar 99. Tabel Hasil Model AR dengan Parameter Order 2

Output Model MA Statify

Coefficients Test for ARIMA (0,0,2)					Selection Criteria for RJP (km)		F-Statistic	91.666
	coef	std. error	t value	p-value		value	Prob(F-Stat)	0.000
Constant	62.521	1.578	39.611	0.000	R-Squared	0.496	Mean Dependent Var	62.353
MA(1)	-0.422	0.059	-7.179	0.000	Adj. R-Squared	0.486	S.D. Dependent Var	8.077
MA(2)	-0.700	0.085	-8.239	0.000	S.E. of Regression	5.793	Akaike Info Crit	6.382
					Sum Squared Resid	3120.904	Schwarz Criterion	6.462
					Log Likelihood	-303.332	Hannan-Quinn	6.414
							Durbin-Watson	1.613

Output Model MA Eviews

Variable	Coefficient	Std. Error	t-Statistic	Prob.
C	62.37273	1.366916	45.63026	0.0000
MA(1)	0.421030	0.092929	4.530690	0.0000
MA(2)	0.695693	0.092406	7.528664	0.0000
SIGMASQ	32.19764	3.693541	8.717281	0.0000
R-squared	0.501217	Mean dependent var	62.35281	
Adjusted R-squared	0.484952	S.D. dependent var	8.076625	
S.E. of regression	5.796338	Akaike info criterion	6.407550	
Sum squared resid	3090.973	Schwarz criterion	6.514398	
Log likelihood	-303.5624	Hannan-Quinn criter.	6.450740	
F-statistic	30.81629	Durbin-Watson stat	1.604181	
Prob(F-statistic)	0.000000			

Gambar 100. Tabel Hasil Model MA dengan Parameter Order 2

Output Model ARMA Statify

Coefficients Test for ARIMA (2,0,1)					Selection Criteria for RJP (km)		Log Likelihood	-297.160
	coef	std. error	t value	p-value		value	F-Statistic	115.743
Constant	62.874	1.764	35.639	0.000	R-Squared	0.557	Prob(F-Stat)	0.000
AR(1)	0.265	0.098	2.696	0.008	Adj. R-Squared	0.543	Mean Dependent Var	62.353
AR(2)	0.447	0.090	4.985	0.000	S.E. of Regression	5.462	S.D. Dependent Var	8.077
MA(1)	-0.323	0.057	-5.701	0.000	Sum Squared Resid	2744.378	Akaike Info Crit	6.274
					Log Likelihood	-297.160	Schwarz Criterion	6.381
							Hannan-Quinn	6.317
							Durbin-Watson	1.970

Output Model ARMA Eviews

Variable	Coefficient	Std. Error	t-Statistic	Prob.
C	62.31846	2.526472	24.66620	0.0000
AR(1)	0.251020	0.518625	0.484011	0.6295
AR(2)	0.447266	0.364149	1.228248	0.2225
MA(1)	0.334875	0.535601	0.625233	0.5334
SIGMASQ	28.45193	2.823278	10.07762	0.0000
R-squared	0.559243	Mean dependent var	62.35281	
Adjusted R-squared	0.539869	S.D. dependent var	8.076625	
S.E. of regression	5.478615	Akaike info criterion	6.299249	
Sum squared resid	2731.385	Schwarz criterion	6.432809	
Log likelihood	-297.3640	Hannan-Quinn criter.	6.353236	
F-statistic	28.86571	Durbin-Watson stat	1.969880	
Prob(F-statistic)	0.000000			

Gambar 101. Tabel Hasil Model ARMA dengan Parameter Order 2,1

Output Model ARIMA Statify

Coefficients Test for ARIMA (2,1,1)					Selection Criteria for RJP (km)		F-Statistic	98.404
	coef	std. error	t value	p-value		value	Prob(F-Stat)	0.000
Constant	-0.117	0.342	-0.341	0.734	R-Squared	0.520	Mean Dependent Var	62.353
AR(1)	-0.699	0.404	-1.731	0.087	Adj. R-Squared	0.498	S.D. Dependent Var	8.077
AR(2)	-0.091	0.093	-0.973	0.333	S.E. of Regression	5.720	Akaike Info Crit	6.356
MA(1)	-0.391	0.385	-1.016	0.312	Sum Squared Resid	2977.397	Schwarz Criterion	6.463
					Log Likelihood	-301.072	Hannan-Quinn	6.399
							Durbin-Watson	1.992

Output Model ARIMA Eviews

Variable	Coefficient	Std. Error	t-Statistic	Prob.
C	-0.114059	0.478026	-0.238605	0.8120
AR(1)	-0.689932	3.102091	-0.222409	0.8245
AR(2)	-0.089495	1.060593	-0.084382	0.9329
MA(1)	0.382620	3.081708	0.124159	0.9015
SIGMASQ	31.31951	3.475630	9.011174	0.0000
R-squared	0.103393	Mean dependent var	-0.115789	
Adjusted R-squared	0.063544	S.D. dependent var	5.941613	
S.E. of regression	5.749737	Akaike info criterion	6.388531	
Sum squared resid	2975.353	Schwarz criterion	6.522945	
Log likelihood	-298.4552	Hannan-Quinn criter.	6.442844	
F-statistic	2.594618	Durbin-Watson stat	1.994777	
Prob(F-statistic)	0.041624			

Gambar 102. Tabel Hasil Model ARIMA dengan Parameter Order 2,1,1

Berdasarkan *output* tabel pada Gambar 99 sampai 102 terlihat bahwa penghitungan model AR, MA, ARMA, dan ARIMA memberikan hasil yang mendekati pada nilai koefisien estimasi parameter model dan sebagian komponen kriteria kebaikan model namun cukup berbeda pada nilai *standard error* koefisien estimasi yang mempengaruhi perbedaan nilai pada nilai *R-Squared* dan *R-Squared Adjusted*, *F-Statistics*, *Probability of F-Statistics*, *Mean* hingga *Standard Deviation Dependent Variable*. Perbedaan nilai-nilai tersebut dikarenakan beberapa aspek. Pertama, Eviews menghitung estimasi parameter model menggunakan metode *maximum likelihood estimation* sedangkan Statify menghitungnya menggunakan metode *conditional least square* dengan fungsi *conditional sum of square*. Peneliti menggunakan metode tersebut pada Statify karena metode tersebut memiliki dokumentasi rumus yang lengkap dibandingkan metode lainnya. Kedua, Statify menghitung nilai *standard error* dan koefisien estimasi menggunakan instrumen yang lebih sederhana, yaitu matriks *Information Fisher* dan metode optimasi *Limited-memory Broyden-Fletcher-Goldfarb-Shanno* (L-BFGS). Peneliti menggunakan instrumen penghitungan tersebut karena kompleksitas pemrograman dan keterbatasan *package* di lingkungan pemrograman Rust.

5.2.2 Usability Testing

Berdasarkan pembahasan di bab sebelumnya tahapan *usability testing* akan dilakukan menggunakan kuesioner SUMI sebagai instrumen evaluasi Statify Modul

Analisis *Time Series*. Adapun peneliti melakukan beberapa tahapan berikut dalam melakukan *usability testing*.

1. Analisis Konteks

Pada tahapan ini, peneliti menentukan target pengguna dengan kriteria mahasiswa Politeknik Statistika STIS yang memperoleh kesempatan belajar Mata Kuliah Analisis Runtun Waktu untuk melakukan uji coba Statify Modul Analisis *Time Series*. Adapun jumlah pengguna yang diperlukan untuk menguji menggunakan kuesioner SUMI minimal berjumlah dua belas orang (Kirakowski, 1994). Selanjutnya peneliti mempersiapkan persyaratan yang diperlukan untuk memperoleh lisensi kuesioner SUMI. Selain itu, peneliti juga membuat panduan penggunaan Statify Modul Analisis *Time Series*.

2. Pengukuran Kepuasan Pengguna

Pada tahapan ini peneliti memilih dua belas orang pengguna dengan kriteria yang telah ditentukan sebelumnya. Selanjutnya pengguna terpilih diminta untuk melakukan uji coba penggunaan dengan mengikuti tahapan-tahapan yang terdapat pada panduan pengguna. Setelah selesai melakukan uji coba, pengguna diminta untuk mengevaluasi Statify Modul Analisis *Time Series* dengan mengisi kuesioner SUMI. Terakhir, peneliti mengumpulkan seluruh jawaban yang diperoleh dari pengguna ke *website* resmi SUMI lalu peneliti membuat email permintaan laporan hasil ke email pemilik resmi SUMI.

3. Analisis Hasil

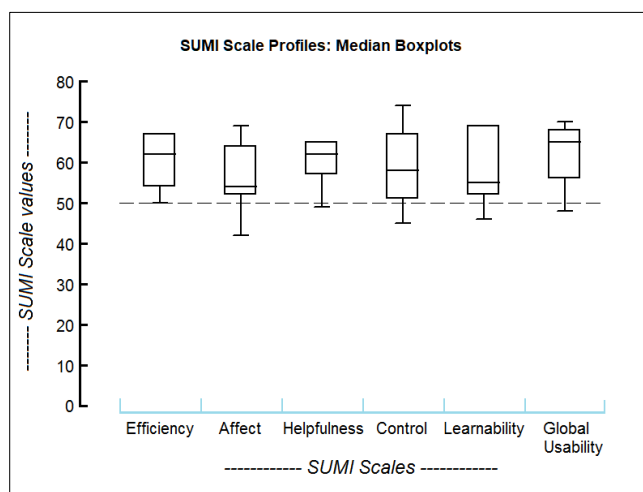
Pada tahapan ini peneliti menganalisis laporan SUMI yang diperoleh dari pemilik resmi lisensi SUMI. Berikut ditampilkan pada Gambar 103 sampai 107 laporan SUMI penggunaan Statify Modul Analisis *Time Series*.

	Mean	St Dev	Median	IQR	Minimum	Maximum
Global	62.08	7.29	65.0	12.0	48	70
Efficiency	60.58	6.72	61.5	12.5	50	67
Affect	57.08	8.32	54.5	11.5	42	69
Helpfulness	59.92	5.76	62.0	8.0	49	65
Controlability	59.17	9.77	58.5	16.0	45	74
Learnability	58.17	8.91	55.0	17.5	46	69

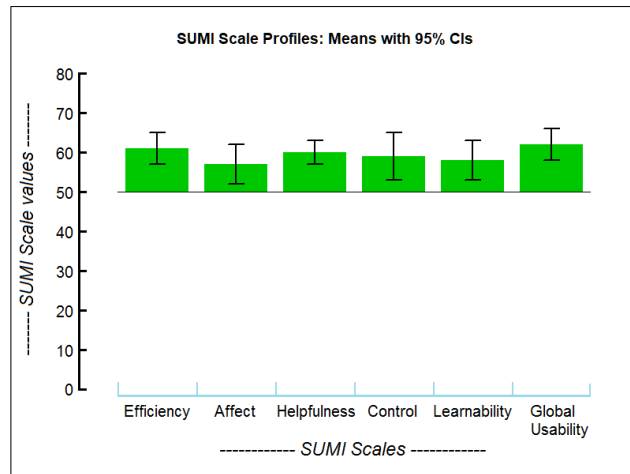
Gambar 103. *Summary Statistics SUMI*

Participant	Global	Efficiency	Affect	Helpfulness	Control	Learnability
1	70	67	69	65	69	69
2	70	63	69	65	60	55
3	68	67	53	62	74	69
4	67	67	49	62	74	55
5	67	67	54	63	65	55
6	65	60	55	65	56	47
7	65	53	64	65	53	53
8	60	59	61	59	58	50
9	57	67	52	55	49	69
10	54	50	64	50	59	69
11	54	56	42	59	45	46
12	48	51	53	49	48	61

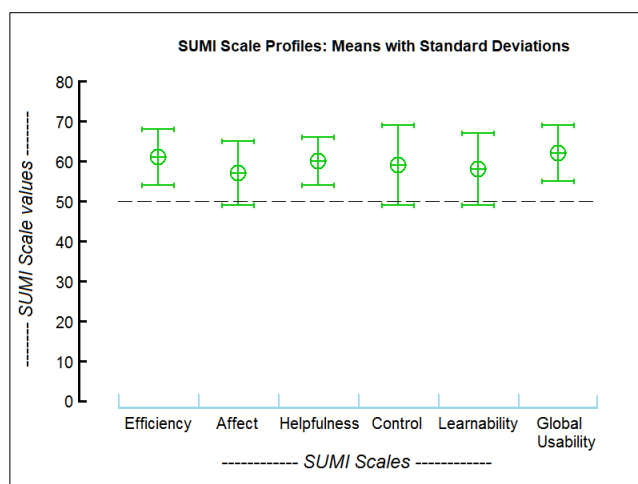
Gambar 104. *Record User SUMI*



Gambar 105. *Median Box-Plot*



Gambar 106. Means with 95% CIs



Gambar 107. Means with Standard Deviations

Berdasarkan laporan evaluasi pada Gambar 103, diketahui bahwa setiap aspek kuesioner SUMI menunjukkan nilai rata-rata di atas 50, yang mengindikasikan bahwa tingkat usability Statify Modul Analisis Time Series pada Statify tergolong baik. Secara spesifik berdasarkan hasil interpretasi Tabel 1, aspek *Global* dan *Efficiency* memperoleh nilai dengan predikat *grade A*, sementara aspek *Affect*, *Helpfulness*, *Controlability*, dan

Learnability masing-masing memperoleh nilai dengan predikat *grade B*. Temuan ini diperkuat oleh grafik *Mean with 95% Confidence Interval (CI)* pada Gambar 106, menunjukkan bahwa batas bawah CI berada di atas nilai ambang 50. Meskipun demikian, aspek *Affect*, *Helpfulness*, *Controlability*, dan *Learnability* tetap perlu mendapatkan perhatian lebih untuk peningkatan usabilitas ke depannya.

“...sengaja dikosongkan...”

BAB VI

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Berdasarkan hasil dan pembahasan pada bagian sebelumnya, diperoleh kesimpulan sebagai berikut:

1. Telah dikembangkan Statify Modul Analisis *Time Series* dengan tampilan antarmuka yang efisien, membantu pengguna, serta mudah digunakan dan dipelajari dibuktikan pada *usability testing* kuesioner SUMI pada aspek *Efficiency*, berpredikat A serta pada aspek *Helpfulness*, *Controlability*, dan *Learnability* masing-masing berpredikat B, yang menunjukkan bahwa kedua aspek tersebut sudah cukup baik. Statify dibangun untuk digunakan secara gratis di lingkungan Politeknik Statistika STIS serta didesain dalam bentuk aplikasi berbasis web sehingga mempermudah akses pengguna tanpa perlu proses instalasi di awal dan hanya memerlukan akses internet pada perangkat untuk mengakses Statify.
2. Telah dikembangkan setiap fitur Statify Modul Analisis *Time Series* dalam bentuk menu *smoothing*, *decomposition*, *autocorrelation*, *unit root test*, dan *box-jenkins model* dengan proses penghitungan yang benar yang dibuktikan dengan hasil uji *white-box testing* dengan keterangan sukses pada setiap *test case*. Lalu dilanjutkan hasil statistik yang valid yang dibuktikan dengan membandingkan *output* antara Statify dan Eviews dengan hasil sebagian besar *output* bernilai sama serta beberapa *output* nilainya sedikit berbeda.

Hal ini disebabkan oleh perbedaan sumber metode penghitungan, proses inisiasi nilai, dan proses optimasi.

3. Telah berhasil mengimplementasikan lima Sub-CPMK mata kuliah Analisis Runtun Waktu yakni pertemuan 2 sampai 6 dalam bentuk menu *smoothing*, *decomposition*, *autocorrelation*, *unit root test*, dan *box-jenkins model*.

6.2 Saran

Berikut beberapa saran dari peneliti untuk penyempurnaan dan pengembangan Statify Modul Analisis *Time Series* ke depannya.

1. Berdasarkan evaluasi validasi *output* menu *time series* antara Statify dan Eviews diperoleh beberapa perbedaan hasil karena metode penghitungan yang digunakan didominasi metode penghitungan klasik. Dengan demikian, peneliti selanjutnya dapat mengembangkan metode penghitungan ke metode penghitungan yang lebih *advanced* untuk memperoleh *output* statistik yang lebih baik.
2. Berdasarkan evaluasi *usability testing* menggunakan kuesioner SUMI diperoleh hasil bahwa aspek *Affect* dan *Learnability* merupakan dua aspek dengan nilai terendah dibandingkan aspek SUMI lainnya. Dengan demikian, peneliti selanjutnya dapat meningkatkan Statify Modul Analisis *Time Series* pada aspek *Affect* dan *Learnability* dengan menambahkan fitur arahan otomatis yang memberi petunjuk awal penggunaan menu yang interaktif untuk mempermudah pemahaman pengguna baru.
3. Berdasarkan batasan penelitian dan implementasi diperoleh hasil bahwa pengembangan Statify Modul Analisis *Time Series* masih terbatas pada Sub-

CPMK mata kuliah Analisis Runtun Waktu yang dikembangkan masih terbatas pada pertemuan 2 sampai 6. Dengan demikian, peneliti selanjutnya dapat menambahkan fitur *time series* yang mengakomodasi materi mata kuliah Analisis Runtun Waktu Sub-CPMK pertemuan 7 hingga 14 untuk menjadikan Statify Modul Analisis *Time Series* sebagai aplikasi statistik yang mampu dalam mendukung pembelajaran Analisis Runtun Waktu secara komprehensif.

“...sengaja dikosongkan...”

DAFTAR PUSTAKA

- Al-Hakim, R. R., Yanuardi, Y., Rumandan, R. J., & Tonggiroh, M. (2024). Pengembangan Sistem Informasi Bimbingan Konseling Menggunakan Metode Feature-Driven Development. *Jurnal Ilmiah FIFO*, 16(2), 193. <https://doi.org/10.22441/fifo.2024.v16i2.009>
- Ammann, P., & Offutt, J. (2013). *Introduction to software testing* (1st ed.). Cambridge University Press.
- Asari, A., Zulkarnaini, Hartatik, Anam, A. C., Suparto, Litamahuputty, J. V., Dewadi, F. D., Prihastuty, D. R., Maswar, Syukrilla, W. A., Murni, N. S., & Sukwika, T. (2023). *Pengantar Statistika* (1st ed.). PT Mafy Media Literasi Indonesia.
- Box, G. E. P., Jenkins, G. M., & Reinsel, G. C. (1994). *Time Series Analysis Forecasting and Control* (3rd ed.). Prentice-Hall, Inc.
- Brockwell, P. J., & Davis, R. A. (1996). *Introduction to Time Series and Forecasting* (1st ed.). Springer Science. <https://doi.org/10.1007/978-1-4757-2526-1>
- Crycer, J. D., & Chan, K.-S. (2008). *Time Series Analysis With Applications in R* (2nd ed.). Springer Science.
- Enders, W. (2014). *Applied Econometric Time Series* (4th ed.). John Wiley & Sons.
- Fowler, M. (2003). *UML Distilled: A Brief Guide to The Standard Object Modeling Language* (3rd ed.). Pearson Education, Inc.
- Gani, I., & Amlia, S. (2018). *Alat Analisis Data: Aplikasi Statistik untuk Penelitian Bidang Ekonomi dan Sosial* (1st ed.). ANDI.

- Gurung, G., Shah, R., & Jaiswal, D. P. (2020). Software Development Life Cycle Models-A Comparative Study. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, 30–37. <https://doi.org/10.32628/cseit206410>
- Hanke, J. E., & Reitsch, A. G. (1995). *Business Forecasting* (5th ed.). Prentice-Hall.
- Irwansyah, E., & Moniaga, J. V. (2014). *Pengantar Teknologi Informasi* (1st ed.). Deepublish.
- Jubhari Phie Joarno, R., Fajar, M., Yunus, A., Informasi, S., & Kharisma Makassar, S. (2022). Implementasi Progressive Web Apps pada Website GetHelp Menggunakan Next.js. *KHARISMA Tech*, 17, 1–15. <https://jurnal.kharisma.ac.id/kharismatech/>
- Kim, J., Kim, R., Oh, J., & Lee, S. E. (2024). Hardware-Based WebAssembly Accelerator for Embedded System. *Electronics (Switzerland)*, 13(20). <https://doi.org/10.3390/electronics13203979>
- Kirakowski, J. (1994). The Use of Questionnaire Methods for Usability Assessment. *Measures & Methods for Quality of Use*.
- Kirakowski, J. (2021). *SUMI Standardisation*.
- Mackinnon, J. G. (1994). Approximate asymptotic distribution functions for unit-root and cointegration tests. *Journal of Business and Economic Statistics*, 12(2), 167–176. <https://doi.org/10.1080/07350015.1994.10510005>
- Makridakis, S. G., Wheelwright, S. C., & Hyndman, R. J. (1997). *Forecasting: Methods and applications*. John Wiley and Sons.

- Nidhra, S. (2012). Black Box and White Box Testing Techniques - A Literature Review. *International Journal of Embedded Systems and Applications*, 2(2), 29–50. <https://doi.org/10.5121/ijesa.2012.2204>
- Nurhasanah, N., Nurpraja, C. A., Himawan, S., & Ramadanti, A. (2022). *Metode Peramalan Deret Waktu* (1st ed.). KBM Indonesia.
- Okazaki, N. (2014, September 2). *LibLBFGS: a Library of Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS)*.
- Paisal, Satyahadewi, N., & Perdana, H. (2021). Pengembangan Aplikasi Statistika Berbasis Web Interaktif untuk Analisis Uji-T. *Bimaster*, 10, 331–340.
- Palmer, S. R., & Felsing, J. M. (2022). *A Practical Guide to Feature-Driven Development* (1st ed.). Prentice-Hall. www.togethersoft.com
- Peraturan Badan Pusat Statistik Nomor 87 Tahun 2018, Badan Pusat Statistik (2018).
https://ppid.bps.go.id/upload/doc/Peraturan_Badan_Pusat_Statistik_Nomor_87_Tahun_2018_1679371754.pdf
- Pratomo, A., & Mantala, R. (2016). Pengembangan Aplikasi Ujian Berbasis Komputer Beserta Analisis Uji guna Sistem Perangkat Lunaknya Menggunakan Metode SUMI (Software Usability Measurement Inventory). *POSITIF*, 2(1), 1–11.
- Rust Team. (2025). *Learn Rust*. Rust Team.
- Sie, J. B. L., Musdar, I. A., & Bahri, S. (2022). Pengujian White-Box Testing Terhadap Website Room Menggunakan Teknik Basis Path. *KHARISMA Tech*, 17, 45–57.

- Sriwahyuni, A. A., Annas, S., & Saleh Ahmar, A. (2018). Pengembangan Paket Analisis Regresi Berbasis Web. *Seminar Nasional Variansi*. <http://ojs.unm.ac.id/index.php/variantsistatistika>
- TypeScript Community. (2025, July 26). *TypeScript for JavaScript Programmers*. Microsoft.
- Vercel. (2025). *Next.js Docs*. Vercel, Inc.
- WebAssembly Community. (2025). *WebAssembly*. WebAssembly.
- Wei, W. W. S. (2006). *Time Series Analysis Univariate and Multivariate Methods* (2nd ed.). Addison Wesley.

LAMPIRAN

Lampiran 1. RPS Mata Kuliah Analisis Runtun Waktu Polstat STIS T.A. 2024/2025

RPS Mata Kuliah Analisis Runtun Waktu Politeknik Statistika STIS Tahun Ajaran 2024/2025 dapat diakses pada tautan berikut.

https://drive.google.com/file/d/1KqmywYXZZ9U89wkj6aEvd4r4OFeAEL7F/view?usp=drive_link

Lampiran 2. Alamat Akses Statify

Aplikasi Statify dapat diakses pada tautan berikut.

<https://statify-dev.student.stis.ac.id/>

Lampiran 3. Source Code Statify Modul Analisis *Time Series*

Source Code Statify Modul Analisis *Time Series* dapat diakses pada tautan berikut.

<https://git.stis.ac.id/222112082/pengembangan-statify-aplikasi-analisis-statistik-modul-analisis-time-series>

Lampiran 4. Dokumentasi Panduan Penggunaan Statify Modul Analisis *Time Series*

Dokumentasi panduan penggunaan dapat diakses pada tautan berikut.

<https://drive.google.com/file/d/1RkrwpeQQqO3YDJdSKxtcos2fdMax49ML/view?usp=sharing>

Lampiran 5. Dokumentasi Algoritma Rumus Statify Modul Analisis *Time Series*

Dokumentasi algoritma rumus dapat diakses pada tautan berikut.

https://drive.google.com/file/d/1KGKAv_QS7ln2mFK5v3800vyzi9ypKzft/view?usp=sharing

Lampiran 6. Data *Testing Perbandingan Output* antara Statify dan Eviews

Data penumpang kereta api bulanan tahun 2017 hingga 2024 yang diperoleh dari *website* resmi BPS yang dapat dilihat di sini atau diakses pada tautan berikut.

<https://www.bps.go.id/id/statistics-table/2/MjM1MyMy/penumpang-angkutan-kereta-api-nasional-bulanan-.html>.

No.	Penumpang (000)	TJTP (000.000 km)	RJP (km)
1	30949	2096	68
2	27342	1718	63
3	32170	1979	62
4	31502	2044	65
5	33745	2112	63
6	30723	2068	67
7	34310	2433	71
8	33791	2164	64
9	32498	2161	67
10	35070	2187	62
11	34361	2137	62
12	36807	2555	69
13	34741	2238	64
14	31278	1951	62
15	35875	2199	61
16	35754	2303	64
17	35482	2177	61
18	33029	2539	77
19	36799	2540	69
20	35190	2393	68
21	34504	2277	66
22	36236	2297	63

23	35298	2330	66
24	37965	2768	73
25	35122	2426	69
26	31899	2114	66
27	35751	2347	66
28	35809	2416	67
29	35102	2190	62
30	35090	2800	80
31	39035	2649	70
32	35189	2460	70
33	35221	2277	65
34	36448	2336	64
35	35877	2341	65
36	37463	2676	71
37	34130	2289	67
38	32283	2066	64
39	23425	1515	65
40	5899	279	47
41	5484	197	36
42	9290	343	37
43	12238	480	39
44	12773	614	48
45	11429	556	49
46	11937	619	52
47	13722	751	55
48	13515	779	58
49	11901	613	52
50	11478	579	50
51	14284	766	54
52	14864	828	56
53	14857	863	58
54	14556	918	63
55	5747	314	55
56	6515	358	55
57	9566	523	55
58	13250	807	61
59	15317	982	64
60	17428	1148	65.87
61	17798	1126	63
62	13103	838	64
63	19505	1268	65
64	19868	1314	66
65	23811	1888	79
66	23811	1637	69

67	25964	1992	77
68	24520	1671	68
69	25809	1654	64
70	27879	1827	66
71	27201	1741	64
72	30153	2095	69
73	29349	1900	65
74	26572	1684	63
75	30169	1838	61
76	28705	2066	72
77	30972	2074	67
78	30655	2112	69
79	33403	2397	72
80	31757	2095	66
81	32132	2087	65
82	34208	2209	65
83	33549	2158	64
84	35757	2537	71
85	39861	2234	56
86	37166	2123	57
87	38335	2005	52
88	40458	2697	67
89	41702	2359	57
90	41888	2445	58
91	45603	2592	57
92	43230	2346	54
93	42481	2302	54
94	45499	2357	52
95	42590	2254	53
96	45829	2631	57

Lampiran 7. Kode Program *White-Box Testing Menu Smoothing*

```
1  /** @jest-environment jsdom */
2
3  import '@testing-library/jest-dom';
4  import { renderHook, act } from '@testing-library/react';
5  import { useAnalyzeHook } from '@components/Modals/Analyze/TimeSeries/Smoothing/hooks/analyzeHook';
6  import { Variable } from '@types/Variable';
7
8  const createMockVar = (type: "NUMERIC" | "STRING" = "NUMERIC"): Variable => ({
9    id: 1,
10    name: "Test Var",
11    columnIndex: 0,
12    type,
13    width: 8,
14    decimals: 2,
15    label: "",
16    values: [],
17    missing: null,
18    columns: 10,
19    align: "right",
20    measure: "scale",
21    role: "input",
22  });
23
24  const periodNotDated = ["0", "Not Dated"];
25  const periodMonthly = ["12", "Years-Months"];
26  const dataEmpty: any[][] = [];
27  const dataBelow20Obs = Array.from({ length: 5 }, () => [3]);
28  const data25Obs = Array.from({ length: 25 }, () => [3]);
29
30  const runAnalyzeError = async (selectedMethod: string[], parameters: number[], period: string[],
31    storeVariables: Variable[], data: any[][] => {
32    const { result } = renderHook(() => {
33      useAnalyzeHook(
34        selectedMethod,
35        parameters,
36        period,
37        storeVariables,
38        data,
39        false,
40        jest.fn()
41      )
42    });
43    await act(async () => {
44      await result.current.handleAnalyzes();
45    });
46    return result;
47  });
48
49  describe("UseAnalyzeHook - Error Cases", () => {
50    it("TCS01: Returns error if selected variable is empty", async () => {
51      const result = await runAnalyzeError(["sma"], [3], periodMonthly, [], dataEmpty);
52      expect(result.current.errorMsg).toBe('Please select at least one variable.');
```

```

48 describe("UseAnalyzeHook - Error Cases", () => {
49   it("TCS10: Returns an error value if double moving average is used without configuring the time settings", async () => {
50     const result = await runAnalyzeError(["dma"], [3], periodNotDated, [createMockVar()], data250bs);
51     expect(result.current.errorMsg).toBe('Please select another time specification.');
```

```

92   });
93   it("TCS12: Returns an error value if the simple exponential smoothing parameter is out of range.", async () => {
94     const result = await runAnalyzeError(["ses"], [1.5], periodMonthly, [createMockVar()], data250bs);
95     expect(result.current.errorMsg).toBe('Simple Exponential Smoothing alpha parameter must be between 0 and 1.');
```

```

97   });
98   it("TCS13: Returns an error value if simple exponential smoothing is used without configuring the time settings", async () => {
99     const result = await runAnalyzeError(["ses"], [0.5], periodNotDated, [createMockVar()], data250bs);
100    expect(result.current.errorMsg).toBe('Please select another time specification.');
```

```

102   });
103   it("TCS15: Returns an error value if the double exponential smoothing parameter is out of range.", async () => {
104     const result = await runAnalyzeError(["des"], [1.5], periodMonthly, [createMockVar()], data250bs);
105     expect(result.current.errorMsg).toBe('Double Exponential Smoothing alpha parameter must be between 0 and 1.');
```

```

107   });
108   it("TCS16: Returns an error value if double exponential smoothing is used without configuring the time settings", async () => {
109     const result = await runAnalyzeError(["des"], [0.5], periodNotDated, [createMockVar()], data250bs);
110     expect(result.current.errorMsg).toBe('Please select another time specification.');
```

```

112   });
113   it("TCS18: Returns an error value if Holt's method alpha parameter is out of range.", async () => {
114     const result = await runAnalyzeError(["holt"], [1.5, 0.5], periodMonthly, [createMockVar()], data250bs);
115     expect(result.current.errorMsg).toBe('Holt's Method alpha parameter must be between 0 and 1.');
```

```

49 describe("UseAnalyzeHook - Error Cases", () => {
120   it("TCS19: Returns an error value if Holt's method beta parameter is out of range.", async () => {
121     const result = await runAnalyzeError(["holt"], [0.5, 1.5], periodMonthly, [createMockVar()], data250bs);
122     expect(result.current.errorMsg).toBe('Holt's Method beta parameter must be between 0 and 1.');
```

```

123   });
124   it("TCS20: Returns an error value if Holt's method is used without configuring the time settings", async () => {
125     const result = await runAnalyzeError(["holt"], [0.5, 0.5], periodNotDated, [createMockVar()], data250bs);
126     expect(result.current.errorMsg).toBe('Please select another time specification.');
```

```

127   });
128   it("TCS22: Returns an error value if Winter's method alpha parameter is out of range.", async () => {
129     const result = await runAnalyzeError(["winter"], [1.5, 0.5, 0.5], periodMonthly, [createMockVar()], data250bs);
130     expect(result.current.errorMsg).toBe('Winter's Method alpha parameter must be between 0 and 1.');
```

```

131   });
132   it("TCS23: Returns an error value if Winter's method beta parameter is out of range.", async () => {
133     const result = await runAnalyzeError(["winter"], [0.5, 1.5, 0.5], periodMonthly, [createMockVar()], data250bs);
134     expect(result.current.errorMsg).toBe('Winter's Method beta parameter must be between 0 and 1.');
```

```

135   });
136   it("TCS24: Returns an error value if Winter's method gamma parameter is out of range.", async () => {
137     const result = await runAnalyzeError(["winter"], [0.5, 0.5, 1.5], periodMonthly, [createMockVar()], data250bs);
138     expect(result.current.errorMsg).toBe('Winter's Method gamma parameter must be between 0 and 1.');
```

```

139   });
140   it("TCS25: Returns an error value if Winter's method is used without configuring the time settings", async () => {
141     const result = await runAnalyzeError(["winter"], [0.5, 0.5, 0.5], periodNotDated, [createMockVar()], data250bs);
142     expect(result.current.errorMsg).toBe('Please select another time specification.');
```

```

151 const mockAddStatistic = jest.fn();
152 const mockHandleSmoothing = jest.fn();
153
154 jest.mock('@stores/useResultStore', () => ({
155   useResultStore: () => ({
156     addLog: jest.fn(() => Promise.resolve(1)),
157     addAnalytic: jest.fn(() => Promise.resolve(1)),
158     addStatistic: mockAddStatistic,
159   }),
160 }));
161
162 jest.mock('@components/Modals/Analyze/TimeSeries/Smoothing/analyze/analyze', () => ({
163   handleSmoothing: (...args: any[]) => mockHandleSmoothing(...args),
164 }));
165
166 const runAnalyzeSuccess = async (
167   method: string[],
168   parameters: number[],
169   selectedPeriod: string[],
170   variables: Variable[],
171   data: any[][])
172 => {
173   const { result } = renderHook(() =>
174     useAnalyzeHook(method, parameters, selectedPeriod, variables, data, false, jest.fn())
175   );
176   await act(async () => {
177     await result.current.handleAnalyzes();
178   });
179   return result;

```

```

182 const assertStatistic = (methodCode: string, seriesLength: string = "25") => {
183   const call = mockAddStatistic.mock.calls.find(
184     ([, stat]) => stat?.title === 'Description Table'
185   );
186   expect(call).toBeDefined();
187   expect(call[1].components).toBe('Description Table');
188   expect(call[1].output_data.tables[0].rows).toEqual(
189     expect.arrayContaining([
190       expect.objectContaining({ rowHeader: ['Smoothing Method'], description: methodCode }),
191       expect.objectContaining({ rowHeader: ['Series Length'], description: seriesLength }),
192     ])
193   );
194 };
195
196 beforeAll(() => {
197   mockHandleSmoothing.mockImplementation(
198     Promise.resolve([
199       'success',
200       {
201         tables: [
202           {
203             title: 'Description Table',
204             columnHeaders: [{ header: '' }, { header: 'description' }],
205             rows: [
206               { rowHeader: ['Smoothing Method'], description: method },
207               { rowHeader: ['Parameter'], description: parameters.join(', ') },
208               { rowHeader: ['Series Name'], description: name },
209               { rowHeader: ['Series Period'], description: '2024-01-01 - 2024-01-25' },
210               { rowHeader: ['Periodicity'], description: 'None' },

```

```

frontend > components > Modals > Analyze > TimeSeries > Smoothing > test > 18 smoothing.tests > 20 assertStatistic
196 beforeAll(() => {
197   mockHandleSmoothing.mockImplementation(
201     tables: [
205       rows: [
211         { rowHeader: ['Series Length'], description: '25' },
212         { rowHeader: ['Smoothing Length'], description: '22' },
213       ],
214     ],
215   ),
216   [1, 2, 3, 4, 5],
217   { graphic: true },
218   { evaluation: true },
219 ];
220 );
221 );
222 });
223
224 describe('useAnalyzeHook - Success Cases', () => {
225   beforeEach(() => {
226     mockAddStatistic.mockClear();
227   });
228
229   const cases = [
230     { code: 'sma', name: 'Simple Moving Average', params: [3], id: 'TCS07' },
231     { code: 'dma', name: 'Double Moving Average', params: [3], id: 'TCS11' },
232     { code: 'ses', name: 'Simple Exponential Smoothing', params: [0.5], id: 'TCS14' },
233     { code: 'des', name: 'Double Exponential Smoothing', params: [0.5], id: 'TCS17' },
234     { code: 'holt', name: 'Holt's Exponential Smoothing', params: [0.5, 0.5], id: 'TCS21' },
235     { code: 'winter', name: 'Winter's Exponential Smoothing', params: [0.5, 0.5, 0.5], id: 'TCS26' },

```

```

237   cases.forEach(({ code, name, params, id }) => {
238     it(`${id}: Returns statistical output if all requirements for ${name.toLowerCase()} are met`, async () => {
239       const result = await runAnalyzeSuccess([code, name], params, periodMonthly, [createMockVar()], data250Obs);
240       expect(result.current.errorMsg).toBe(null);
241       assertStatistic(code);
242     });
243   });
244 });
245 });

```


Lampiran 8. Hasil *White-Box Testing Menu Smoothing*

```
PS C:\Users\guswa\Documents\Skripsi\3.2. statify - GitLab Stis> npm test -- smoothing.test.ts

> statify-monorepo@1.0.0 test
> npm test --workspaces smoothing.test.ts

> statify-frontend@0.1.0 test
> jest --logHeapUsage smoothing.test.ts

⚠ Mismatching @next/swc version, detected: 15.3.5 while Next.js is on 15.3.4. Please ensure these match
console.log
  { graphic: true }

    at log (components/Modals/Analyze/TimeSeries/Smoothing/hooks/analyzeHook.tsx:208:17)

console.log
  { graphic: true }

    at log (components/Modals/Analyze/TimeSeries/Smoothing/hooks/analyzeHook.tsx:208:17)

console.log
  { graphic: true }

    at log (components/Modals/Analyze/TimeSeries/Smoothing/hooks/analyzeHook.tsx:208:17)

console.log
  { graphic: true }

    at log (components/Modals/Analyze/TimeSeries/Smoothing/hooks/analyzeHook.tsx:208:17)
```

```
console.log
  { graphic: true }

    at log (components/Modals/Analyze/TimeSeries/Smoothing/hooks/analyzeHook.tsx:208:17)

console.log
  { graphic: true }

    at log (components/Modals/Analyze/TimeSeries/Smoothing/hooks/analyzeHook.tsx:208:17)
```

```
PASS components/Modals/Analyze/TimeSeries/Smoothing/test/smoothing.test.ts (115 MB heap size)
useAnalyzeHook - Error Cases
  ✓ TCS01: Returns error if selected variable is empty (36 ms)
  ✓ TCS02: Returns error if selected variable is not numeric (17 ms)
  ✓ TCS03: Returns error if selected variable data is empty (6 ms)
  ✓ TCS04: Returns error if data observations are less than 20 (7 ms)
  ✓ TCS05: Returns an error value if the simple moving average parameter is out of range. (5 ms)
  ✓ TCS06: Returns an error value if simple moving average is used without configuring the time settings (7 ms)
  ✓ TCS08: Returns an error value if the double moving average parameter is out of range. (4 ms)
  ✓ TCS09: Returns error if observations are less than 3 × double moving average parameter (4 ms)
  ✓ TCS10: Returns an error value if double moving average is used without configuring the time settings (6 ms)
  ✓ TCS12: Returns an error value if the simple exponential smoothing parameter is out of range. (5 ms)
  ✓ TCS13: Returns an error value if simple exponential smoothing is used without configuring the time settings (7 ms)
  ✓ TCS15: Returns an error value if the double exponential smoothing parameter is out of range. (5 ms)
  ✓ TCS16: Returns an error value if double exponential smoothing is used without configuring the time settings (9 ms)
  ✓ TCS18: Returns an error value if Holt's method alpha parameter is out of range. (5 ms)
  ✓ TCS19: Returns an error value if Holt's method beta parameter is out of range. (2 ms)
  ✓ TCS20: Returns an error value if Holt's method is used without configuring the time settings (6 ms)
  ✓ TCS22: Returns an error value if Winter's method alpha parameter is out of range. (3 ms)
  ✓ TCS23: Returns an error value if Winter's method beta parameter is out of range. (3 ms)
  ✓ TCS24: Returns an error value if Winter's method gamma parameter is out of range. (3 ms)
  ✓ TCS25: Returns an error value if Winter's method is used without configuring the time settings (8 ms)
useAnalyzeHook - Success Cases
  ✓ TCS07: Returns statistical output if all requirements for simple moving average are met (77 ms)
  ✓ TCS11: Returns statistical output if all requirements for double moving average are met (11 ms)
  ✓ TCS14: Returns statistical output if all requirements for simple exponential smoothing are met (7 ms)
  ✓ TCS17: Returns statistical output if all requirements for double exponential smoothing are met (8 ms)
  ✓ TCS21: Returns statistical output if all requirements for holt's exponential smoothing are met (7 ms)
  ✓ TCS26: Returns statistical output if all requirements for winter's exponential smoothing are met (9 ms)
```

```
Test Suites: 1 passed, 1 total
Tests:       26 passed, 26 total
Snapshots:   0 total
Time:        5.224 s
Ran all test suites matching smoothing.test.ts.

> statify-backend@1.0.0 test
> jest --passWithNoTests smoothing.test.ts
```


Lampiran 9. Kode Program *White-Box Testing Menu Decomposition*

```
1  /** @jest-environment jsdom */
2
3  import '@testing-library/jest-dom';
4  import { renderHook, act } from '@testing-library/react';
5  import { useAnalyzeHook } from '@components/Modals/Analyze/TimeSeries/Decomposition/hooks/analyzeHook';
6  import { Variable } from '@types/Variable';
7
8  const createMockVar = (type: "NUMERIC" | "STRING": Variable => ({
9    id: 1,
10    name: "Test Var",
11    columnIndex: 0,
12    type,
13    width: 8,
14    decimals: 2,
15    label: "",
16    values: [],
17    missing: null,
18    columns: 10,
19    align: "right",
20    measure: "scale",
21    role: "input",
22  });
23
24  const periodNotDated = ["0", "Not Dated"];
25  const periodMonthly = ["12", "Years-Months"];
26  const periodYearly = ["0", "Years"];
27  const periodWorkingDay = ["5", "Weeks-Work Days(5)"];
28
29  const dataEmpty: any[][] = [];
30  const dataBelow200Obs = Array.from({ length: 5 }, () => [3]);
31  const data250Obs = Array.from({ length: 25 }, () => [3]);
32  const data490Obs = Array.from({ length: 49 }, () => [3]);
33
34  const runAnalyzeError = async (decomposition: string[], trended: string[], period: string[],
35    storeVariables: Variable[], data: any[][] => {
36    const { result } = renderHook(() => {
37      useAnalyzeHook(
38        decomposition,
39        trended,
40        period,
41        storeVariables,
42        data,
43        false,
44        jest.fn()
45      );
46    });
47    await act(async () => {
48      await result.current.handleAnalyzes();
49    });
50    return result;
51  });
52
53  describe("useAnalyzeHook - Error Cases", () => {
54    it("TCD01: Returns error if selected variable is empty", async () => {
55      const result = await runAnalyzeError(["additive"], ["linear"], periodMonthly, [], dataEmpty);
56      expect(result.current.errorMsg).toBe('Please select at least one variable.');
```

```
57  });
58
59  describe("useAnalyzeHook - Error Cases", () => {
60    it("TCD02: Returns error if selected variable is not numeric", async () => {
61      const result = await runAnalyzeError(["additive"], ["linear"], periodMonthly, [createMockVar("STRING")], [dataBelow200Obs]);
62      expect(result.current.errorMsg).toBe('Selected variable is not numeric.');
```

```
63  });
64    it("TCD03: Returns error if selected variable data is empty", async () => {
65      const result = await runAnalyzeError(["additive"], ["linear"], periodMonthly, [createMockVar()], dataEmpty);
66      expect(result.current.errorMsg).toBe('No data available for the selected variable.');
```

```
67  });
68    it("TCD04: Returns an error value if running without configuring the time settings", async () => {
69      const result = await runAnalyzeError(["additive"], ["linear"], periodNotDated, [createMockVar()], data250Obs);
70      expect(result.current.errorMsg).toBe('Please select another time specification.');
```

```
71  });
72    it("TCD05: Returns an error value if time specification don't have periodicity", async () => {
73      const result = await runAnalyzeError(["additive"], ["linear"], periodYearly, [createMockVar()], data250Obs);
74      expect(result.current.errorMsg).toBe('Selected time specification doesn't have periodicity.');
```

```
75  });
76    it("TCD06: Returns an error if the number of data observations is less than four times the periodicity", async () => {
77      const result = await runAnalyzeError(["additive"], ["linear"], periodMonthly, [createMockVar()], data250Obs);
78      expect(result.current.errorMsg).toBe('Data length is less than 4 times the periodicity.');
```

```
79  });
80    it("TCD07: Returns an error if the number of data observations is not a multiple of the periodicity", async () => {
81      const result = await runAnalyzeError(["additive"], ["linear"], periodWorkingDay, [createMockVar()], data490Obs);
82      expect(result.current.errorMsg).toBe('Data length is not a multiple of the periodicity.');
```

```

90 const mockAddStatistic = jest.fn();
91 const mockHandleDecomposition = jest.fn();
92
93 jest.mock('@stores/useResultStore', () => ({
94   useResultStore: () => ({
95     addLog: jest.fn(() => Promise.resolve(1)),
96     addAnalytic: jest.fn(() => Promise.resolve(1)),
97     addStatistic: mockAddStatistic,
98   }),
99 }));
100
101 jest.mock('@components/Models/Analyze/TimeSeries/Decomposition/analyze/analyze', () => ({
102   handleDecomposition: (...args: any[]) => mockHandleDecomposition(...args),
103 }));
104
105 const runAnalyzeSuccess = async (
106   decomposition: string[],
107   trended: string[],
108   selectedPeriod: string[],
109   variables: Variable[],
110   data: any[][]
111 ) => {
112   const { result } = renderHook(() =>
113     useAnalyzeHook(decomposition, trended, selectedPeriod, variables, data, false, jest.fn())
114   );
115   await act(async () => {
116     await result.current.handleAnalyzes();
117   });
118   return result;

```

```

121 const assertStatistic = (
122   decompositionMethod: string,
123   trendMethod: string,
124   seriesLength = "25",
125   period = "Weeks-Work Days(5)"
126 ) => {
127   const call = mockAddStatistic.mock.calls.find(
128     ([, stat]) => stat?.title === 'Description Table'
129   );
130
131   const seriesName = "Test Var";
132
133   expect(call).toBeDefined();
134   const rows = call[1].output_data.tables[0].rows;
135
136   expect(rows).toEqual(
137     expect.arrayContaining([
138       { rowHeader: ['Decomposition Method'], description: decompositionMethod },
139       { rowHeader: ['Formula of Calculating Decomposition'], description: 'Classical Decomposition' },
140       { rowHeader: ['Trend Method'], description: trendMethod },
141       { rowHeader: ['Series Name'], description: seriesName },
142       { rowHeader: ['Series Period'], description: '2024-01-01 - 2024-01-25' },
143       { rowHeader: ['Periodicity'], description: period },
144       { rowHeader: ['Observations'], description: seriesLength },
145     ])
146   );
147 };
148

```

```

175   ],
176   [], [], [], [], // testing, seasonal, trend, irregular, forecasting, evaluation
177   {}, // evaluation
178   [], // seasonIndices
179   'y = a + bt', // equation
180   { graphic: 'forecast' }, // graphicForecasting
181   { graphic: 'original' }, // graphicData
182   { graphic: 'trend' }, // graphicTrend
183   { graphic: 'seasonal' }, // graphicSeasonal
184   { graphic: 'irregular' }, // graphicIrregular
185   []);
186 });

```

```

188 describe('useAnalyzeHook - Success Cases', () => {
189   beforeEach(() => {
190     mockAddStatistic.mockClear();
191   });
192
193   const cases = [
194     {
195       id: 'TCD08',
196       code: 'additive',
197       name: 'Additive Decomposition',
198       trend: 'none',
199       periodicity: '5',
200       periodicityName: 'Weeks-Work Days(5)',
201       dataLength: 25,
202     },
203     {
204       id: 'TCD09',
205       code: 'multiplicative',
206       name: 'Multiplicative Decomposition',
207       trend: 'linear',
208       periodicity: '5',
209       periodicityName: 'Weeks-Work Days(5)',
210       dataLength: 25,
211     },

```

```

212     {
213       id: 'TCD10',
214       code: 'multiplicative',
215       name: 'Multiplicative Decomposition',
216       trend: 'exponential',
217       periodicity: '5',
218       periodicityName: 'Weeks-Work Days(5)',
219       dataLength: 25,
220     },
221   ];
222
223   cases.forEach(({ id, code, name, trend, periodicity, periodicityName, dataLength }) => {
224     it(`${id}: Returns statistical output if all requirements for ${name.toLowerCase()} are met`, async () => {
225       const result = await runAnalyzeSuccess(
226         [code, name],
227         [trend],
228         [periodicity, periodicityName],
229         [createMockVar()],
230         Array.from({ length: dataLength }, () => [3])
231       );
232
233       expect(result.current.errorMsg).toBe(null);

```

```

235       assertStatistic(
236         code,
237         code === 'additive' ? 'none' : trend,
238         dataLength.toString(),
239         periodicityName
240       );
241     });
242   });
243 });
244

```

Lampiran 10. Hasil White-Box Testing Menu Decomposition

```

PS C:\Users\guswa\Documents\Skrripsi\3.2. statify - GitLab Stis> npm test -- decomposition.test.ts

> statify-monorepo@1.0.0 test
> npm test --workspaces decomposition.test.ts

> statify-frontend@0.1.0 test
> jest --logHeapUsage decomposition.test.ts

```

```

⚠ Mismatching @next/swc version, detected: 15.3.5 while Next.js is on 15.3.4. Please ensure these match
[PASS] components/Modals/Analyze/TimeSeries/Decomposition/test/decomposition.test.ts (114 MB heap size)
useAnalyzeHook - Error Cases
  ✓ TCD01: Returns error if selected variable is empty (20 ms)
  ✓ TCD02: Returns error if selected variable is not numeric (13 ms)
  ✓ TCD03: Returns error if selected variable data is empty (4 ms)
  ✓ TCD04: Returns an error value if running without configuring the time settings (3 ms)
  ✓ TCD05: Returns an error value if time specification don't have periodicity (3 ms)
  ✓ TCD06: Returns an error if the number of data observations is less than four times the periodicity (4 ms)
  ✓ TCD07: Returns an error if the number of data observations is not a multiple of the periodicity (3 ms)
useAnalyzeHook - Success Cases
  ✓ TCD08: Returns statistical output if all requirements for additive decomposition are met (5 ms)
  ✓ TCD09: Returns statistical output if all requirements for multiplicative decomposition are met (4 ms)
  ✓ TCD10: Returns statistical output if all requirements for multiplicative decomposition are met (4 ms)

Test Suites: 1 passed, 1 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        2.791 s, estimated 3 s
Ran all test suites matching decomposition.test.ts.

> statify-backend@1.0.0 test
> jest --passWithNoTests decomposition.test.ts

```

Lampiran 11. Kode Program White-Box Testing Menu Autocorrelation

```
1  /** @jest-environment jsdom */
2  import '@testing-library/jest-dom';
3
4  import { renderHook, act } from '@testing-library/react';
5  import { useAnalyzeHook } from '@components/Models/Analyze/TimeSeries/Autocorrelation/hooks/analyzeHook';
6  import { Variable } from '@types/Variable';
7
8  const createMockVar = (type: "NUMERIC" | "STRING" = "NUMERIC"): Variable => ({
9    id: 1,
10    name: "Test Var",
11    columnIndex: 0,
12    type,
13    width: 8,
14    decimals: 2,
15    label: "",
16    values: [],
17    missing: null,
18    columns: 10,
19    align: "right",
20    measure: "scale",
21    role: "input",
22  });
23
24  const dataEmpty: any[][] = [];
25  const dataBelow20Obs = Array.from({ length: 5 }, () => [3]);
26  const data25Obs = Array.from({ length: 25 }, () => [3]);
```

```
28  const periodNotDated = ["0", "Not Dated"];
29  const periodYearly = ["0", "Years"];
30  const periodWithSeason = ["5", "Weeks-Work Days(5)"];
31
32  const runAnalyzeError = async (
33    variables: Variable[],
34    data: any[][],
35    diff: string[],
36    period: string[],
37    lag: number,
38    seasonal: boolean
39  ) => {
40    const { result } = renderHook(() =>
41      useAnalyzeHook(variables, data, diff, period, lag, seasonal, jest.fn())
42    );
43    await act(async () => {
44      await result.current.handleAnalyzes();
45    });
46    return result;
47  };
48
```

```
49  describe("useAnalyzeHook - Error Cases", () => {
50    it("TCA01: Returns error if variable is not selected", async () => {
51      const result = await runAnalyzeError([], data25Obs, ["level"], periodWithSeason, 12, true);
52      expect(result.current.errorMsg).toBe("Please select at least one variable.");
53    });
54
55    it("TCA02: Returns error if selected variable is not numeric", async () => {
56      const result = await runAnalyzeError([createMockVar("STRING")], data25Obs, ["level"], periodWithSeason, 12, true);
57      expect(result.current.errorMsg).toBe("Selected variable is not numeric");
58    });
59
60    it("TCA03: Returns error if data is empty", async () => {
61      const result = await runAnalyzeError([createMockVar()], [dataEmpty], ["level"], periodWithSeason, 12, true);
62      expect(result.current.errorMsg).toBe("No data available for the selected variable.");
63    });
64
65    it("TCA04: Returns error if data length is less than 20", async () => {
66      const result = await runAnalyzeError([createMockVar()], dataBelow20Obs, ["level"], periodWithSeason, 12, true);
67      expect(result.current.errorMsg).toBe("Data length must be at least 20 observations.");
68    });
69
70    it("TCA05: Returns error if lag without seasonality is out of range", async () => {
71      const result = await runAnalyzeError([createMockVar()], data25Obs, ["level"], periodWithSeason, 5, true);
72      expect(result.current.errorMsg).toBe("Lag length must be between 10 and 20.");
73    });
74
```

```
75    it("TCA09: Returns an error value if running without configuring the time settings", async () => {
76      const result = await runAnalyzeError([createMockVar()], data25Obs, ["level"], periodNotDated, 12, true);
77      expect(result.current.errorMsg).toBe("Please select another time specification.");
78    });
79
80    it("TCA10: Returns error if seasonality is selected but period has no periodicity", async () => {
81      const result = await runAnalyzeError([createMockVar()], data25Obs, ["level"], periodYearly, 12, true);
82      expect(result.current.errorMsg).toBe("Please select a time specification with periodicity.");
83    });
84
85    it("TCA11: Returns error if lag with seasonality is out of range", async () => {
86      const result = await runAnalyzeError([createMockVar()], data25Obs, ["level"], periodWithSeason, 5, true);
87      expect(result.current.errorMsg).toBe("Lag length must be between 10 and 20.");
88    });
89  });
```

```

92 const mockAddStatistic = jest.fn();
93 const mockHandleAutocorrelation = jest.fn();
94
95 jest.mock('@stores/useResultStore', () => ({
96   useResultStore: () => ({
97     addLog: jest.fn(() => Promise.resolve(1)),
98     addAnalytic: jest.fn(() => Promise.resolve(1)),
99     addStatistic: mockAddStatistic,
100   }),
101 }));
102
103 jest.mock('@components/Modals/Analyze/TimeSeries/Autocorrelation/analyze/analyze', () => ({
104   handleAutocorrelation: (...args: any[]) => mockHandleAutocorrelation(...args),
105 }));
106
107 const assertDescriptionStatistic = (expectedRows: { rowHeader: string[]; description: string }[]) => {
108   const call = mockAddStatistic.mock.calls.find(
109     ([, stat]) => stat?.title === 'Description Table'
110   );
111   expect(call).toBeDefined();
112   const parsed = JSON.parse(call[1].output_data);
113   const rows = parsed.tables[0].rows;
114   expectedRows.forEach((expectedRow) => {
115     expect(rows).toContainEqual(expectedRow);
116   });
117 };

```

```

119 describe('useAnalyzeHook - Success Case', () => {
120   beforeEach(() => {
121     mockAddStatistic.mockClear();
122     mockHandleAutocorrelation.mockImplementation(
123       (
124         {
125           _dataValues: number[],
126           dataHeader: string,
127           lag: number,
128           difference: string,
129           useSeasonal: boolean,
130           seasonal: number
131         }
132       ) => {
133         const seasonalDescription = useSeasonal ? 'with seasonal-differencing' : 'none';
134         const differenceDescription = difference === 'level' ? 'none' : difference;

```

```

134     const mockDescription = JSON.stringify({
135       tables: [
136         {
137           title: 'Description Table',
138           columnHeaders: [{ header: '' }, { header: 'description' }],
139           rows: [
140             { rowHeader: ['Name Method'], description: 'Autocorrelation' },
141             { rowHeader: ['Series Name'], description: dataHeader },
142             { rowHeader: ['Differencing'], description: differenceDescription },
143             { rowHeader: ['Seasonal Differencing'], description: seasonalDescription },
144             { rowHeader: ['Periodicity'], description: useSeasonal ? seasonal.toString() : '' },
145             { rowHeader: ['Maximum Lag'], description: lag.toString() },
146             { rowHeader: ['Approach Method of Standard Error'], description: 'Bartlett Formula' },
147             { rowHeader: ['Observations'], description: '25' },
148             { rowHeader: ['Number Observations of Computable First Lags'], description: '19' },
149           ],
150         },
151       ],
152     });
153     return Promise.resolve(['success', mockDescription, [], '', '', '', '']);
154   });
155 }
156 );
157 );

```

```

159 const successCases = [
160   {
161     id: 'TCAB6',
162     title: 'No differencing, no seasonal differencing',
163     diff: 'level',
164     diffLabel: 'Level',
165     seasonal: false,
166     period: ['5', 'Weeks-Work Days(5)'],
167     lag: 12,
168     expectedRows: [
169       { rowHeader: ['Name Method'], description: 'Autocorrelation' },
170       { rowHeader: ['Series Name'], description: 'Test Var' },
171       { rowHeader: ['Differencing'], description: 'none' },
172       { rowHeader: ['Seasonal Differencing'], description: 'none' },
173       { rowHeader: ['Periodicity'], description: '' },
174       { rowHeader: ['Maximum Lag'], description: '12' },
175       { rowHeader: ['Approach Method of Standard Error'], description: 'Bartlett Formula' },
176       { rowHeader: ['Observations'], description: '25' },
177       { rowHeader: ['Number Observations of Computable First Lags'], description: '19' },
178     ],
179   },

```



```

180 {
181   id: 'TCA07',
182   title: 'First differencing, no seasonal differencing',
183   diff: 'first-difference',
184   diffLabel: 'First Difference',
185   seasonal: false,
186   period: ['5', 'Weeks-Work Days(5)'],
187   lag: 12,
188   expectedRows: [
189     { rowHeader: ['Name Method'], description: 'Autocorrelation' },
190     { rowHeader: ['Series Name'], description: 'Test Var' },
191     { rowHeader: ['Differencing'], description: 'first-difference' },
192     { rowHeader: ['Seasonal Differencing'], description: 'none' },
193     { rowHeader: ['Periodicity'], description: '' },
194     { rowHeader: ['Maximum Lag'], description: '12' },
195     { rowHeader: ['Approach Method of Standard Error'], description: 'Bartlett Formula' },
196     { rowHeader: ['Observations'], description: '25' },
197     { rowHeader: ['Number Observations of Computable First Lags'], description: '19' },
198   ],
199 },

```

```

200 {
201   id: 'TCA08',
202   title: 'Second differencing, no seasonal differencing',
203   diff: 'second-difference',
204   diffLabel: 'Second Difference',
205   seasonal: false,
206   period: ['5', 'Weeks-Work Days(5)'],
207   lag: 12,
208   expectedRows: [
209     { rowHeader: ['Name Method'], description: 'Autocorrelation' },
210     { rowHeader: ['Series Name'], description: 'Test Var' },
211     { rowHeader: ['Differencing'], description: 'second-difference' },
212     { rowHeader: ['Seasonal Differencing'], description: 'none' },
213     { rowHeader: ['Periodicity'], description: '' },
214     { rowHeader: ['Maximum Lag'], description: '12' },
215     { rowHeader: ['Approach Method of Standard Error'], description: 'Bartlett Formula' },
216     { rowHeader: ['Observations'], description: '25' },
217     { rowHeader: ['Number Observations of Computable First Lags'], description: '19' },
218   ],
219 },

```

```

221 id: 'TCA12',
222 title: 'No differencing, with seasonal differencing',
223 diff: 'level',
224 diffLabel: 'Level',
225 seasonal: true,
226 period: ['5', 'Weeks-Work Days(5)'],
227 lag: 12,
228 expectedRows: [
229   { rowHeader: ['Name Method'], description: 'Autocorrelation' },
230   { rowHeader: ['Series Name'], description: 'Test Var' },
231   { rowHeader: ['Differencing'], description: 'none' },
232   { rowHeader: ['Seasonal Differencing'], description: 'with seasonal-differencing' },
233   { rowHeader: ['Periodicity'], description: '5' },
234   { rowHeader: ['Maximum Lag'], description: '12' },
235   { rowHeader: ['Approach Method of Standard Error'], description: 'Bartlett Formula' },
236   { rowHeader: ['Observations'], description: '25' },
237   { rowHeader: ['Number Observations of Computable First Lags'], description: '19' },
238 ],
239 },

```

```

241 id: 'TCA13',
242 title: 'First differencing, with seasonal differencing',
243 diff: 'first-difference',
244 diffLabel: 'First Difference',
245 seasonal: true,
246 period: ['5', 'Weeks-Work Days(5)'],
247 lag: 12,
248 expectedRows: [
249   { rowHeader: ['Name Method'], description: 'Autocorrelation' },
250   { rowHeader: ['Series Name'], description: 'Test Var' },
251   { rowHeader: ['Differencing'], description: 'first-difference' },
252   { rowHeader: ['Seasonal Differencing'], description: 'with seasonal-differencing' },
253   { rowHeader: ['Periodicity'], description: '5' },
254   { rowHeader: ['Maximum Lag'], description: '12' },
255   { rowHeader: ['Approach Method of Standard Error'], description: 'Bartlett Formula' },
256   { rowHeader: ['Observations'], description: '25' },
257   { rowHeader: ['Number Observations of Computable First Lags'], description: '19' },
258 ],
259 },

```

```

260 {
261   id: 'TCA14',
262   title: 'Second differencing, with seasonal differencing',
263   diff: 'second-difference',
264   diffLabel: 'Second Difference',
265   seasonal: true,
266   period: ['5', 'Weeks-Work Days(5)'],
267   lag: 12,
268   expectedRows: [
269     { rowHeader: ['Name Method'], description: 'Autocorrelation' },
270     { rowHeader: ['Series Name'], description: 'Test Var' },
271     { rowHeader: ['Differencing'], description: 'second-difference' },
272     { rowHeader: ['Seasonal Differencing'], description: 'with seasonal-differencing' },
273     { rowHeader: ['Periodicity'], description: '5' },
274     { rowHeader: ['Maximum Lag'], description: '12' },
275     { rowHeader: ['Approach Method of Standard Error'], description: 'Bartlett Formula' },
276     { rowHeader: ['Observations'], description: '25' },
277     { rowHeader: ['Number Observations of Computable First Lags'], description: '19' },
278   ],
279 },
280 ];

```

```

282 successCases.forEach(({ id, title, diff, diffLabel, seasonal, period, lag, expectedRows }) => {
283   it(`${id}: ${title}`, async () => {
284     const { result } = renderHook(() =>
285       useAnalyzeHook(
286         [createMockVar()],
287         data25Obs,
288         [diff, diffLabel],
289         period,
290         lag,
291         seasonal,
292         jest.fn()
293       )
294     );
295     await act(async () => {
296       await result.current.handleAnalyzes();
297     });
298     expect(result.current.errorMsg).toBe(null);
299     assertDescriptionStatistic(expectedRows);
300   });
301 });
302 });

```

Lampiran 12. Hasil *White-Box Testing* Menu *Autocorrelation*

```
PS C:\Users\guswa\Documents\Skrripsi\3.2. statify - GitLab Stis> npm test -- autocorrelation.test.ts
```

```

> statify-monorepo@1.0.0 test
> npm test --workspaces autocorrelation.test.ts

> statify-frontend@0.1.0 test
> jest --logHeapUsage autocorrelation.test.ts

```

```

⚠ Mismatching @next/swc version, detected: 15.3.5 while Next.js is on 15.3.4. Please ensure these match
PASS components/Modals/Analyze/TimeSeries/Autocorrelation/test/autocorrelation.test.ts (105 MB heap size)
useAnalyzeHook - Error Cases
  ✓ TCA01: Returns error if variable is not selected (20 ms)
  ✓ TCA02: Returns error if selected variable is not numeric (7 ms)
  ✓ TCA03: Returns error if data is empty (4 ms)
  ✓ TCA04: Returns error if data length is less than 20 (3 ms)
  ✓ TCA05: Returns error if lag without seasonality is out of range (3 ms)
  ✓ TCA09: Returns an error value if running without configuring the time settings (3 ms)
  ✓ TCA10: Returns error if seasonality is selected but period has no periodicity (3 ms)
  ✓ TCA11: Returns error if lag with seasonality is out of range (3 ms)

```

```

useAnalyzeHook - Success Case
  ✓ TCA06: No differencing, no seasonal differencing (9 ms)
  ✓ TCA07: First differencing, no seasonal differencing (5 ms)
  ✓ TCA08: Second differencing, no seasonal differencing (5 ms)
  ✓ TCA12: No differencing, with seasonal differencing (7 ms)
  ✓ TCA13: First differencing, with seasonal differencing (9 ms)
  ✓ TCA14: Second differencing, with seasonal differencing (4 ms)

Test Suites: 1 passed, 1 total
Tests:       14 passed, 14 total
Snapshots:   0 total
Time:        1.921 s, estimated 2 s
Ran all test suites matching autocorrelation.test.ts.

> statify-backend@1.0.0 test
> jest --passWithNoTests autocorrelation.test.ts

```

Lampiran 13. Kode Program *White-Box Testing Menu Unit Root Test*

```
1  /** @jest-environment jsdom */
2
3  import '@testing-library/jest-dom';
4  import { renderHook, act } from '@testing-library/react';
5  import { useAnalyzeHook } from '@components/Modals/Analyze/TimeSeries/UnitRootTest/hooks/analyzeHook';
6  import { Variable } from '@types/Variable';
7
8  const createMockVar = (type: "NUMERIC" | "STRING" = "NUMERIC"): Variable => ({
9    id: 1,
10    name: "Test Var",
11    columnIndex: 0,
12    type,
13    width: 8,
14    decimals: 2,
15    label: "",
16    values: [],
17    missing: null,
18    columns: 10,
19    align: "right",
20    measure: "scale",
21    role: "input",
22  });
23
24  const periodNotDated = ["0", "Not Dated"];
25  const periodMonthly = ["12", "Years-Months"];
26  const periodYearly = ["0", "Years"];
27  const periodWorkingDay = ["5", "Weeks-Work Days(5)"];
28
29  const dataEmpty: any[][] = [];
30  const dataBelow20Obs = Array.from({ length: 5 }, () => [3]);
31  const data250Obs = Array.from({ length: 25 }, () => [3]);
32  const data490Obs = Array.from({ length: 49 }, () => [3]);
33
34  const runAnalyzeError = async (
35    variables: Variable[],
36    data: any[],
37    selectedMethod: string[],
38    selectedEquation: string[],
39    selectedDifference: string[],
40    lagLength: number
41  ) => {
42    const { result } = renderHook(() =>
43      useAnalyzeHook(variables, data, selectedMethod, selectedEquation, selectedDifference, lagLength, jest.fn())
44    );
45    await act(async () => {
46      await result.current.handleAnalyzes();
47    });
48    return result;
49  };
50
51  describe("useAnalyzeHook - Error Cases", () => {
52    it("TCU01: Returns error if selected variable is empty", async () => {
53      const result = await runAnalyzeError([], data250Obs, ["dickey-fuller", ""], ["no_trend", ""], ["level", ""], 1);
54      expect(result.current.errorMsg).toBe("Please select at least one variable.");
55    });
56
57    it("TCU02: Returns error if selected variable is not numeric", async () => {
58      const result = await runAnalyzeError([createMockVar("STRING")], data250Obs, ["dickey-fuller", ""], ["no_trend", ""], ["level", ""], 1);
59      expect(result.current.errorMsg).toBe("Selected variable is not numeric");
60    });
61
62    it("TCU03: Returns error if selected variable data is empty", async () => {
63      const result = await runAnalyzeError([createMockVar()], [dataEmpty], ["dickey-fuller", ""], ["no_trend", ""], ["level", ""], 1);
64      expect(result.current.errorMsg).toBe("No data available for the selected variable.");
65    });
66
67    it("TCU04: Returns error if data observations are less than 20", async () => {
68      const result = await runAnalyzeError([createMockVar()], dataBelow20Obs, ["dickey-fuller", ""], ["no_trend", ""], ["level", ""], 1);
69      expect(result.current.errorMsg).toBe("Data length is less than 20 observations.");
70    });
71
72    it("TCU14: Returns error if lag length is out of range", async () => {
73      const result = await runAnalyzeError([createMockVar()], data250Obs, ["dickey-fuller", ""], ["no_trend", ""], ["level", ""], 11);
74      expect(result.current.errorMsg).toBe("Lag length must be between 1 and 10.");
75    });
76  });
```



```

78 const mockAddStatistic = jest.fn();
79 const mockHandleUnitRootTest = jest.fn();
80
81 jest.mock('@stores/useResultStore', () => ({
82   useResultStore: () => ({
83     addLog: jest.fn(() => Promise.resolve(1)),
84     addAnalytic: jest.fn(() => Promise.resolve(1)),
85     addStatistic: mockAddStatistic,
86   }),
87 }));
88
89 jest.mock('@components/Modals/Analyze/TimeSeries/UnitRootTest/analyze/analyze', () => ({
90   handleUnitRootTest: (...args: any[]) => mockHandleUnitRootTest(...args),
91 }));
92
93 const assertDescriptionStatistic = (expectedRows: { rowHeader: string[]; description: string }[]) => {
94   const call = mockAddStatistic.mock.calls.find(
95     ([, stat]) => stat?.title === "Description Table"
96   );
97   expect(call).toBeDefined();
98   const parsed = JSON.parse(call[1].output_data); // perbaikan di sini
99   const rows = parsed.tables[0].rows;
100   expectedRows.forEach((expectedRow) => {
101     expect(rows).toContainEqual(expectedRow);
102   });
103 };

```

```

105 describe("UseAnalyzeHook - Success Case", () => {
106   beforeEach(() => {
107     mockAddStatistic.mockClear();
108     mockHandleUnitRootTest.mockImplementation((
109       _dataValues,
110       name,
111       method,
112       lag,
113       equation,
114       difference
115     ) => {
116       const methodName = method === "dickey-fuller" ? "Dickey-Fuller" : "Augmented Dickey-Fuller";
117       const lagUsed = method === "dickey-fuller" ? 0 : lag;
118       const mockDescription = JSON.stringify({
119         tables: [
120           {
121             title: "Description Table",
122             columnHeaders: [{ header: "" }, { header: "description" }],
123             rows: [
124               { rowHeader: ["Name Method"], description: methodName },
125               { rowHeader: ["Series Name"], description: name },
126               { rowHeader: ["Equation"], description: equation },
127               { rowHeader: ["Number of Lags"], description: lagUsed.toString() },
128               { rowHeader: ["Differencing"], description: difference },
129               { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
130               { rowHeader: ["Observations"], description: "25" },
131               { rowHeader: ["Number Observations After Computing"], description: "20" },
132             ],
133           },
134         ],
135       });

```

```

134     },
135   );
136   return Promise.resolve([
137     "success", mockDescription, [], "df_stat", "coef_stat", "sel_crit", methodName,
138   ]);
139 });
140 });

```

```

143 const successCases = [
144   {
145     id: "TCU05",
146     method: "dickey-fuller",
147     methodLabel: "Dickey-Fuller",
148     equation: "no_constant",
149     equationLabel: "No Constant",
150     difference: "level",
151     differenceLabel: "level",
152     lag: 1,
153     expectedRows: [
154       { rowHeader: ["Name Method"], description: "Dickey-Fuller" },
155       { rowHeader: ["Series Name"], description: "Test Var" },
156       { rowHeader: ["Equation"], description: "no_constant" },
157       { rowHeader: ["Number of Lags"], description: "0" },
158       { rowHeader: ["Differencing"], description: "level" },
159       { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
160       { rowHeader: ["Observations"], description: "25" },
161       { rowHeader: ["Number Observations After Computing"], description: "20" },
162     ],
163   },

```

```

164 {
165   id: "TCU06",
166   method: "dickey-fuller",
167   methodLabel: "Dickey-Fuller",
168   equation: "no_trend",
169   equationLabel: "No Trend",
170   difference: "level",
171   differenceLabel: "level",
172   lag: 1,
173   expectedRows: [
174     { rowHeader: ["Name Method"], description: "Dickey-Fuller" },
175     { rowHeader: ["Series Name"], description: "Test Var" },
176     { rowHeader: ["Equation"], description: "no_trend" },
177     { rowHeader: ["Number of Lags"], description: "0" },
178     { rowHeader: ["Differencing"], description: "level" },
179     { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
180     { rowHeader: ["Observations"], description: "25" },
181     { rowHeader: ["Number Observations After Computing"], description: "20" },
182   ],
183 },

```

```

184 {
185   id: "TCU07",
186   method: "dickey-fuller",
187   methodLabel: "Dickey-Fuller",
188   equation: "with_trend",
189   equationLabel: "With Trend",
190   difference: "level",
191   differenceLabel: "level",
192   lag: 1,
193   expectedRows: [
194     { rowHeader: ["Name Method"], description: "Dickey-Fuller" },
195     { rowHeader: ["Series Name"], description: "Test Var" },
196     { rowHeader: ["Equation"], description: "with_trend" },
197     { rowHeader: ["Number of Lags"], description: "0" },
198     { rowHeader: ["Differencing"], description: "level" },
199     { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
200     { rowHeader: ["Observations"], description: "25" },
201     { rowHeader: ["Number Observations After Computing"], description: "20" },
202   ],
203 },

```

```

204 {
205   id: "TCU08",
206   method: "dickey-fuller",
207   methodLabel: "Dickey-Fuller",
208   equation: "no_constant",
209   equationLabel: "No Constant",
210   difference: "first-difference",
211   differenceLabel: "first-difference",
212   lag: 1,
213   expectedRows: [
214     { rowHeader: ["Name Method"], description: "Dickey-Fuller" },
215     { rowHeader: ["Series Name"], description: "Test Var" },
216     { rowHeader: ["Equation"], description: "no_constant" },
217     { rowHeader: ["Number of Lags"], description: "0" },
218     { rowHeader: ["Differencing"], description: "first-difference" },
219     { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
220     { rowHeader: ["Observations"], description: "25" },
221     { rowHeader: ["Number Observations After Computing"], description: "20" },
222   ],
223 },

```

```

224 {
225   id: "TCU09",
226   method: "dickey-fuller",
227   methodLabel: "Dickey-Fuller",
228   equation: "no_trend",
229   equationLabel: "No Trend",
230   difference: "first-difference",
231   differenceLabel: "first-difference",
232   lag: 1,
233   expectedRows: [
234     { rowHeader: ["Name Method"], description: "Dickey-Fuller" },
235     { rowHeader: ["Series Name"], description: "Test Var" },
236     { rowHeader: ["Equation"], description: "no_trend" },
237     { rowHeader: ["Number of Lags"], description: "0" },
238     { rowHeader: ["Differencing"], description: "first-difference" },
239     { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
240     { rowHeader: ["Observations"], description: "25" },
241     { rowHeader: ["Number Observations After Computing"], description: "20" },
242   ],
243 },

```

```

244 {
245     id: "TCU10",
246     method: "dickey-fuller",
247     methodLabel: "Dickey-Fuller",
248     equation: "with_trend",
249     equationLabel: "With Trend",
250     difference: "first-difference",
251     differenceLabel: "first-difference",
252     lag: 1,
253     expectedRows: [
254         { rowHeader: ["Name Method"], description: "Dickey-Fuller" },
255         { rowHeader: ["Series Name"], description: "Test Var" },
256         { rowHeader: ["Equation"], description: "with_trend" },
257         { rowHeader: ["Number of Lags"], description: "0" },
258         { rowHeader: ["Differencing"], description: "first-difference" },
259         { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
260         { rowHeader: ["Observations"], description: "25" },
261         { rowHeader: ["Number Observations After Computing"], description: "20" },
262     ],
263 },

```

```

264 {
265     id: "TCU11",
266     method: "dickey-fuller",
267     methodLabel: "Dickey-Fuller",
268     equation: "no_constant",
269     equationLabel: "No Constant",
270     difference: "second-difference",
271     differenceLabel: "second-difference",
272     lag: 1,
273     expectedRows: [
274         { rowHeader: ["Name Method"], description: "Dickey-Fuller" },
275         { rowHeader: ["Series Name"], description: "Test Var" },
276         { rowHeader: ["Equation"], description: "no_constant" },
277         { rowHeader: ["Number of Lags"], description: "0" },
278         { rowHeader: ["Differencing"], description: "second-difference" },
279         { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
280         { rowHeader: ["Observations"], description: "25" },
281         { rowHeader: ["Number Observations After Computing"], description: "20" },
282     ],
283 },

```

```

284 {
285     id: "TCU12",
286     method: "dickey-fuller",
287     methodLabel: "Dickey-Fuller",
288     equation: "no_trend",
289     equationLabel: "No Trend",
290     difference: "second-difference",
291     differenceLabel: "second-difference",
292     lag: 1,
293     expectedRows: [
294         { rowHeader: ["Name Method"], description: "Dickey-Fuller" },
295         { rowHeader: ["Series Name"], description: "Test Var" },
296         { rowHeader: ["Equation"], description: "no_trend" },
297         { rowHeader: ["Number of Lags"], description: "0" },
298         { rowHeader: ["Differencing"], description: "second-difference" },
299         { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
300         { rowHeader: ["Observations"], description: "25" },
301         { rowHeader: ["Number Observations After Computing"], description: "20" },
302     ],
303 },

```

```

304 {
305     id: "TCU13",
306     method: "dickey-fuller",
307     methodLabel: "Dickey-Fuller",
308     equation: "with_trend",
309     equationLabel: "With Trend",
310     difference: "second-difference",
311     differenceLabel: "second-difference",
312     lag: 1,
313     expectedRows: [
314         { rowHeader: ["Name Method"], description: "Dickey-Fuller" },
315         { rowHeader: ["Series Name"], description: "Test Var" },
316         { rowHeader: ["Equation"], description: "with_trend" },
317         { rowHeader: ["Number of Lags"], description: "0" },
318         { rowHeader: ["Differencing"], description: "second-difference" },
319         { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
320         { rowHeader: ["Observations"], description: "25" },
321         { rowHeader: ["Number Observations After Computing"], description: "20" },
322     ],
323 },

```

```

324 {
325   id: "TCU15",
326   method: "augmented-dickey-fuller",
327   methodLabel: "Augmented Dickey-Fuller",
328   equation: "no_constant",
329   equationLabel: "No Constant",
330   difference: "level",
331   differenceLabel: "level",
332   lag: 3,
333   expectedRows: [
334     { rowHeader: ["Name Method"], description: "Augmented Dickey-Fuller" },
335     { rowHeader: ["Series Name"], description: "Test Var" },
336     { rowHeader: ["Equation"], description: "no_constant" },
337     { rowHeader: ["Number of Lags"], description: "3" },
338     { rowHeader: ["Differencing"], description: "level" },
339     { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
340     { rowHeader: ["Observations"], description: "25" },
341     { rowHeader: ["Number Observations After Computing"], description: "20" },
342   ],
343 },

```

```

344 {
345   id: "TCU16",
346   method: "augmented-dickey-fuller",
347   methodLabel: "Augmented Dickey-Fuller",
348   equation: "no_trend",
349   equationLabel: "No Trend",
350   difference: "level",
351   differenceLabel: "level",
352   lag: 3,
353   expectedRows: [
354     { rowHeader: ["Name Method"], description: "Augmented Dickey-Fuller" },
355     { rowHeader: ["Series Name"], description: "Test Var" },
356     { rowHeader: ["Equation"], description: "no_trend" },
357     { rowHeader: ["Number of Lags"], description: "3" },
358     { rowHeader: ["Differencing"], description: "level" },
359     { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
360     { rowHeader: ["Observations"], description: "25" },
361     { rowHeader: ["Number Observations After Computing"], description: "20" },
362   ],
363 },

```

```

364 {
365   id: "TCU17",
366   method: "augmented-dickey-fuller",
367   methodLabel: "Augmented Dickey-Fuller",
368   equation: "with_trend",
369   equationLabel: "With Trend",
370   difference: "level",
371   differenceLabel: "level",
372   lag: 3,
373   expectedRows: [
374     { rowHeader: ["Name Method"], description: "Augmented Dickey-Fuller" },
375     { rowHeader: ["Series Name"], description: "Test Var" },
376     { rowHeader: ["Equation"], description: "with_trend" },
377     { rowHeader: ["Number of Lags"], description: "3" },
378     { rowHeader: ["Differencing"], description: "level" },
379     { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
380     { rowHeader: ["Observations"], description: "25" },
381     { rowHeader: ["Number Observations After Computing"], description: "20" },
382   ],
383 },

```

```

384 {
385   id: "TCU18",
386   method: "augmented-dickey-fuller",
387   methodLabel: "Augmented Dickey-Fuller",
388   equation: "no_constant",
389   equationLabel: "No Constant",
390   difference: "first-difference",
391   differenceLabel: "first-difference",
392   lag: 3,
393   expectedRows: [
394     { rowHeader: ["Name Method"], description: "Augmented Dickey-Fuller" },
395     { rowHeader: ["Series Name"], description: "Test Var" },
396     { rowHeader: ["Equation"], description: "no_constant" },
397     { rowHeader: ["Number of Lags"], description: "3" },
398     { rowHeader: ["Differencing"], description: "first-difference" },
399     { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
400     { rowHeader: ["Observations"], description: "25" },
401     { rowHeader: ["Number Observations After Computing"], description: "20" },
402   ],
403 },

```



```

404 {
405   id: "TCU19",
406   method: "augmented-dickey-fuller",
407   methodLabel: "Augmented Dickey-Fuller",
408   equation: "no_trend",
409   equationLabel: "No Trend",
410   difference: "first-difference",
411   differenceLabel: "first-difference",
412   lag: 3,
413   expectedRows: [
414     { rowHeader: ["Name Method"], description: "Augmented Dickey-Fuller" },
415     { rowHeader: ["Series Name"], description: "Test Var" },
416     { rowHeader: ["Equation"], description: "no_trend" },
417     { rowHeader: ["Number of Lags"], description: "3" },
418     { rowHeader: ["Differencing"], description: "first-difference" },
419     { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
420     { rowHeader: ["Observations"], description: "25" },
421     { rowHeader: ["Number Observations After Computing"], description: "20" },
422   ],
423 },

```

```

424 {
425   id: "TCU20",
426   method: "augmented-dickey-fuller",
427   methodLabel: "Augmented Dickey-Fuller",
428   equation: "with_trend",
429   equationLabel: "With Trend",
430   difference: "first-difference",
431   differenceLabel: "first-difference",
432   lag: 3,
433   expectedRows: [
434     { rowHeader: ["Name Method"], description: "Augmented Dickey-Fuller" },
435     { rowHeader: ["Series Name"], description: "Test Var" },
436     { rowHeader: ["Equation"], description: "with_trend" },
437     { rowHeader: ["Number of Lags"], description: "3" },
438     { rowHeader: ["Differencing"], description: "first-difference" },
439     { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
440     { rowHeader: ["Observations"], description: "25" },
441     { rowHeader: ["Number Observations After Computing"], description: "20" },
442   ],
443 },

```

```

445 {
446   id: "TCU21",
447   method: "augmented-dickey-fuller",
448   methodLabel: "Augmented Dickey-Fuller",
449   equation: "no_constant",
450   equationLabel: "No Constant",
451   difference: "second-difference",
452   differenceLabel: "second-difference",
453   lag: 3,
454   expectedRows: [
455     { rowHeader: ["Name Method"], description: "Augmented Dickey-Fuller" },
456     { rowHeader: ["Series Name"], description: "Test Var" },
457     { rowHeader: ["Equation"], description: "no_constant" },
458     { rowHeader: ["Number of Lags"], description: "3" },
459     { rowHeader: ["Differencing"], description: "second-difference" },
460     { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
461     { rowHeader: ["Observations"], description: "25" },
462     { rowHeader: ["Number Observations After Computing"], description: "20" },
463   ],
464 },

```

```

465 {
466   id: "TCU22",
467   method: "augmented-dickey-fuller",
468   methodLabel: "Augmented Dickey-Fuller",
469   equation: "no_trend",
470   equationLabel: "No Trend",
471   difference: "second-difference",
472   differenceLabel: "second-difference",
473   lag: 3,
474   expectedRows: [
475     { rowHeader: ["Name Method"], description: "Augmented Dickey-Fuller" },
476     { rowHeader: ["Series Name"], description: "Test Var" },
477     { rowHeader: ["Equation"], description: "no_trend" },
478     { rowHeader: ["Number of Lags"], description: "3" },
479     { rowHeader: ["Differencing"], description: "second-difference" },
480     { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
481     { rowHeader: ["Observations"], description: "25" },
482     { rowHeader: ["Number Observations After Computing"], description: "20" },
483   ],
484 },

```

```

484     id: "TCU23",
485     method: "augmented-dickey-fuller",
486     methodLabel: "Augmented Dickey-Fuller",
487     equation: "with_trend",
488     equationLabel: "With Trend",
489     difference: "second-difference",
490     differenceLabel: "second-difference",
491     lag: 3,
492     expectedRows: [
493       { rowHeader: ["Name Method"], description: "Augmented Dickey-Fuller" },
494       { rowHeader: ["Series Name"], description: "Test Var" },
495       { rowHeader: ["Equation"], description: "with_trend" },
496       { rowHeader: ["Number of Lags"], description: "3" },
497       { rowHeader: ["Differencing"], description: "second-difference" },
498       { rowHeader: ["Probability Value"], description: "Use MacKinnon (1996) one-sided p-values" },
499       { rowHeader: ["Observations"], description: "25" },
500       { rowHeader: ["Number Observations After Computing"], description: "20" },
501     ],
502   },
503 ],
504 };

```

```

506     successCases.forEach(
507       ({
508         id,
509         method,
510         methodLabel,
511         equation,
512         equationLabel,
513         difference,
514         differenceLabel,
515         lag,
516         expectedRows,
517       }) => {
518         it(`${id}: returns correct output for ${methodLabel} (${differenceLabel}, ${equationLabel})`, async () => {
519           const { result } = renderHook(() =>
520             useAnalyzeHook(
521               [createMockVar()],
522               data25Obs,
523               [method, methodLabel],
524               [equation, equationLabel],
525               [difference, differenceLabel],
526               lag,
527               jest.fn()
528             )
529           );

```

```

530         await act(async () => {
531           await result.current.handleAnalyzes();
532         });
533         expect(result.current.errorMessage).toBe(null);
534         assertDescriptionStatistic(expectedRows);
535       });
536     });
537   });
538 });
539

```

Lampiran 14. Hasil *White-Box Testing Menu Unit Root Test*

```

PS C:\Users\guswa\Documents\Skrripsi\3.2. statify - GitLab Stis> npm test -- unitroottest.test.ts

> statify-monorepo@1.0.0 test
> npm test --workspaces unitroottest.test.ts

> statify-frontend@0.1.0 test
> jest --logHeapUsage unitroottest.test.ts

```

```

PASS components/Modals/Analyze/TimeSeries/UnitRootTest/test/unitroottest.test.ts (113 MB heap size)
  useAnalyzeHook - Error Cases
    ✓ TCU01: Returns error if selected variable is empty (18 ms)
    ✓ TCU02: Returns error if selected variable is not numeric (5 ms)
    ✓ TCU03: Returns error if selected variable data is empty (5 ms)
    ✓ TCU04: Returns error if data observations are less than 20 (4 ms)
    ✓ TCU14: Returns error if lag length is out of range (4 ms)

```

```

useAnalyzeHook - Success Case
✓ TC085: returns correct output for Dickey-Fuller (level, No Constant) (48 ms)
✓ TC086: returns correct output for Dickey-Fuller (level, No Trend) (13 ms)
✓ TC087: returns correct output for Dickey-Fuller (level, With Trend) (10 ms)
✓ TC088: returns correct output for Dickey-Fuller (first-difference, No Constant) (9 ms)
✓ TC089: returns correct output for Dickey-Fuller (first-difference, No Trend) (31 ms)
✓ TC090: returns correct output for Dickey-Fuller (first-difference, With Trend) (10 ms)
✓ TC091: returns correct output for Dickey-Fuller (second-difference, No Constant) (21 ms)
✓ TC092: returns correct output for Dickey-Fuller (second-difference, No Trend) (11 ms)
✓ TC093: returns correct output for Dickey-Fuller (second-difference, With Trend) (8 ms)
✓ TC094: returns correct output for Augmented Dickey-Fuller (level, No Constant) (8 ms)
✓ TC095: returns correct output for Augmented Dickey-Fuller (level, No Trend) (7 ms)
✓ TC096: returns correct output for Augmented Dickey-Fuller (level, With Trend) (7 ms)
✓ TC097: returns correct output for Augmented Dickey-Fuller (first-difference, No Constant) (7 ms)
✓ TC098: returns correct output for Augmented Dickey-Fuller (first-difference, No Trend) (12 ms)
✓ TC099: returns correct output for Augmented Dickey-Fuller (first-difference, With Trend) (7 ms)
✓ TC100: returns correct output for Augmented Dickey-Fuller (second-difference, No Constant) (7 ms)
✓ TC101: returns correct output for Augmented Dickey-Fuller (second-difference, No Trend) (12 ms)
✓ TC102: returns correct output for Augmented Dickey-Fuller (second-difference, With Trend) (7 ms)

Test Suites: 1 passed, 1 total
Tests: 23 passed, 23 total
Snapshots: 0 total
Time: 1.89 s, estimated 2 s
Ran all test suites matching unitroottest.test.ts.

> statify-backend@1.0.0 test
> jest --passWithNoTests unitroottest.test.ts

```

Lampiran 15. Kode Program *White-Box Testing Menu Box-Jenkins Model*

```

1  /** @jest-environment jsdom */
2
3  import '@testing-library/jest-dom';
4  import { renderHook, act } from '@testing-library/react';
5  import { useAnalyzeHook } from '@components/Modals/Analyze/TimeSeries/BoxJenkinsModel/hooks/analyzeHook';
6  import { Variable } from '@types/Variable';
7
8  const createMockVar = (type: "NUMERIC" | "STRING" = "NUMERIC"): Variable => ({
9    id: 1,
10   name: "Test Var",
11   columnIndex: 0,
12   type,
13   width: 8,
14   decimals: 2,
15   label: "",
16   values: [],
17   missing: null,
18   columns: 10,
19   align: "right",
20   measure: "scale",
21   role: "input",
22 });
23
24 const periodNotDated = ["0", "Not Dated"];
25 const periodMonthly = ["12", "Years-Months"];
26 const periodWorkingDay = ["5", "Weeks-Work Days(5)"];
27
28 const dataEmpty: any[][] = [];
29 const dataBelow200bs = Array.from({ length: 5 }, () => [3]);
30 const data250bs = Array.from({ length: 25 }, () => [3]);
31
32 const runAnalyzeError = async (
33   storeVariables: Variable[],
34   data: any[][],
35   selectedPeriod: string[],
36   arOrder: number,
37   diffOrder: number,
38   maOrder: number,
39   checkedForecasting = false
40 ) => {
41   const { result } = renderHook(() =>
42     useAnalyzeHook(
43       storeVariables,
44       data,
45       selectedPeriod,
46       arOrder,
47       diffOrder,
48       maOrder,
49       checkedForecasting,
50       jest.fn()
51     )
52   );
53   await act(async () => {
54     await result.current.handleAnalyzes();
55   });
56   return result;
57 }

```

```

59 describe("useAnalyzeHook - Error Cases", () => {
60   it("TCB01: Returns error if variable is not selected", async () => {
61     const result = await runAnalyzeError([], dataEmpty, periodMonthly, 1, 1, 1);
62     expect(result.current.errorMsg).toBe('Please select at least one variable.');
```

```

85   it("TCB06: Returns error if AR order (p) out of range", async () => {
86     const result = await runAnalyzeError([createMockVar()], data250Obs, periodMonthly, 6, 1, 1);
87     expect(result.current.errorMsg).toBe('AR order must be between 0 and 5.');
```

```

101 const mockAddStatistic = jest.fn();
102 const mockHandleBoxJenkinsModel = jest.fn();
103
104 jest.mock('@stores/useResultStore', () => ({
105   useResultStore: () => ({
106     addLog: jest.fn(() => Promise.resolve(1)),
107     addAnalytic: jest.fn(() => Promise.resolve(1)),
108     addStatistic: mockAddStatistic,
109   }),
110 }));
111
112 jest.mock('@components/Modals/Analyze/TimeSeries/BoxJenkinsModel/analyze/analyze', () => ({
113   handleBoxJenkinsModel: (...args: any[]) => mockHandleBoxJenkinsModel(...args),
114 }));
115
116 const assertDescriptionStatistic = (expectedRows: { rowHeader: string[]; description: string }[]) => {
117   const call = mockAddStatistic.mock.calls.find(
118     ([, stat]) => stat?.title === "Description Table"
119   );
120   expect(call).toBeDefined();
121   const parsed = JSON.parse(call[1].output_data);
122   const rows = parsed.tables[0].rows;
123   expectedRows.forEach((expectedRow) => {
124     expect(rows).toContainEqual(expectedRow);
125   });
126 };
127
```



```

128 const assertStatisticTitles = (expectedTitles: string[]) => {
129   const calls = mockAddStatistic.mock.calls.map((call) => call[1]?.title);
130   expect(calls).toEqual(expect.arrayContaining(expectedTitles));
131 };
132
133 describe('useAnalyzeHook - Success Cases', () => {
134   beforeEach(() => {
135     mockAddStatistic.mockClear();
136     mockHandleBoxJenkinsModel.mockImplementation((
137       dataValues,
138       name,
139       orderParams,
140       forecasting
141     ) => {
142       const mockDescription = JSON.stringify({
143         tables: [
144           {
145             title: 'Description Table',
146             columnHeaders: [{ header: '' }, { header: 'description' }],
147             rows: [
148               { rowHeader: ['Name Method'], description: 'ARIMA (${orderParams.join(', ')}' }},
149               { rowHeader: ['Estimation Method'], description: 'Conditional Least Squares (CLS)' },
150               { rowHeader: ['Function Estimation'], description: 'Conditional Sum of Squares (CSS)' },
151               { rowHeader: ['Optimization Method'], description: 'L-BFGS' },
152               { rowHeader: ['Series Name'], description: name },
153               { rowHeader: ['Series Period'], description: '2024-01-01 - 2024-01-25' },
154               { rowHeader: ['Series Length'], description: dataValues.length.toString() },
155             ],
156           },
157         ],
158       });

```

```

160       const mockCoefficient = JSON.stringify({
161         tables: [
162           {
163             title: 'Coefficients Test for ARIMA (${orderParams.join(',')}',
164             columnHeaders: [
165               { header: '' },
166               { header: 'coef' },
167               { header: 'std. error' },
168               { header: 't value' },
169               { header: 'p-value' },
170             ],
171             rows: [
172               {
173                 rowHeader: ['Constant'],
174                 coef: '1.200',
175                 'std. error': '0.100',
176                 't value': '12.000',
177                 'p-value': '0.000',
178               },
179             ],
180           },
181         ],
182       });

```

```

184       const mockCriteria = JSON.stringify({
185         tables: [
186           {
187             title: 'Selection Criteria for Test Var',
188             columnHeaders: [{ header: '' }, { header: 'value' }],
189             rows: [
190               { rowHeader: ['Akaike Info Crit'], value: '123.456' },
191             ],
192           },
193         ],
194       });
195
196       const mockEvaluation = JSON.stringify({
197         tables: [
198           {
199             title: 'Arima Forecasting Evaluation',
200             columnHeaders: [{ header: '' }, { header: 'value' }],
201             rows: [{ rowHeader: ['RMSE'], value: '2.345' }],
202           },
203         ],
204       });

```

```

206     const mockGraphic = JSON.stringify({
207       charts: [
208         {
209           chartType: 'Multiple Line Chart',
210           chartMetadata: { title: 'ARIMA Forecasting of Test Var' },
211           chartData: [],
212         },
213       ],
214     });
215
216     const forecast = Array.from({ length: 25 }, () => 4.56);
217     const testArray = [1.2, 2.3, 3.4];
218
219     return Promise.resolve([
220       'success',
221       mockDescription,
222       testArray,
223       mockCoefficient,
224       mockCriteria,
225       mockEvaluation,
226       forecast,
227       mockGraphic,
228     ]);
229   });
230 }

```

```

232 const successCases = [
233   {
234     id: 'TCB09',
235     modelLabel: 'ARIMA(1,1,1)',
236     p: 1,
237     d: 1,
238     q: 1,
239     enableForecast: false,
240     expectedTitles: [
241       'Description Table',
242       'Coefficient Test for ARIMA(1,1,1)',
243       'Criteria Selection for ARIMA(1,1,1)',
244     ],
245     expectedRows: [
246       { rowHeader: ['Name Method'], description: 'ARIMA (1, 1, 1)' },
247       { rowHeader: ['Estimation Method'], description: 'Conditional Least Squares (CLS)' },
248       { rowHeader: ['Function Estimation'], description: 'Conditional Sum of Squares (CSS)' },
249       { rowHeader: ['Optimization Method'], description: 'L-BFGS' },
250       { rowHeader: ['Series Name'], description: 'Test Var' },
251       { rowHeader: ['Series Period'], description: '2024-01-01 - 2024-01-25' },
252       { rowHeader: ['Series Length'], description: '25' },
253     ],
254   },
255 ];

```

```

257 successCases.forEach(({ id, modelLabel, p, d, q, enableForecast, expectedTitles, expectedRows }) => {
258   it(`${id}: Success with ${modelLabel}`, async () => {
259     const { result } = renderHook(() => {
260       useAnalyzeHook(
261         [createMockVar()],
262         data25Obs,
263         periodWorkingDay,
264         p, d, q,
265         enableForecast,
266         jest.fn()
267       );
268     });
269
270     await act(async () => {
271       await result.current.handleAnalyzes();
272     });
273
274     expect(result.current.errorMsg).toBe(null);
275     assertStatisticTitles(expectedTitles);
276     assertDescriptionStatistic(expectedRows);
277   });
278 });
279 }

```

Lampiran 16. Hasil *White-Box Testing Menu Box-Jenkins Model*

```

PS C:\Users\guswa\Documents\Skrpsi\3.2. statify - GitLab Stis> npm test -- boxjenkinsmodel.test.ts
> statify-monorepo@1.0.0 test
> npm test --workspaces boxjenkinsmodel.test.ts
> statify-frontend@0.1.0 test
> jest --logHeapUsage boxjenkinsmodel.test.ts

```

```

▲ Mismatching @next/swc version, detected: 15.3.5 while Next.js is on 15.3.4. Please ensure these match
PASS components/Models/Analyze/TimeSeries/BoxJenkinsModel/test/boxjenkinsmodel.test.ts (112 MB heap size)
useAnalyzeHook - Error Cases
  ✓ TCB01: Returns error if variable is not selected (21 ms)
  ✓ TCB02: Returns error if selected variable is not numeric (9 ms)
  ✓ TCB03: Returns error if data is empty (5 ms)
  ✓ TCB04: Returns error if data length is less than 20 (3 ms)
  ✓ TCB05: Returns error if time is not configured (3 ms)
  ✓ TCB06: Returns error if AR order (p) out of range (2 ms)
  ✓ TCB07: Returns error if differencing order out of range (3 ms)
  ✓ TCB08: Returns error if MA order (q) out of range (2 ms)
useAnalyzeHook - Success Cases
  ✓ TCB09: Success with ARIMA(1,1,1) (9 ms)

Test Suites: 1 passed, 1 total
Tests: 9 passed, 9 total
Snapshots: 0 total
Time: 1.994 s, estimated 3 s
Ran all test suites matching boxjenkinsmodel.test.ts.

> statify-backend@1.0.0 test
> jest --passWithNoTests boxjenkinsmodel.test.ts

```

Lampiran 17. Surat Permohonan Lisensi SUMI



POLITEKNIK STATISTIKA STIS
 Jl. Otto Iskandardinata No.64 C, Jakarta 13330
 Telp. (021) 8508812, 8191437, Fax. 8197577
 Website: www.stis.ac.id, Email: info@stis.ac.id

To:

Mr. Jurek Kirakowski
 Director of User eXPerience solutions
 Ireland

Dear Mr. Jurek

With this letter, I would like to inform you, I, the undersigned:

Name : Ibnu Santoso
 Position : Head of Statistical Computing Department

Certify that,

Name : Guswana Adventus
 Student's ID : 222112082
 Major : Statistical Computing
 Interest : Statistical Information System

is really a student at STIS Polytechnic of Statistics that performs research of system development leading to bachelor's degree completion. I guarantee that the student is not working in a consultancy relationship with any commercial interest as far as their use of SUMI is concerned, and that I will help the student to take reasonable steps to protect the intellectual property rights and copyright of SUMI.

Jakarta, 14th July 2025
 Head of Statistical Computing Department


 Ibnu Santoso

Lampiran 18. Kuesioner SUMI

<i>Statements 1 - 10 of 50.</i>	Agree	Undecided	Disagree
This software responds too slowly to inputs.	☐	☐	☐
I would recommend this software to my colleagues.	☐	☐	☐
The instructions and prompts are helpful.	☐	☐	☐
This software has at some time stopped unexpectedly.	☐	☐	☐
Learning to operate this software initially is full of problems.	☐	☐	☐
I sometimes don't know what to do next with this software.	☐	☐	☐
I enjoy the time I spend using this software.	☐	☐	☐
I find that the help information given by this software is not very useful.	☐	☐	☐
If this software stops it is not easy to restart it.	☐	☐	☐
It takes too long to learn the software functions.	☐	☐	☐

<i>Statements 11 - 20 of 50.</i>	Agree	Undecided	Disagree
I sometimes wonder if I am using the right function.	☐	☐	☐
Working with this software is satisfying.	☐	☐	☐
The way that system information is presented is clear and understandable.	☐	☐	☐
I feel safer if I use only a few familiar functions.	☐	☐	☐
The software documentation is very informative.	☐	☐	☐
This software seems to disrupt the way I normally like to arrange my work.	☐	☐	☐
Working with this software is mentally stimulating.	☐	☐	☐
There is never enough information on the screen when it's needed.	☐	☐	☐
I feel in command of this software when I am using it.	☐	☐	☐
I prefer to stick to the functions that I know best.	☐	☐	☐

<i>Statements 21 - 30 of 50.</i>	Agree	Undecided	Disagree
I think this software is inconsistent.	☐	☐	☐
I would not like to use this software every day.	☐	☐	☐
I can understand and act on the information provided by this software.	☐	☐	☐
This software is awkward when I want to do something which is not standard.	☐	☐	☐
There is too much to read before you can use the software.	☐	☐	☐
Tasks can be performed in a straight forward manner using this software.	☐	☐	☐
Using this software is frustrating.	☐	☐	☐
The software has helped me overcome any problems I have had in using it.	☐	☐	☐
The speed of this software is fast enough.	☐	☐	☐
I keep having to go back to look at the guides.	☐	☐	☐

<i>Statements 31 - 40 of 50.</i>	Agree	Undecided	Disagree
It is obvious that user needs have been fully taken into consideration.	☐	☐	☐
There have been times in using this software when I have felt quite tense.	☐	☐	☐
The organisation of the menus seems quite logical.	☐	☐	☐
The software allows the user to be economic of keystrokes.	☐	☐	☐
Learning how to use new functions is difficult.	☐	☐	☐
There are too many steps required to get something to work.	☐	☐	☐
I think this software has sometimes given me a headache.	☐	☐	☐
Error messages are not adequate.	☐	☐	☐
It is easy to make the software do exactly what you want.	☐	☐	☐
I will never learn to use all that is offered in this software.	☐	☐	☐

Statements 41 - 50 of 50.

	Agree	Undecided	Disagree
The software hasn't always done what I was expecting.	☐	☐	☐
The software presents itself in a very attractive way.	☐	☐	☐
Either the amount or quality of the help information varies across the system.	☐	☐	☐
It is relatively easy to move from one part of a task to another.	☐	☐	☐
It is easy to forget how to do things with this software.	☐	☐	☐
This software occasionally behaves in a way which can't be understood.	☐	☐	☐
This software is really very awkward.	☐	☐	☐
It is easy to see at a glance what the options are at each stage.	☐	☐	☐
Getting data files in and out of the system is not easy.	☐	☐	☐
I have to look for assistance most times when I use this software.	☐	☐	☐

Lampiran 19. Tingkat Kesiapan Teknologi (TKT) Statify



POLITEKNIK STATISTIKA STIS

Jl. Otto Iskandardinata No.64C Jakarta 13330
Telp. (021) 8508812, 8191437, Fax. (021) 81975777
Website: ww.stis.ac.id; Email: info@stis.ac.id

SURAT PERNYATAAN

PENERIMAAN PRODUK PENELITIAN POLITEKNIK STATISTIKA STIS

Yang bertanda tangan di bawah ini:

Nama : Eko Putra Wahyudin, S.Tr.Stat.
Jabatan : Pranata Komputer Pertama
Nama Institusi / Lembaga : Politeknik Statistika STIS
Alamat : Jl. Otto Iskandardinata No.64C 1, RT.1/RW.4, Bidara Cina,
Jatinegara, Jakarta Timur, DKI Jakarta

Telah menerima produk penelitian dari Politeknik Statistika STIS atas nama:

Nama : Himawan Wahid Ikhwansyah, Muhammad Anja Taufani,
Guswana Adventus, Umar Hadi Pranoto, Muhamad Iqbal Putra
Pratama, Naufal putra Ihsan Marlin
NIM/NIP : 222112094, 222112205, 222112082, 222112404, 222112197,
222112246
Jenis Produk : Aplikasi Web
Judul Produk : Aplikasi Analisis Statistik (Statify)

Pada tanggal 14 Agustus 2025 di Politeknik Statistika STIS.

Dengan status tingkat kesiapterapan teknologi (TKT) sebagaimana terlampir.

Demikian pernyataan ini dibuat untuk dapat digunakan sebagai dokumentasi hasil pengembangan penelitian di Politeknik Statistika STIS.

Jakarta, 14 Agustus 2025

Yang Menyatakan,
Eko Putra Wahyudin, S.Tr.Stat.
Pranata Komputer Pertama



POLITEKNIK STATISTIKA STIS

Jl. Otto Iskandardinata No.64C Jakarta 13330
Telp. (021) 8508812, 8191437, Fax. (021) 81975777
Website: www.stis.ac.id; Email: info@stis.ac.id

Lampiran

Lembar Isian Tingkat Kesiapterapan Teknologi (TKT)

TKT	Deskripsi	Capaian TKT*
TKT 1	Prinsip dasar dari suatu teknologi telah diteliti	Tercapai / Belum Tercapai
TKT 2	Konsep teknologi dan aplikasi telah di formulasikan	Tercapai / Belum Tercapai
TKT 3	Konsep dan karakteristik teknologi telah dibuktikan secara analitis dan eksperimental	Tercapai / Belum Tercapai
TKT 4	Komponen teknologi telah divalidasi dalam lingkungan laboratorium	Tercapai / Belum Tercapai
TKT 5	Komponen teknologi telah divalidasi dalam lingkungan yang relevan	Tercapai / Belum Tercapai
TKT 6	Model atau Purwarupa telah diuji dalam lingkungan yang relevan	Tercapai / Belum Tercapai
TKT 7	Purwarupa telah diuji dalam lingkungan sebenarnya	Tercapai / Belum Tercapai
TKT 8	Sistem teknologi telah lengkap dan memenuhi syarat (<i>qualified</i>)	Tercapai / Belum Tercapai
TKT 9	Teknologi benar-benar teruji/terbukti melalui keberhasilan pengoperasian	Tercapai / Belum Tercapai

*) Coret yang tidak sesuai

Acuan Dasar Hukum:

1. Peraturan Menteri Riset, Teknologi, dan Pendidikan Tinggi No.42/2016
2. Panduan Penelitian dan Pengabdian kepada Masyarakat Edisi XII, Direktorat Jenderal Penguatan Riset dan Pengembangan, Kementerian Riset, Teknologi, dan Pendidikan Tinggi Tahun 2018

RIWAYAT HIDUP

Penulis bernama Guswana Adventus Angkatan 63, lahir di Jakarta pada tanggal 2 Desember 2001. Penulis merupakan anak laki-laki dari pasangan Bapak Halim Simarmata dan Ibu Utari Debora dan merupakan anak pertama dari dua bersaudara. Penulis tinggal di Kota Jakarta hingga saat ini. Penulis mulai mengenyam pendidikan dasar di SD Strada Santo Aloysius 1 Tangerang mulai tahun 2008 hingga lulus tahun 2014. Pada tahun 2017, penulis menyelesaikan pendidikan menengah pertama di SMP Swasta Katolik Assisi Medan. Pada tahun yang sama penulis melanjutkan sekolah di SMA Negeri 15 Medan. Pada tahun 2021, penulis diterima untuk mengikuti pendidikan di Politeknik Statistika STIS.

Akhirnya pada tahun 2025, atas izin Tuhan Yang Maha Esa, dukungan dan doa dari keluarga dan teman, penulis berhasil menyelesaikan pendidikan Program Diploma IV di Politeknik Statistika STIS.